

BIPN 145 Digital Manual

Contents

Course Information

- Syllabus for SSII (2026)
- Course schedule for SSII

Lab Manual

- Modeling Neural Membranes
- String Nervous Systems
- Earthworm Electrophysiology
- *Drosophila* Behavior & Optogenetics
- Measuring Alpha Wave Activity Using EEG

Tools & Data Analysis

- Working with LabChart Files
- Visualizing Data in Python

Final Projects

- Final Projects

Welcome to our course! 🐍

This website contains the syllabus, lab protocols, and Python code to help you complete your experiments for Neurobiology Lab (BIPN 145) at UC San Diego.

Most of the pages in this manual contain written or video guides to completing experiments in BIPN 145. Other pages are interactive coding notebooks, which you can use to analyze and visualize your data.

Lab Manual

This website contains a full online version of our lab manual. You can also [download the full lab manual as a PDF](#). You're encouraged to print this manual so that you have a hard copy in lab with you.

Saving a page as a PDF

Each page on this site can be downloaded as a PDF for offline use or printing.

1. Navigate to the page you want to save.
2. Click the **download icon** (↓) in the top-right toolbar.
3. Select **".pdf"** from the dropdown.
4. Your browser's print dialog will open. Set the following options:
 - **Destination:** Save as PDF
 - **Paper size:** Letter
 - *Optional* Under **More settings**, uncheck **Headers and footers** to remove the URL and date from the printed page.
5. Click **Save**.

Interactive coding notebooks

To interact with the coding tutorials presented here, navigate to the page with the tutorial (e.g., "Importing Labchart (Python)"). On these pages, you'll see a rocket 🚀 icon in the top right corner. Click on this, and choose Colab. This will launch Google Colab, where you can run the code in the notebook.

Alternatively, you can copy and paste the code you see here into a Python editor of your choice.

```
# Code blocks look like this!  
# If you hover, there's an option to copy the contents.
```

Note that only notebooks with code in them will have the option to open them in Colab.

Syllabus for SSII (2026)

Welcome to BIPN 145! This is a course about how we study the nervous system. You'll be asked to think, plan, and write like a neuroscientist.

You may be surprised about some of the topics in this course. You'll spend a lot of time learning about nervous systems, but we'll also spend some time talking about experimental design, technology, ethics, and analyzing big data sets.

Instructional Team

Role	Name	Email
Instructor	Ashley Juavinett	ajuavine@ucsd.edu
Teaching Assistant	Catherine Ngoc Phan	cnp005@ucsd.edu
Reader	Julianne Lee	jul144@ucsd.edu
Staff Research Associate	Breonna Gonzalez	—

Office hours: *By appointment*

Lecture: TuWThF 11:00a–12:20p in MALK B90

Lab: TuWThF 1:00p–4:20p in YORK 1310

Course learning objectives

- **Collect** and **evaluate** neural data from various organisms
- **Apply** principles of neural communication to multiple model systems
- **Describe** the breadth of techniques in neuroscience and the experimental questions they are suited to answer
- **Develop** an appreciation for and practical insight into the process of research
- **Communicate** research to peers as well as a broader audience

Attendance policies

You are expected to attend our in-person lab sessions, but lecture attendance is not mandatory. However, as you soon will see, even our lecture sessions will not be unidirectional — these will be active learning sessions where we co-create our learning. These lecture sessions will be podcasted/recorded, but you will be expected to make up any in-class activities that you missed.

If you cannot attend a lab session due to illness or for any other reason, **please contact both Dr. J and our teaching assistant(s) ASAP so that we can work with you on accommodations.**

Enrolled and waitlisted students must attend the first lab session. Additional details: <http://biology.ucsd.edu/go/ug-labs>. You do *not* need to inform us if you will be missing a lecture session.

Additional resources

There is a list of resources listed [here](#) to help you thrive in this course and on campus. If there is anything you think we can help you out with, please feel free to reach out to the TA or Dr. J.

Grading

- **QUIZZES & ASSIGNMENTS** (350 pts, 20–50 pts each) — Includes quizzes for individual lab activities and pre-lab quizzes.

- **EARTHWORM REPORT** (125 pts) — Your first lab report will summarize our earthworm experiments.
- **FINAL PROJECT** (250 pts) — Project proposal, presentation, and written report.
- **MIDTERMS** (250 pts, 125 pts each)
- **PROFESSIONALISM** (25 pts)

Late assignments

- For **individual assignments only**, you have a 3-day late bank that you may use over the course of the session to extend your deadlines without penalty.
- You can request a late bank **before the deadline** by filling out this form: <https://forms.gle/SAeRJKyCWRtdhbFx7>.
- Assignments not protected by the late bank will lose -10% for each day they are late.
- In the case of extenuating circumstances requiring a submission more than three days late, please contact Dr. J directly.

Additional notes on grading

- Final scores will be converted to letter grades, where A = 90–100%, B = 80–89.99%, C = 70–79.99%, D = 60–69.99%, and F = 0–59.99%. For plus and minus grades, A+ = 97–100, A = 93–96.99, A- = 90–92.99, B+ = 87–89.99, B = 83–86.99, B- = 80–82.99, and so on.
- Final scores are as you see them on Canvas, once all of your assignments are graded. There is no rounding up to the closest score.

Please note that add/drop deadlines are different for lab courses than lecture courses.

Students who drop a Biology lab class after the end of the second class meeting will be assigned a "W". Additional details: <http://biology.ucsd.edu/go/ug-labs>. Summer Session II enrollment deadlines: **drop without a "W" by August 14, 2026**, and **drop with a "W" by August 21, 2026** ([full calendar](#)).

Course philosophy

A note on our course's environment

We'll be working together to create an equitable and inclusive environment of mutual respect, in which we all feel comfortable to share our moments of confusion, ask questions, and challenge our understanding. Everyone should be able to succeed in this course. If you do not feel that is the case please let me know.

To help accomplish this:

- I'll ask for your preferred name & pronouns on our incoming survey. If these change over the course of the session, please let me know.
- Please don't hesitate to come and talk with me if you feel like your performance in the class is being impacted by your experiences outside of class.
- I am constantly learning about diverse perspectives and identities. If something was said in class (by anyone) that made you feel uncomfortable, please talk to me about it.
- As a participant, you should also strive to honor the diversity of your classmates.

On the equity & diversity of our course content

Despite the way we typically conceive of the scientific process, much of science is *subjective* and is historically built on a small subset of privileged voices. In this class, we will make an effort to show the work of diverse scientists, but limits still exist on this diversity. I acknowledge that it is possible that there may be both overt and covert biases in the material due to the lens with which

it was written, even though the material is primarily of a scientific nature. Integrating a diverse set of experiences is important for a more comprehensive understanding of science. To this end, we will discuss diversity in neuroscience as part of the course from time to time.

Course accommodations

If you need accommodations for this course due to a disability, please contact the Office for Students with Disabilities (osd@ucsd.edu) for an Authorization for Accommodation letter. Please speak with me in the first week of class if you intend to apply for accommodations. For more information, visit <http://disabilities.ucsd.edu>.

This course, and the work it entails, is for *you*

So, you won't benefit if others (or a generative AI) do your work. Cases of academic dishonesty or cheating will be first handled by me, and then by the Academic Integrity Office. If you become aware of cheating in this class, you can anonymously report it: <https://academicintegrity.ucsd.edu/>

Lab safety is important

Enrolled and waitlisted students must successfully complete the Biology Lab Safety Training and Assessment before the first lab session: <https://biolabclass-safetyquiz.ucsd.edu/introduction>. **You will *not* be allowed into the lab for the second in-person lab session unless you have successfully passed the safety assessment.** Note that you **do not need to bring your own personal protective equipment** (e.g. a lab coat or goggles) for this course. We will provide gloves when needed.

Course management & texts

Lab manual

The lab manual is contained on the [same website as this syllabus](#).

Canvas

This course will be using Canvas to manage content and grades. You can log in by going to <http://canvas.ucsd.edu>. If you need any technical assistance with Canvas, please alert your instructor and send an email to servicedesk@ucsd.edu.

Textbook

There is no mandated textbook for this course, but most of the background material can be found in Purves et al. (2018) *Neuroscience*. We'll also use Carter & Shieh (2015) *Guide to Research Techniques in Neuroscience*, which can be found online [here](#) (link is also under *Resources* on Canvas). In addition, for each module I have curated resources that will be useful to you. You can find these on Canvas, or on the [course website](#).

Software for this class

It will be helpful to have the following software on your computer, since you'll often need to rely on a personal computer. If you have any issues with these or would prefer not to download anything onto your personal computer, you are welcome to rely on a teammate or reach out to us for additional accommodations. If you need a laptop for the session, [you can request one](#).

Microsoft Office

It may be useful to have Microsoft Office in this course. You can find it [here](#).

LabChart Reader

If you can, please download [LabChart Reader](#) on your personal computer. We'll be using this to analyze data you collect.

Course schedule for SSII

SUBJECT TO CHANGE — You can find readings & due dates for assignments on Canvas. Lecture meets 11:00a–12:20p in MALK B90; lab meets 1:00p–4:20p in YORK 1310.

Date	Lecture	Lab	Due / Reading
Week 1			
Tue Aug 4	Introduction to BIPN 145 & nervous systems	Safety talk & lab group formation	Take the incoming survey (link on Canvas); take the online safety test
Wed Aug 5	The passive membrane & modeling neural activity	Lab #1: Modeling Neural Membranes <i>in silico</i>	READ the protocol and WATCH the video tutorial
Thu Aug 6	Recording from the nervous system & coding in BIPN 145	Lab #2: String Lab	DUE Modeling Neural Membranes Quiz; Extracellular Pre-Lab Quiz
Fri Aug 7	Earthworm nervous systems; the action potential	Lab #3: Earthworm Experiments	READ Hodgkin & Huxley (1939); WATCH/READ Exporting data from LabChart & Experiment #2/#3 protocols; DUE String Lab (<i>Group Assignment</i>)
Week 2			
Tue Aug 11	Writing lab reports & statistics	Lab #3: Earthworm Experiments (continued)	
Wed Aug 12	Two-sample statistics; the speed of the nervous system	Lab #4: Intracellular Equipment	DUE Intracellular Pre-Lab Quiz
Thu Aug 13	Leech physiology & cell types; intracellular & patch clamp recording	Lab #4: Recording from the Retzius Cell of the Leech	
Fri Aug 14 (drop w/o "W" deadline)	Review for Exam #1	Lab #4: Filling a cell in the leech	
Week 3			
Tue Aug 18	Exam #1	Analysis & catch-up day	DUE Earthworm Lab Report
Wed Aug 19	How is science funded? & introduction to final projects	Final project brainstorm;	DUE Leech Lab (<i>Group Assignment</i>)

Date	Lecture	Lab	Due / Reading
		equipment & needs	
Thu Aug 20	Ethology & behavior; <i>Drosophila</i> behavior	Lab #7: The Case of the Mislabeled Vials	READ your Drosophila behavior handout
Fri Aug 21 (drop w/ "W" deadline)	<i>Drosophila</i> genetics & optogenetics	Lab #7: The Case of the Missing Methods	DUE Project proposals
Week 4			
Tue Aug 25	Electroencephalography (EEG)	Lab #8: EEG	DUE <i>Drosophila</i> optogenetics methods & results write-up
Wed Aug 26	EEG analysis	Lab #8: EEG (analysis)	DUE (Wed @ 11:59 pm) <i>Drosophila</i> Presentation slides
Thu Aug 27	<i>Drosophila</i> Presentations	Allen Mouse Brain Connectivity	DUE EEG Quiz & Results; Submit Final Project Equipment & Needs [Spreadsheet]
Fri Aug 28	Visualizing the nervous system; mapping neural circuits & two-photon calcium imaging	Visual perception in a mouse (Allen Observatory)	DUE Allen Connectivity Lab
Week 5			
Tue Sep 1	Careers in neuroscience	Work on final projects	DUE Allen Observatory Lab
Wed Sep 2	Review for Exam #2	Work on final projects	
Thu Sep 3	Final project workshop / open Q&A	Work on final projects	
Fri Sep 4	Final project presentations	Final project presentations (<i>in MALK B90</i>)	
Finals			
Sat Sep 5	Exam #2 — 11:30a–2:29p, MALK B90	—	DUE Final project lab reports

Modeling Neural Membranes

In this lab, we'll use an online membrane simulator and a breadboard to model the properties of neural membranes and passive potentials.

Neuromembrane

Neuromembrane was made by a team of University of Alberta researchers and a company known as Atmist. Many thanks to Christelle Sabatier, Michael Wright, and Declan Ali for sharing similar lesson plans. Their ideas are integrated here.

Part I: Resting membrane potential

First, we'll explore the properties of the membrane that lead to a resting membrane potential. By the end of this part, you will be able to:

- Explain how a negative resting membrane potential is maintained
- Identify the factors that determine and maintain resting membrane potential
- Use the Goldman-Hodgkin-Katz to calculate the membrane potential given the concentrations and permeabilities of K^+ and Na^+
- Generate simulation data to illustrate the dependence of the resting membrane potential on K^+

Steps

1. Go to <https://neuromembrane.biology.ualberta.ca/> and log in as a guest.
2. To open the resting membrane potential simulation, click on the top left to open a menu of stimulations. Choose "Resting Potential Simulation."
On the right hand side, you'll see a membrane. The area below the membrane is the inside of the cell. The area above the membrane is the outside of the cell.
3. This simulation will open by default without any leak channels or pumps. Let's first add a couple of leak channels to the membrane and observe what happens. Click the plus sign next to "LEAK CHANNELS" to add a Na^+ and K^+ channel to the membrane.

These single channels represent many, many channels in the membrane that are allowing either Na^+ or K^+ to cross. By default, these will open with a relative permeability of 5:100 (Na^+ : K^+). Note also that the concentration settings are also set for you — at first, there is equal $[Na^+]$ and $[K^+]$ inside and outside of the membrane.

4. First, we'll observe what happens when we have more K^+ permeability, but equal ion concentrations. Click on "CREATE SIMULATION" to open the stimulation page. **Before running the simulation, write a 1-2 sentence prediction for what will happen in Table 1.**
5. To run Simulation #1, click the play button. You can also change the speed so that it will run faster. **What happens to the voltage over time? Why is the voltage across the membrane this value? Write your observations in Table 1.**
6. Go back to the Settings page by choosing "Back to settings" in the simulation play menu.
7. In order to modify the concentration across the membrane, we'll need a Na^+/K^+ pump. Click the plus button next to Na^+/K^+ pump to add one to the membrane.
8. Notice that the CONCENTRATION SETTINGS have now changed as well. Change these concentrations back to the previous settings — where all of the concentrations are 50 mM. **Before running Simulation #2, write your 1-2 sentence prediction in Table 1.**
9. Run Simulation #2 and observe what happens to the membrane potential. **Has adding a Na^+/K^+ pump changed anything about the resting membrane potential? Why or why**

not? Add your observations to the table. For this simulation, note that we're just using the Na/K pump to be able to change the concentration across the membrane. The Na/K is not operational (yet).

10. Go back to the settings page. In real neurons, ion concentrations are not equivalent. Change the concentrations of K⁺ and Na⁺ back to their default values to reflect more biological values across the membrane. **Write your prediction for Simulation #3 in the table.**
11. Run the simulation again and observe what happens to the membrane potential. **What happens to the voltage over time? Why is the voltage across the membrane this value? Write your observations in Table 1.**

Table 1. Simulation predictions & observations

Sim #	Types of Channels	[Ion]	PNa/K	Prediction	Observation
1	Leak channels only	All 50 mM	5 Na ⁺ ; 100 K ⁺		
2	Leak channels & Na ⁺ /K ⁺ pump	All 50 mM	5 Na ⁺ ; 100 K ⁺		
3	Leak channels & Na ⁺ /K ⁺ pump	Na ⁺ high outside; K ⁺ high inside	5 Na ⁺ ; 100 K ⁺		
4	Leak channels & Na ⁺ /K ⁺ pump	Na ⁺ high outside; K ⁺ high inside	100 Na ⁺ ; 5 K ⁺		
5	Leak channels & Na ⁺ /K ⁺ pump	Na ⁺ high outside; K ⁺ high inside	100 Na ⁺ ; 100 K ⁺		

12. Using the Goldman-Hodgkin-Katz (GHK) equation for Na⁺ and K⁺ only, calculate what the voltage across the membrane should be for simulation #3 and show your work. Does this match the voltage in the simulation? **Answer Q1 on the quiz. Report your answer in mV.**

Here are a few hints to help you:

- P is relative permeability; other constants can be found on the lecture slides.
- To simplify, you may determine a constant for the first portion of the equation (RT/F) and use that for future calculations.
- You can use "Insert > Equation" in Google Docs or Microsoft Word to show your work, or simply jot it on paper, take a picture, and paste it here.
- Google search has a built-in scientific calculator! You can bring it up by typing any formula into the search bar. For example:



=5*150



Q1. Calculation of GHK

Using the GHK equation for Na⁺ and K⁺ only, calculate the voltage across the membrane for simulation #3 and show your work. Does this match the voltage in the simulation? Report your answer in mV.

13. Finally, let's check to see how much ion permeability matters. Go back to the settings page and reverse the Na⁺ and K⁺ permeabilities.
14. Write your prediction for Simulation #4 in Table 1.
15. Run Simulation #4 and observe what happens. Record your observations in Table 1 and respond to the question below. **Answer Q2 on the quiz. Report your answer in mV.**

 Q2.

With the high permeability to Na⁺ and low permeability to K⁺, what is the resting membrane potential? Report your value in mV out to one decimal point.

16. On the settings page, set both the Na⁺ and K⁺ permeabilities to 100. Write your prediction in Table 1.
17. Run the simulation to see what happens, record your observations, and respond to the question below. **Answer Q3 on the quiz.**

 Q3.

Which of these simulations best models a typical neural membrane?

18. Next, we'll check to see whether or not the membrane potential depends on [K⁺]_{out}. Reset the permeabilities and ion concentrations to the default values.
19. Systematically change [K⁺]_{out} to fill in the table below. Hint: if you hover over the chart on the left, you'll be able to see the precise voltage. **Answer Q4 on the quiz.**

Table 2. Dependence of V_{rest} on [K⁺]_{out}

[K ⁺] _{out} (mM)	V _{rest} (mV)
4	
10	
20	
50	
200	

20. Click the "Toggle Circuit Diagram" button on the top right hand corner to overlay circuit components. **Answer Q5 on the quiz.** You can also fill out the table below for your own notes.

💡 Q5.

In a few words, describe what each of the following are modeling in the neuron. The first one has been done for you.

Component	What it models
I_Na	sodium current across the membrane
g_Na	
E_Na	
I_K	
g_K	
E_K	

Part II: Passive membrane simulation

In this part, you'll set up a simulation that models one portion of an axon or dendrite. By the end of this part, you will be able to:

- Describe how voltage passively spreads through an axon or dendrite
- Understand length and time constants
- Determine how the passive spread of voltage is affected by diameter and membrane capacitance

Steps

1. Go to <https://neuromembrane.biology.ualberta.ca/> (if you're not already there) and open up the "Passive Membrane Simulation" (or Cable Theory Simulation) in the menu in the top left corner.
2. Add an external recording electrode that is two length constants (λ) away from your stimulating electrode by clicking the **+** button and changing the value after the second recording site. **Answer Q6 on the quiz.**

💡 Q6.

Before running the simulation or changing any parameters, describe the setup of this "experiment" in 1-2 sentences.

3. Click the "Toggle Circuit Diagram" button on the top right hand corner to overlay circuit components. **Answer Q7 on the quiz.**

💡 Q7.

Describe what this circuit diagram is showing us. Be sure to define each of the different components (r_m , r_i , c_m).

- Re-set the cable settings and click on "CREATE SIMULATION" to run our first iteration of this passive current injection. The current will be injected starting at 10 ms, and will be injected for 30 ms total. Observe the top plot to fill out row one in Table 3. For your "observations," take a close look at the Voltage vs. Time plot. **Why is our current injection this shape when recorded across the membrane?**

Table 3. Current injection results

Sim #	Diameter	Cm	Peak voltage at 2	Distance to 2 electrode	Voltage at 0 electrode @ 28 ms	Voltage at 2 electrode @ 28 ms	Observations
1	5 μm	1					
2	2 μm	1					
3	2 μm	2					

- The dorsal root ganglion axon of a giant blue whale has a diameter of $\sim 2 \mu\text{m}$. In the CABLE SETTINGS on the left, change the diameter of the simulated axon to model this axon.

Reflection

With this smaller diameter of $2 \mu\text{m}$, do other parameters of the membrane (bottom of the CABLE SETTINGS window) change? If so, why?

- Run the simulation with the smaller diameter and observe the Voltage vs Time plot again. Hint: you can use the Autoscale button () if the trace goes off the plot. Fill out Table 3 for this $2 \mu\text{m}$ cable simulation.
- Double the capacitance of the membrane, from 1 to $2 \mu\text{F}/\text{cm}^2$. Describe what happens to the time constant and the resulting voltage over time plot with more capacitance in Table 3.

Optional: Later in the course, you may want to use the Neuromembrane simulator to model [action potential](#).

RC Circuits

Let's build a circuit that models the passive membrane simulations you performed above.

Building an RC Circuit

Read pages 4-7 of the RC circuit lab exercise in your lab manual. Then, build the circuit below. We'll use the OUTPUT of your Powerlab to send a pulse of current through the circuit.

- It helps to start with the output ends in the power rails and work your way around. The **red** wire from the PowerLab is the positive terminal.

CAUTION: Do not connect the + and - terminals. This will short the circuit, heat, and melt the plastic. Check that your circuit is set up properly before turning on the pulse from your computer.

- Attach single BNC adapters to the + and - outputs of the PowerLab.

- Alternatively, connect a **BNC to double banana cable adapter** to the + output of your PowerLab.

3. Attach a red banana cable to the + terminal and a black banana cable to the - terminal of your PowerLab.
4. Attach the banana cables to red and black jumper wires using alligator clips, and insert the black and red jumper wires to the power rails, as shown in the circuit.
NOTE: Ensure that all circuit components are properly inserted into the breadboard. The wires should make contact with the metal strips inside the breadboard.
5. Add additional jumper wires, a resistor, and a capacitor, as shown in the circuit.
6. We'll use the PowerLab as our voltmeter, to record voltage in our circuit. Plug a **BNC to DIN8 adapter** and **double banana adapter** into Input 1 of the PowerLab.
7. Connect the ground/reference (**black**, -) side of the double banana adapter to the circuit ground with alligator clips.
NOTE: The "ground" in this circuit is really going to be our reference electrode. It should be connected to the black side of Input 1.
8. Connect the "hot" pin (**red**, +) of the double banana adaptor to the "record here" point on the circuit diagram.
9. Open LabChart with the **Circuit Lab** settings (<http://bit.ly/labchart>)
10. Set up the Stimulator to stimulate with 1 V for **1 second**. Make sure the range of your recording channel is 1 V. Switch the stimulator to "on".
11. Open Scope View and press **>Start**. You should see a waveform on your screen.
NOTE: You may need to Autoscale the channel (right click) to be able to see the entire trace.

The time constant should equal $\tau = RC$. What *should* the time constant be (in seconds), given the resistors and capacitors in the circuit? Mind your units. Answer Q8 on the quiz.

$\tau =$ _____

We can also measure the time constant by measuring how long it takes the curve to rise or decay. One time constant (τ) is defined by when the circuit rises to **63.2%** of its total charge (alternately, we could see how long it takes to decay $1/e$, about 37%).

To empirically measure your time constant, follow these steps.

1. Make sure your voltage is very close to zero at baseline.
2. Find the maximum voltage (V_{max}) that your circuit reaches.
3. Calculate $0.632 * V_{max}$
4. Using the marker tool (right click anywhere in the window and choose "Clamp to trace") find the point in your trace that is equal to the value calculated in Step 3.
5. Move your cursor to the start of the stimulus. In the upper right corner of the window, you'll see something that says Δt . This is the change in time since the mark on the curve and your time constant.

What is your recorded time constant with 1 V stimulation, a 100 k Ω resistor, and a 1 μ F capacitor?

$\tau =$ _____

How close is the recorded time constant to the value you calculated with $\tau = RC$?

In engineering, 5τ is consisted the time it takes for a capacitor to fully charge (technically, its at 99% of full charge). The time constant is symmetrical for charging as well as losing charge. **How long would it take this patch of membrane to return to baseline, then? What does that mean for computation happening in this membrane?**

Record and plot your time constant with different resistors

Next, we'll see how the time constant of the circuit changes in response to the resistance strength. Use the following resistors in your circuit, and use the steps above to determine the time constant for each.

Resistance	Time Constant
3 k Ω	
15 k Ω	
33 k Ω	
100 k Ω	
220 k Ω	
330 k Ω	

Plot the time constant against the resistance. Answer Q9a and Q9b on the quiz.

The plot should have:

- A line fit to it, with a description of what the trendline is (you can either show the equation or say in the figure caption what kind of trendline is plotted with the R² value.
- Labeled axes & units
- At least six data points

Plotting Tips:

- Python: Use the [Plot Scatter](#) notebook
- Excel:
 - Insert your plot as a scatter plot (this will tell Excel to treat your x-axis values (resistance) as actual values, not categories.
 - Add a trendline by right clicking on the data, and choosing "Add Trendline". You should add a trendline for the trend you're expecting (we talked about this in lecture).
 - Make sure that none of the markers for your points are cut off. If they are, change your axes so that they fit.

Check your time constant against the literature

Let's circle back to biology. Are the time constants you measured similar to what you'd find in a real neuron? Check this website to look at a collection of examples from the literature:

https://neuroelectro.org/ephys_prop/index.html

Choose a paper from the example above and look for the reported time constant. Is the time constant in your circuit similar? If not, why might it be different?

Troubleshooting

Observation	Likely Issue	Possible Solution
There isn't any voltage in your circuit	The power terminals are not connected properly Your stimulator is not on	Make sure everything is connected correctly on your breadboard. Consult https://wiring.org.co/learning/tutorials/breadboard/index.html Check that the stimulator panel is switched on, and that you pressed Start
Your signal is really noisy	You're not grounded. Your pulse is not being properly sent.	Double check your circuit is connected correctly
You don't see a fin-shaped waveform	Your Scope/Chart View is not scaled properly. Your capacitor is not properly connected or broken.	Try autoscaling Check the capacitor placement or replace it.
You've got "rabbit ears" on your waveform	Your voltage is reversed somewhere	Check the power terminals are connected in the right direction

String Nervous Systems

Background & Goals

Why are we recording from a string?

The purpose of this lab is to get familiar with the recording equipment and software that we'll be using in this course.

First, we'll learn how you can configure LabChart 8 to acquire data and send out pulses of electricity. Then, you'll record the **stimulus artifact** (the passively conducted signal from the stimulus) from a saline-soaked piece of string. The string mimics some of the properties of the earthworm that you'll observe in a future experiment. This will give you the opportunity to learn how to use the stimulating and recording devices on the PowerLab, as well as reduce noise and stimulus artifacts from your recordings, before you actually have to do all of this in an experiment. Asking questions during this lab is a great idea, and will help you succeed in future experiments.

Pre-Lab

Before the lab, watch this video and read the protocol below:

Today you will:

- Configure a recording experiment in LabChart 8.
- Determine how to record signals from a string.
- Identify sources of electrical noise in your rig.

This lab was adapted from Experiment #1 from the BIPN 105 & BIPN 145 Lab Manuals, as well as AD Instruments.

Lab Protocol

Supplies	Cables & connectors	Solutions
String	BNC to single banana adapter (2)	Saline
Dissection tray	BNC to double banana adapter	
PowerLab 26T, connected to computer	BNC to DIN8 adapter	
BioAmp	Alligator clip adapters (2)	
Sewing pins (2)	Banana plug cords (2)	
Faraday Cage		
Large Petri Dish		
Eyedropper		

I. Setting up LabChart 8

LabChart 8 is the software we'll be using to record electrophysiology data. Before performing our experiment, we need to set up LabChart to **acquire** and **display** the data as we want.

1. Turn on the PowerLab 26T.
2. Open LabChart 8. You should see that the PowerLab is connected with a green check mark.
3. Open a new experiment by choosing the **New** button (bottom left-hand corner).
4. First, we need to decide how many channels to record. Go to **Setup > Channel Settings**.
 - At the bottom of the window, change the number of channels to **3**, and make sure that they are all on (box on the left is checked).
 - Rename the first channel **"Stimulus"** and the second channel **"Raw Recording"**.
 - Set the range for Channel 1 (your "Stimulus" channel) to **5 V**.

Note: The range determines the range of values the PowerLab will record, and when using the BioAmp (later in this lab), it will actually change the amplification.

- Set the range for Channel 2 (your "Raw Recording" channel) to **100 mV**.
- Make the sampling rate for Channel 1 **"40k/s"** and ensure that *"Same sampling rate for all channels"* is checked.
- Make sure that the far-right column says **"No calculation"** for all three channels.

II. Finding sources of electrical noise

One of the biggest issues in electrophysiology is noise from overhead lights and other electronics in the room. It'll help to get an understanding of where noise in your rig is coming from.

1. Connect a DIN8-BNC adapter and a double 3-way BNC adapter to **Input 2** of the PowerLab. Plug in a red banana cable to the positive (red) side of the connector.
2. Hold the lead (the end of the banana cable) in your hand.
3. In the Channel Settings window, click on **Input Amplifier** for Channel 2. The Input Amplifier window will show the voltage that you're recording. This allows for precise setting of the

input range for a recording channel and provides filtering options.

4. The signal at your rig may be variable. To adjust the sensitivity of the channel, choose an appropriate range setting from the **Range** drop-down list in the Input Amplifier dialog.

Note: The number displayed in the range menu indicates the maximum input voltage currently selected. Notice how as you decrease the range the vertical scale changes and the small rhythmic deflections that appear on the signal trace increase in amplitude.

5. Choose a range such that your signal fills about $\frac{2}{3}$ of the window.
6. Observe the change in the recorded voltage with the electrode lead held overhead (near the lights) and then near the ground (away from the lights).
7. Try the lead inside of your Faraday cage. Is the signal more or less noisy?

Tip: Is your Faraday cage actually grounded? How could you ground it?

Questions for reflection

- How does the signal change as you move the electrode lead around?
- What do these sources of noise mean for our recording configuration, and our attempts to record electrical activity?

III. Recording stimulator outputs

We'll use our PowerLab as a stimulator that can send voltage pulses into our specimen. We can change the timing, shape, amplitude, and repetition of these stimuli via the Stimulator window in LabChart. We'll directly record these stimuli by sending the output of the PowerLab into an input.

1. Change your view to **Scope view**, which looks like the sweeps of an oscilloscope. The advantage of Scope View is that it allows the overlay and averaging of your data.
2. In Scope view, set the **Duration** to **20 ms**. This changes how long each sweep of data is.
3. Remove the DIN8-BNC adapter & cable from Input 2.
4. On the PowerLab, connect the **(+) Output** to **Input 1** using the BNC to DIN8 cable.
5. Go to **Setup > Stimulator** to open the Stimulator window. This is the window where you can modify the shape, timing, and amplitude of the stimulus output.
6. Make sure that the **Waveform Name** is set to **Pulse**.
7. Change the range of the Pulse Height to **-5 to 5 V** by clicking the ruler button next to the Pulse Height option.
8. Configure the following by clicking and then editing the values in the window:

Parameter	Value
Start Delay	1 ms
Pulse Width	0.1 ms
Pulse Height	0.150 V

9. Close the Stimulator window.
10. Go to **Setup > Stimulator Panel** to open a smaller window where you can directly modify the stimulus. This window can stay open while you're recording.

11. Press **> Start** in the main LabChart GUI.
12. Increase the stimulator voltage (Pulse Height) and watch the level change in the window.
13. Try a negative stimulus voltage.
14. Open the Stimulator window again and vary the delay, duration, and waveform shape, and see what happens. If your recording is cut off, how can you fix it? (*Hint: see Visualization tips in Meet Your PowerLab.*)



Questions for reflection

- Why would you want to record the stimulus output?
- Why would you want to delay the stimulus output?
- Why would you want to send different shapes of waveforms?

IV. Recording the stimulus artifact

Lastly, we'll add an amplifier (the BioAmp) to our circuit, and use this to record from a string soaked in saline. The string doesn't have a nervous system, but we'll add some voltage with our stimulator and observe the change in voltage from another point on the string.

Set up your string stimulation experiment

Your setup for the string stimulation will look like this:

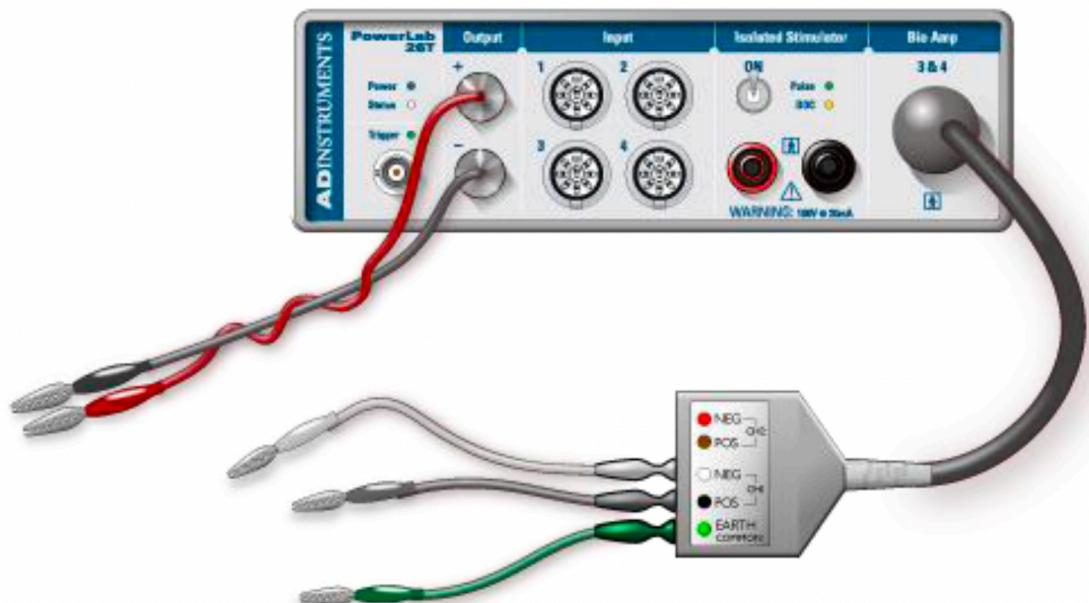


Image courtesy of ADInstruments ©

1. Remove all of the cords from the PowerLab.
2. Connect your two single BNC to banana cable adapters into the **(+)** and **(-)** Outputs of the PowerLab.

Note: The color of these does not matter, but it can help to be consistent — red for the anode (+ Output) and black for the cathode (- Output).

3. Using banana cables, connect the **(+) Output** to your **anode** stimulating electrode (the sewing pin) and the **(-) Output** to the **cathode** stimulating electrode.

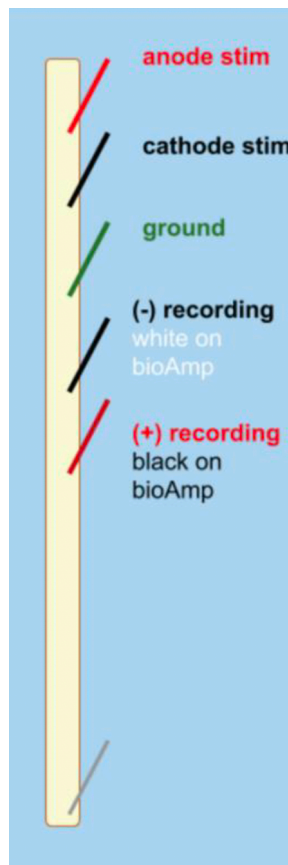
4. Connect the BioAmp to the PowerLab.
5. Connect the (+) recording, (-) recording, and ground needle electrodes to your string and into the corresponding spots on the BioAmp.

Note: The colors on the BioAmp defy convention — consult the diagram above for clarity.

6. Change the configuration of your recording so that you are now using the BioAmp:
 - Go into **Setup > Channel Settings**.
 - Make sure that **BioAmp** is listed under the Input Amplifier column for Channel 3.
 - **Uncheck** Channel 2 — we don't need to record from it anymore.
 - Change the name of Channel 3 to **"BioAmp Recording."**
7. We've removed our ability to directly record the stimulus output, so we need another way to know when the stimulator sends outputs. Open the Stimulator Settings and set the **"Marker Channel"** to Ch1 (your "Stimulus" channel). With this setting, LabChart will put a small tick on Channel 1 to show you when the stimulus was sent.

Note: This marker tells you *when* the stimulus was sent, but does not indicate the height or length of the stimulus. This means you need to label your pages with your stimulation parameters.

Stimulate your string!



1. Place your string in saline in the small petri dish, and give it a few minutes to soak. If it's not completely soaked, you will not get a stimulus artifact.

Note: If your string dries out during the experiment, you can use an eyedropper to add saline to it.

2. Lay the string out on the dissection tray, with the stimulating and recording electrodes laid out as in the diagram above.
3. Set the stimulator voltage to **1 V**, and press **> Start**. You should see your stimulus recording on Channel 1, as you did before, but you should also now have a recording on Channel 3 (BioAmp Recording).
4. Change the scaling if you cannot see the entire deflection.
5. If you do not see anything, increase the voltage (no more than 5 V).

Note: This recorded deflection is called the **stimulus artifact**. This artifact can exceed the range of our amplifier. Recovery from the artifact should be rapid, however, so that the baseline is reached within one millisecond or so after the stimulus pulse.

Question for reflection

Is the amplitude of the stimulus artifact the same as the amplitude of the stimulus? Why or why not?

6. Increase the stimulator voltage, and stimulate again. What happens?

Note: Change the name of Scope pages, or make comments on the recording, to indicate what you've done during that run of the stimulus. You need to know the metadata for the experiment once you're ready to export.

7. Change the stimulus to a **negative voltage**, and stimulate again. Does the stimulus artifact change?
8. Choose one of your pages where you have a stimulus artifact with a clear peak. Using the **Marker tool**, measure the latency from the stimulus to the beginning of your artifact. How much of a delay is there? How much of a delay *should* there be? (How fast does an electrical current flow?)
9. Change the location of the stimulating electrodes. How does this change the shape of the artifact?
10. Remove the ground electrode. How does this change your recording?

Save your file & export your data

LabChart buffers files to disk in case of a power failure or computer crash, but it's wise to save work frequently. Saving files in LabChart is the same as saving any file on your personal computer. Please backup your LabChart data files to the Google Drive you set up at the start of the quarter.

1. Save the **settings file** for your experiment by going to **File > Save As Settings File** in LabChart. Save it in a safe place (not locally on the computer). You will need this settings file for the earthworm experiment.
2. Save the **data file** for your experiment by going to **File > Save As** and save as a LabChart Data File (.adicht).
3. Go to the Scope page for **two of your favorite string stimulation experiments** that you can compare and contrast — for example, two different voltages, or with two different distances between the stimulator and recording electrodes.

4. Follow [these instructions for exporting your data](#).
5. Plot your recordings (separately or on the same axes — be sure to label two different recordings if overlaid). Make sure your axes are labeled with units. You can create these plots in [Python](#) or [Excel](#).
6. Copy your plots into a Word or Google document.
7. See your instructor's Canvas assignment for detailed directions about what else you need to submit.

Troubleshooting

Observation	Likely issue	Possible solution
You can see the stimulus marker on Channel 1, but no stimulus artifact on Channel 3	Your string may not be soaked in saline well enough	Add saline to the string
The recording trace is cut off	Your visualization needs to be re-scaled, or your range is too small	Right-click on channel and choose Auto Scale (see Visualization tips above); change the range on your recording channel
Your string recording trace has clear 60 Hz noise	You're not grounded, or there are ground loops	Make sure your Faraday cage and ground electrode are both grounded to the back of the PowerLab. If this doesn't help, consult an instructor.

Earthworm Electrophysiology

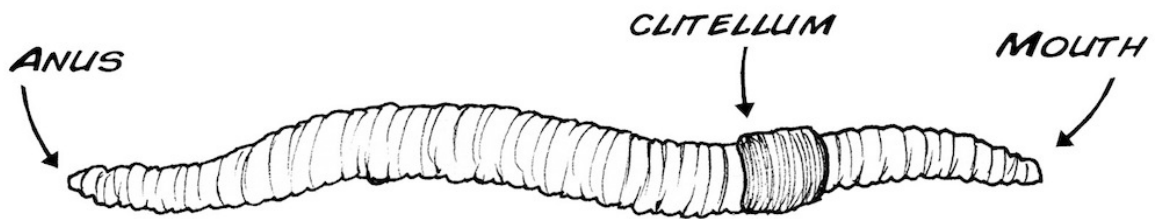
Background & Goals

Earthworm essentials

Earthworms are a member of the Annelida phylum, which includes segmented worms such as the ragworm and our soon-to-be-friend, the leech. We'll be working with earthworms in the Lumbricidae family.

Earthworms are laid out with a mouth at one end, and an anus at the other. In the middle is the clitellum, a reproductive gland that can secrete a material that helps them form a cocoon. The clitellum is only visible in older worms that are ready to reproduce.

Earthworms have a closed nervous system, with one main blood vessel running down the length of their body. They breathe through their skin, so it is important to keep it moist. They crawl using muscles that are located just underneath the epidermis.



TOP (DORSAL) SIDE OF WORM DARKER THAN BOTTOM (VENTRAL)

Earthworm nervous system

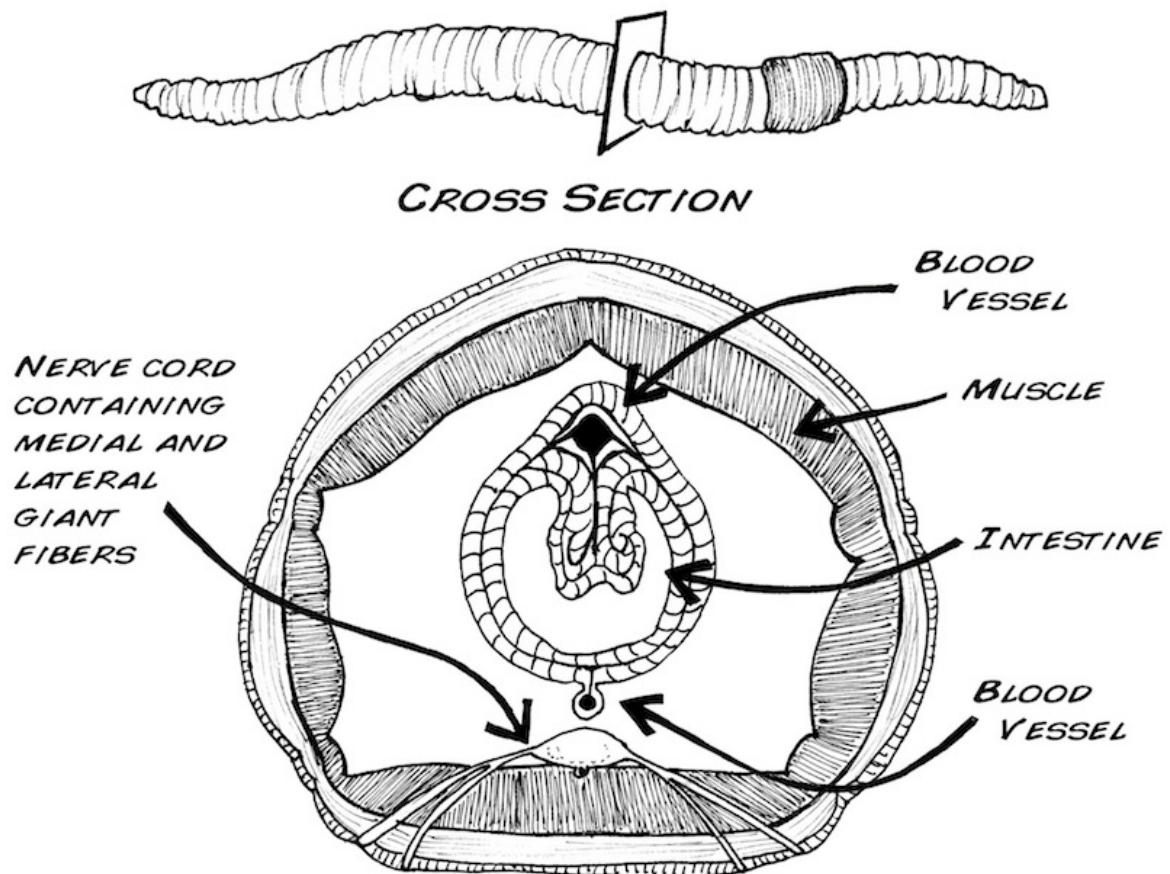
Earthworms have a pretty simple brain: it's actually a fused pair of nerve ganglia, mostly located in their third segment. The earthworm has one main nerve cord on its ventral (bottom) side with three fibers running through it: one medial giant fiber, and two lateral giant fibers.

These fibers are big bundles of axons. They serve as high-speed motor pathways that enable the worm to send movement signals throughout its body for quick, stereotyped reflexes.

Each giant fiber is made up of large individual cells (one per segment) that are electrically coupled to each other through gap junctions. This results in the rapid conduction of action potentials from cell to cell so that each giant fiber behaves as though it were a single axon. There are cross-connections between the medial and lateral fibers, so they often function as one unit.

These fibers differ in important ways. The medial giant fiber transmits information from skin cells in the anterior (mouth) end of the worm, and the lateral giant fibers transmit information from the skin cells of the posterior end of the worm.

It's quite easy to listen in to the activity of these large fibers, even from the skin of the earthworm. Importantly, these fibers fire action potentials very rarely. This type of nervous system activity is often called sparse coding. That means that we should be able to elicit one action potential and cleanly record it.



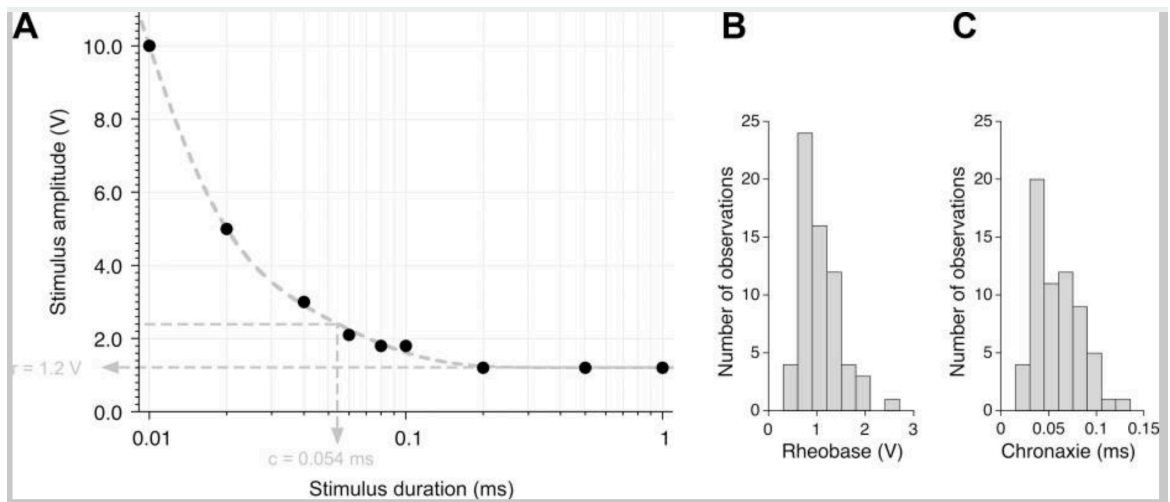
Images from [Backyard Brains](#), © 2009–2026, used under a Creative Commons Attribution-ShareAlike license.

Properties of action potentials

In this lab, we'll determine the **threshold**, **rheobase & chronaxie**, **refractory period**, and **conduction velocity** for the earthworm giant fiber system. We're recording compound action potentials (CAPs) from many axons in these fiber bundles, but the underlying principles are the same.

The **threshold** is defined as the minimum stimulus voltage that elicits an action potential 50% of the time. We'll determine this with short stimuli. Stimulus voltages that don't elicit an action potential are called subthreshold; stimulus voltages that do elicit an action potential are called suprathreshold.

In some neurons, we can elicit an action potential with a lower stimulus voltage if the stimulus length is longer. A longer stimulus length enables charge to build up that can ultimately elicit an action potential. However, there is a minimum stimulus voltage needed, even at infinite stimulus durations. The **rheobase** is defined as the minimum stimulus voltage that can elicit an action potential with very long durations of the stimulus. The stimulus duration at 2× the rheobase is called the **chronaxie**. Chronaxie is a useful measure of the excitability of a nerve — the most excitable nerves have the smallest chronaxie. We can plot this data in a strength duration curve:



The **absolute refractory period** is the minimum amount of time needed for a neuron to fire a second action potential. This is caused by the inactivation of sodium channels after an action potential — it takes time for them to close, to then be reopened by a depolarizing stimulus. Since we're recording from many axons, your measured refractory period will be more variable than for a single axon. We'll define the absolute refractory period as the time when the amplitude of the second CAP is 30% of the first.

After an action potential, the membrane of the axon is also hyperpolarized, due to the slowness of K^+ channels closing. So, there is a period of time where the neuron requires more voltage to fire an action potential. This period is called the **relative refractory period**. In this lab, we'll identify it as the period where the amplitude of the second CAP is 30–90% of the first.

In these experiments you will:

- Stimulate the ventral nerve cord of the earthworm to elicit an action potential
- Determine the voltage that is needed to recruit the lateral giant fibers
- Estimate the absolute and relative refractory period between action potentials
- Calculate the speed of an action potential
- Propose your own hypothesis about conduction velocity
- Conduct your experiment to test your hypothesis

References

- Bähring, R. & Bauer C.K. (2014) Easy method to examine single nerve fiber excitability and conduction parameters using intact nonanesthetized earthworms. *Advances in Physiology Education* 38(3): 253–264.
- Bullock, T.H. (1945) Functional organization of the giant fibre system of *Lumbricus*. *J Neurophysiol* 8:54–71.
- Drewes CD, Landa KB, McFall JL (1978) Giant nerve fibre activity in intact, freely moving earthworms. *J Exp Biol* 72:217–227.
- Kladt et al. (2010) Teaching basic neurophysiology in intact earthworms. *JUNE* 9(1):20–35.
- Rudell & Fox (1991) The Propagation potential: An Axonal response with implications for scalp-recorded EEG. *Biophysical Journal* 60: 556–567.
- Shannon et al. (2014) Portable conduction velocity experiments using earthworms for the college and high school neuroscience teaching laboratory. *Advances in Physiology Education* 38(1): 62–70.

Lab Protocol

There are multiple days dedicated to these earthworm experiments. You are encouraged to complete as many of the experiments below while you have a functioning earthworm.

Supplies	Cables & connectors	Solutions
Earthworm	BNC to single banana adapter (2)	Earthworm saline
Dissection tray	Banana plug cords (2)	Earthworm anesthetic (10% ethanol in Ringers)
PowerLab 26T, connected to computer	Alligator clip adapters (2)	
BioAmp	Needle electrodes	
Sewing pins (3)		
Ruler		
Thermometer		
Faraday Cage		
Large Petri Dish		
Eyedropper		

I. Preparation

Setting up the PowerLab 26T

Our first step is making sure our PowerLab & Amplifier are wired up correctly.

Make sure the PowerLab is turned off.

1. **Connect a single BNC adapter to the (+) Output** of the PowerLab. Plug a red banana plug cable into the adapter.
2. **Connect a single BNC to banana adapter to the (–) Output.** Plug a black banana plug cable into the adapter.
3. Make sure that the BioAmp is connected to the PowerLab, with three different needle electrodes plugged into it.
4. Once you're confident everything is connected correctly, turn on the PowerLab.

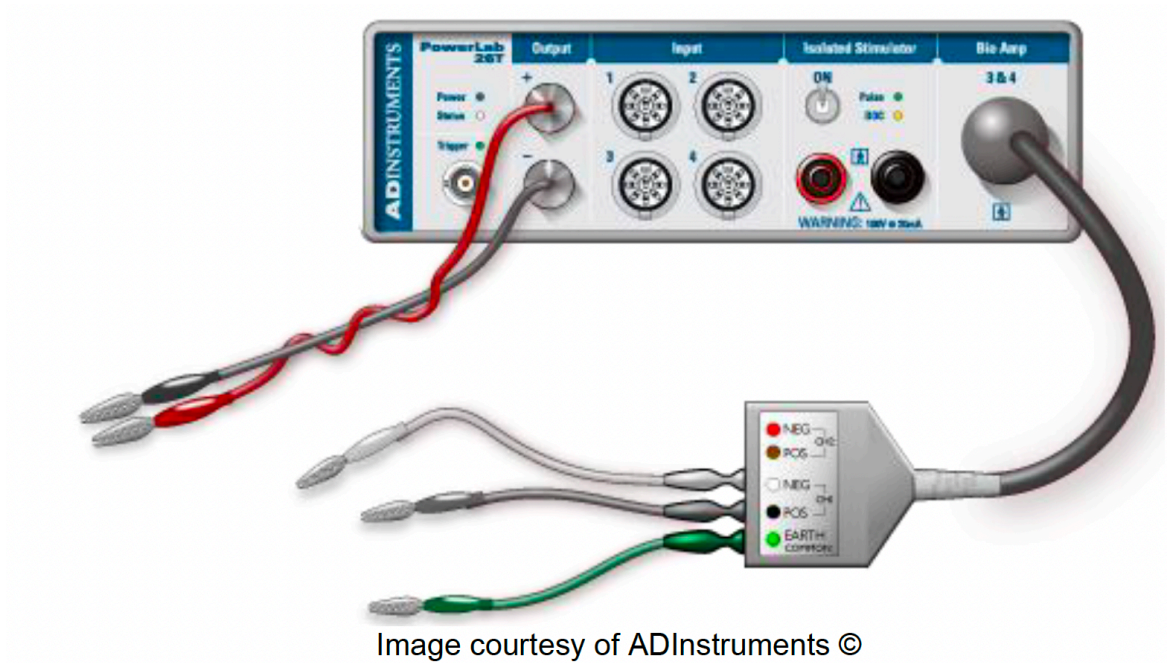


Image courtesy of ADInstruments ©

Image courtesy of ADInstruments ©

Setting up LabChart 8

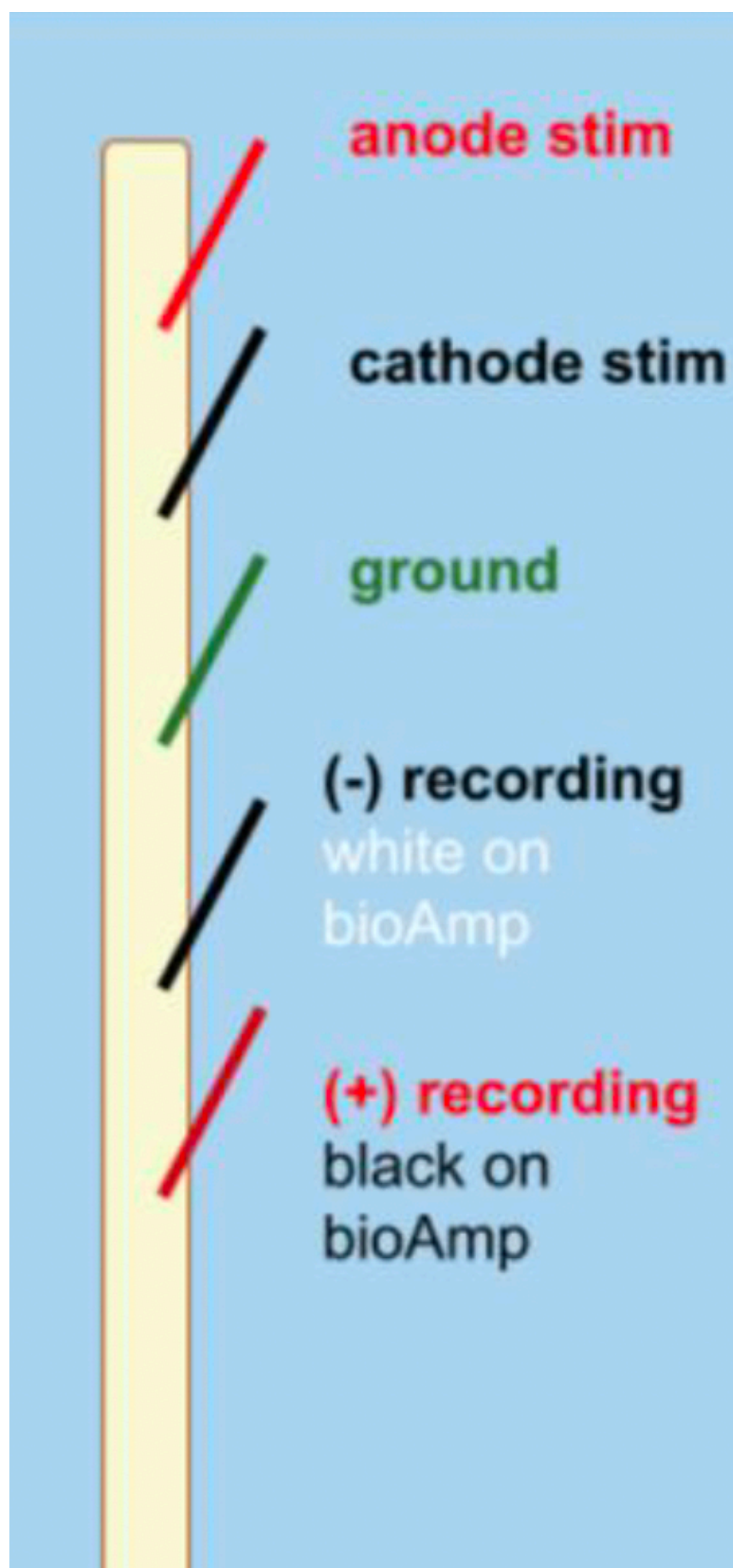
1. **Open LabChart 8 with the settings file you saved during the string experiment.** Most settings are saved within this settings file. However, it's always good to ensure that you have the correct settings before beginning.

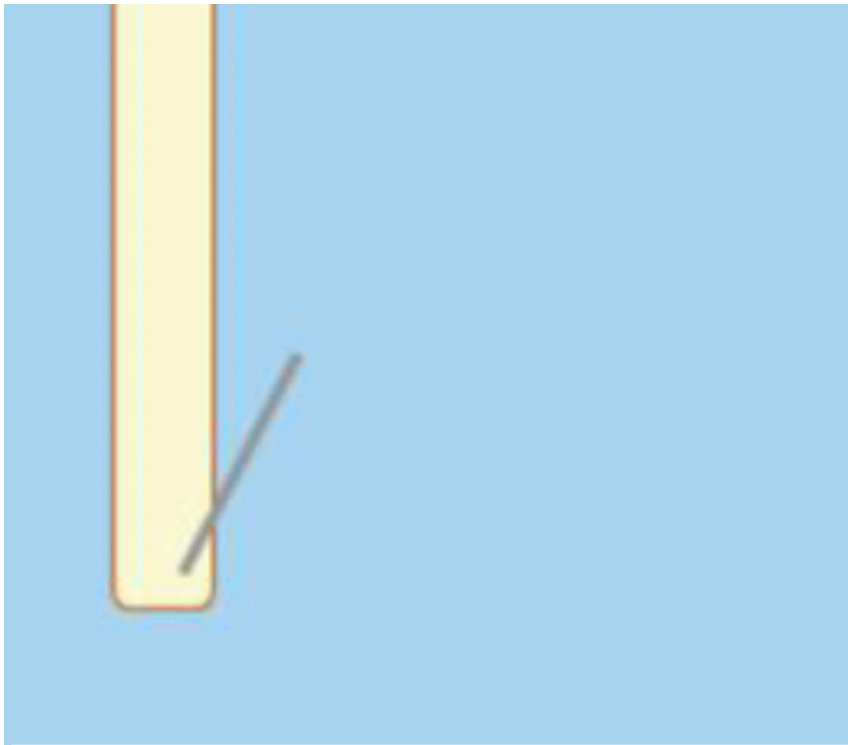
Note: If for some reason you cannot access your settings file, you can find one at <http://bit.ly/labchart>. Use the "Earthworm bioAmp Settings."

2. **Go to Setup > Channel Settings** and confirm that you have three recording channels, and that the third ("BioAmp Recording") is connected to the BioAmp.
3. **Go to Setup > Stimulator** and change the Stimulator settings as follows. Make sure your Marker Channel is Channel 1.

Parameter	Value
Start Delay	0.1 ms
Repeats	1
Max Repeat Rate	1 Hz
Pulse Height	0.2 V
Pulse Width	0.2 ms

Setting up your earthworm





Note: Throughout every step of this procedure, be careful not to stretch the earthworm. This will damage the nerve and your experiment will not work.

1. Obtain a large, lively earthworm. Rinse it with saline if it is dirty.
2. **Anesthetize the earthworm** by placing it in the petri dish with 10% ethanol (in Earthworm Ringer's solution) for 5 minutes.

Note: Do not leave your earthworm unattended in anesthesia. With too little anesthesia, the earthworm will move around during the experiment, and the resulting muscle electrical activity (electromyogram) will drown out the small neural electrical signals you are interested in. Too much anesthesia and the nerves will not fire.

Note #2: Your earthworm may still react to pressure after 5 minutes — you can proceed with the experiment as long as you are able to handle the worm and pin it down.

3. **When your earthworm is anesthetized enough to pin it down**, remove the earthworm from the ethanol and place it on the dissection tray with the dorsal (darker side) up.
4. Pin the earthworm to the tray using one sewing pin on each end of the worm.
5. Place the ruler down next to your worm.
6. **Insert a third sewing pin into the anterior end of the earthworm**, ~1 cm from the first pin. Connect the anode & cathode stimulating electrodes to the first two pins.

Note: The anterior end is the end nearest the clitellum, a smooth, non-segmented band.

7. To keep the worm anesthetized, apply a few drops of the earthworm anesthetic along the length of the earthworm with the eyedropper. Soak up any excess fluid.

Note: Make sure there isn't a large puddle of solution — too much may interfere with your recording.

8. Insert the three recording needle electrodes into the worm (same configuration as the string experiment!).

Note: Make sure no wire is touching any other wire. Be careful when positioning the wires to avoid damaging the worm.

Note #2: It helps to place your recording electrodes as lateral as possible, while still holding down the worm.

II. Determining the threshold of your fibers ★

In this experiment, you will send a stimulus into the earthworm while recording from it to determine the threshold voltage of the medial and lateral giant fibers.

Determine the threshold to activate the Medial Giant Fiber

1. **Open your Input Amplifier (BioAmp)** for your Action Potential channel to make sure that your signal looks reasonable. You shouldn't see 60 Hz noise, but it should look like a physiological signal. (If you have noise, see Troubleshooting.)
2. **Open the Stimulator Panel** by going to the Setup menu. Your stimulator should start with:
 - Max Repeat Rate: 1 Hz
 - Pulse Height: 0.2 V
 - Pulse Width: 0.2 ms

Note: Leave the Stimulator Panel open — we'll need it throughout the experiment.

3. **Press > Start.** LabChart should be in Scope View and display one 20 ms sweep every time you click Start. For each 20 ms sweep, a stimulus pulse is generated through the PowerLab Stimulator outputs.
4. **Examine the recording** — you should see a quick deflection very shortly after your stimulus pulse. The deflection just after the start of the sweep is your stimulus artifact, caused by spread of the stimulus voltage to the recording electrodes.

Note: The stimulus artifact can be very large in amplitude, and can exceed the range of the recording amplifier. It is okay if it is cut off.

5. **Change the Pulse Height to 1 V.**
6. Each time you change the stimulus, click Start and a new page will appear to the left of the Scope View Window. This is the Page Explorer. **Label the individual pages** of your recording as you go along.
7. **Increase the Pulse Height in 0.1 V steps** until you see an inflection after the stimulus artifact. This means you've successfully elicited a response from the medial giant fiber! Note the stimulus voltage as an estimate of the threshold stimulus strength.

Note: If you can't see anything, right-click on the channel to change the scaling (or use the – and + buttons on the bottom left of the channel).

Note #2: If no response is seen with a stimulus of 3.5 V or greater, contact your instructor for assistance.

8. **Find a more accurate threshold** by first reducing the stimulus by 0.2 V (or more) to diminish your response. Then, increase the stimulus by 0.05 V between each sweep. You should report a threshold (Table 1) to the hundredth decimal point (e.g. 1.25 V).
9. Save your data, but do not close the file.

Determine the threshold to recruit the Lateral Giant Fibers

In the next stage, you will attempt to elicit an action potential in the lateral giant fibers.

Note: Remember to keep the earthworm anesthetized!

1. Set the Pulse Height to your threshold for the MGF.
2. Keep increasing the stimulus in 0.1 V steps until you see a second inflection in your recorded trace.
3. Once you see a second inflection, follow the same procedure as above to determine the threshold where you can elicit an action potential from the lateral giant fiber about 50% of the time. Record the stimulus threshold in Table 1 below.
4. Save your data, but do not close the file.

Note: In some worms, the median and lateral fiber responses may not be separated — they will appear to have the same threshold. Other worms may not show the second response, possibly because the lateral giant fibers are damaged.

5. Save your LabChart file.

Table 1. MGF and LGF compound action potential (CAP) properties.

	Threshold	Amplitude	Latency
MGF			
LGF			

Questions for reflection

- Did the shape or amplitude of the CAPs change with more stimulation voltage? What does this tell us about the action potential that is firing?
- Which fiber fires an action potential sooner (with a shorter latency)?
- Why would the MGF and LGF latencies differ?

III. Calculate the rheobase ★

In Part III, we'll change the stimulus amplitude and duration to calculate the rheobase and chronaxie of the earthworm medial giant fiber (read the Background if you forget what these are).

In the previous experiment, we gave a stimulus for 0.2 ms. Now, we'll modify the stimulus duration and amplitude to determine the minimum stimulus requirement, regardless of duration.

1. Write your MGF threshold in the column for 0.20 ms stimulus duration in Table 2.

2. Change your Pulse Height to the threshold for your MGF.
3. Systematically change the duration of your stimulus (Pulse Width) to the values in the table below. As necessary, change the stimulus voltage to successfully get a CAP from your MGF.
4. For each duration, record the stimulus voltage (Pulse Height) values that successfully elicit a CAP in Table 2.

Note: Be sure to label your pages in Scope view accordingly. You don't need to save the pages that do not elicit an action potential.

Table 2. Data for strength-duration curve.

Duration (ms)	0.06	0.08	0.10	0.20	0.40	0.60	1.00
Stimulus Amplitude (V)							

IV. Determine the refractory period

In the next step, you'll determine the shortest time between stimuli where you can elicit two different CAPs from the MGF. In other words, we'll determine the relative and absolute refractory period of these fibers.

1. Set your pulse height so that you can comfortably elicit a response from the MGF.
2. **In Setup > Stimulator, change Repeats to 2** in order to play multiple pulses.
3. We also need to change the time between each stimulus. PowerLab thinks of this interval as Hz. To start, we want 15 ms between each stimulus.

Note: Remember that Hertz (Hz) means events per second. So, 1 Hz = 1 event per second, and 10 Hz = 10 events per second. $1 \div \text{frequency (in Hz)} = \text{time between stimuli (in seconds)}$.

💡 Questions for reflection

- How many milliseconds would there be between each stimulus at 10 Hz stimulation?
- For 15 ms between each stimulus, what should the frequency of stimulation (in Hz) be?

4. Fill out this chart with the corresponding Hz values for each interval length:

Interval (ms)	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1
Freq. (Hz)															

5. **Change your Max Repeat Rate** to the equivalent frequency for 15 ms intervals.
6. Change your Pulse Width to 0.2 ms and your Pulse Height to the threshold for your MGF.
7. **Press > Start** to record a trial with two stimuli, separated by 15 ms.
8. Right-click and choose **Add Marker > Clamp to Trace** to place a marker at the moment the stimulus occurs. Move your cursor to the peak of the action potential. Look at the ΔV on the

right-hand side to see the change in voltage from that point.

9. **Measure the amplitude of your first and second action potentials.** Determine how strong the second was relative to the first (e.g. if AP #2 amplitude was half as high as #1, it would be 50%). Record your response in row one of Table 3.
10. Decrease the duration between stimuli by 1 ms by changing the Repeat Rate to the Hz for a 14 ms interval.
11. **Record the amplitudes and corresponding percentages for each stimulus interval in Table 3.**
12. At some point, AP #2 will diminish below 30% of AP #1. We can consider this our absolute refractory period.

Note: Depending on the latency of your MGF CAP, it may be difficult to see a CAP with a stimulus interval of 1 or 2 ms. Reduce the stimulus interval to as short as you can before the MGF CAP is cut off by the second stimulus. If the shortest stimulus interval is >3 ms, you should move your pins closer together.

13. Save your LabChart file.

Note: If you have data that doesn't seem quite right, you should repeat your experiment. You must be upfront about your number of repeats in your lab report, comprehensively reporting all of the data that you collected.

Table 3. Amplitudes of CAPs elicited by stimuli with decreasing stimulus intervals.

Stimulus Interval (ms)	#1 CAP Amplitude	#2 CAP Amplitude	% #2/#1
15			
14			
13			
12			
11			
10			
9			
8			
7			
6			
5			
4			
3			
2			
1			



Questions for reflection

- What was the shortest duration between stimuli where the second action potential was less than 90% but more than 30% of the original CAP amplitude?
- What was the shortest duration between stimuli where the second CAP was diminished below 30%?
- Which of these values is your absolute refractory period, and which is the relative refractory period?

V. Compute the conduction velocity ★

(If you're doing this on a different day, follow the same steps for preparing LabChart & your earthworm as you did in the previous earthworm experiment.)

How far is your first recording pin (–) from your cathode (–) stimulation pin? Be sure to include units: _____

1. **In PowerLab, press Start.** Confirm that you have two recording channels.
2. Follow the same protocol as in the previous experiments to determine the threshold required to elicit an action potential from your medial giant fiber.

What is the stimulus voltage needed to elicit an MGF action potential from this worm?

Determine the MGF latency to spike

- 1. Using the Marker tool, measure the latency (in seconds) from the peak of the stimulus artifact to your elicited action potential.
- 2. Repeat this 5 times and record your data in Table 4.

Table 4. MGF conduction velocity data.

Trial	MGF Baseline Latency (s)	Speed (m/s)	MGF Experimental Latency (s)	Speed (m/s)
1				
2				
3				
4				
5				

- 3. Since we know when the AP occurred and the distance between your electrodes, we can calculate the speed of the action potential. Record these values in the "Speed" column of the table above. (Reminder: $speed = distance/time$.)

How fast, on average, is your action potential traveling down the MGF?

VI. Design your own experiment to test the impact of temperature on MGF conduction velocity

For this final part, you will design your own experiment to test the impact of temperature on conduction velocity. We will provide ice and a thermometer, but the rest is up to you! If you are performing this experiment on a separate worm, be sure to measure the threshold for the MGF as described above.

Work with your group to determine a protocol for running your experiment — you'll need to include this in the Methods section of your lab report. Think about what someone needs to know in order to replicate this experiment exactly as you did.

We will test the effect of _____ on _____.

We will do this by...

Conduct your experiment

Work with your group to determine a protocol for running your experiment — you'll need to include this in the Methods section of your lab report.

Record your results in the MGF Experimental AP Latency column of Table 4. You'll need these later for the Results section of your lab report.

Troubleshooting

Observation	Likely issue	Possible solution
The recording trace is flat	Something is disconnected	Check all of your connections
I can't see a stimulus trace in the Stimulus channel	You don't have a marker channel set up	Go to Setup > Stimulator to set up the Marker Channel
I can't see a stimulus artifact in my action potential channel	Your electrodes may not be in the worm, or stimulus amplitude is too weak	Ensure all electrodes are in the worm (but not too medial), and try stimulating at 0.5 V
My worm is bleeding	The electrode is through the blood vessel	Move the electrode more lateral; if you don't get an AP after this, get a new worm
There is a stimulus artifact, but no visible response to stimuli between 1–3 V	Your electrodes may not be placed well, or your worm may be damaged	Try to reposition the electrodes on a different place in the worm. If this doesn't work, get a new worm.
My recording just looks like a big up and down squiggle (like a sine wave)	You're picking up 60 Hz noise	Ground everything (Faraday cage, ground recording pin, amplifier) to the ground pin on the back of the PowerLab. Averaging multiple trials can also get rid of noise.

Possible opportunities for student projects

- Application of chemicals, nerve blockers, etc. to test for effects on the nervous system
- Earthworm dissection & recordings (as in Gunther, 1976 or 1972)
- Demonstrating facilitation of conduction rate (Bullock, 1951)

Drosophila Behavior & Optogenetics

Background & Goals

Drosophila as a model in neuroscience

With just over 100,000 neurons but incredible genetic tractability, the fruit fly is an important model organism in neuroscience. About 65% of human disease-associated genes have a homolog in *Drosophila melanogaster* (Ugur et al., 2016). Like many model organisms, fruit flies are easy to reproduce, cheap, and easy to take care of in a lab. On top of that, they've been studied for more than a century. *Drosophila* have been used to study learning, memory, vision, sleep, addiction, courtship, and aggression, among many other behaviors.

In these labs, we'll aim to characterize the gustatory, ethological, and locomotor behavior of transgenic fruit flies, solving a few mysteries along the way.

What's coming up

This module spans **two days** of lab:

- [Day 1: The Case of the Mislabeled Fly Vials](#) — three behavioral assays (proboscis extension reflex, ethological observation, and locomotor tracking) to characterize an unknown transgenic line.
- [Day 2: The Case of the Missing Methods](#) — design your own optogenetic experiment to figure out what the genetically modified neurons control.

In these labs you will:

- Operationalize and quantify the behavior of a fruit fly
- Use a computer vision program to track locomotor behaviors
- Identify different types of transgenic flies used for neuroscience research
- Design an optogenetic experiment to interrogate neural circuit function

Relevant references

- Al-Anzi, B., Tracey, W. D., & Benzer, S. (2006). Response of *Drosophila* to wasabi is mediated by *painless*, the fly homolog of mammalian TRPA1/ANKTM1. *Current Biology* 16(10): 1034–1040.
- Chen, Y.-C. D., & Dahanukar, A. (2020). Recent advances in the genetic basis of taste detection in *Drosophila*. *Cellular and Molecular Life Sciences* 77(6): 1087–1101.
- Devineni, A. V., & Heberlein, U. (2009). Preferential ethanol consumption in *Drosophila* models features of addiction. *Current Biology* 19(24): 2126–2132.
- Mandel, S. J., Shoaf, M. L., Bostwick, J. R., Combs, D. J., Trudeau, V. L., & Johnson, E. C. (2018). Behavioral aggression is regulated by *Drosophila* CCAP signaling. *Frontiers in Neural Circuits* 12: 45.
- Nakamura, M., Baldwin, D., Hannaford, S., Palka, J., & Montell, C. (2002). Defective proboscis extension response (DPR), a member of the Ig superfamily required for the gustatory response to salt. *Journal of Neuroscience* 22(9): 3463–3472.
- Trisal, S., VijayRaghavan, K., & Ramaswami, M. (2023). The proboscis extension reflex assay for evaluating taste responses in *Drosophila*. *Bio-protocol* 13(20).
- Ugur, B., Chen, K., & Bellen, H. J. (2016). *Drosophila* tools and assays for the study of human diseases. *Disease Models & Mechanisms* 9(3): 235–244.

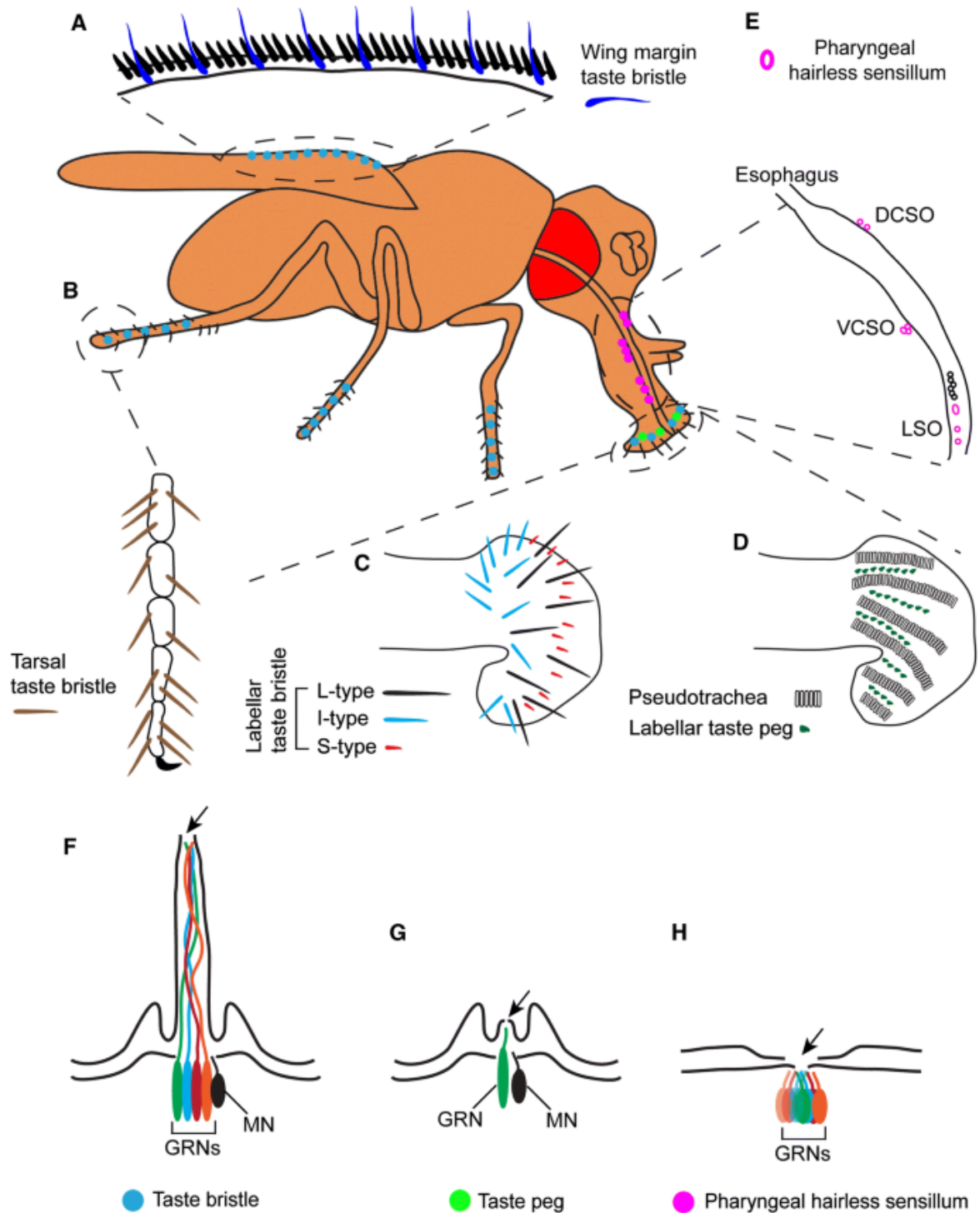
Day 1: The Case of the Mislabeled Fly Vials

Something terrible has happened in the lab. The flies you once knew, that were once perfectly organized and labeled, are now completely jumbled. You're pretty sure the flies are mutated, in one way or another. It's up to you to figure out what type of flies these are, based on three behavioral assays: a **proboscis extension reflex (PER) assay**, an **ethological survey**, and a **locomotion test**.

Everyone will receive one batch of wildtype flies, and one batch of an experimental group. Ultimately, **you will present your findings to the class** (see Canvas for details) so that we can determine the phenotype of our mutant flies.

Supplies	Solutions
Two vials of flies (~10 WT, ~10 Mutant/Knockout)	DI water
Dissecting microscope	100 mM sucrose solution
Paintbrush	40% EtOH in 100 mM sucrose solution
Large petri dish (lid or base)	
Clear cyanoacrylate glue	
4 mL transfer pipette	
Coverslips (6 per fly group)	
Paper towels & KimWipes	
Plastic "fly recovery" box	
Petri dish with sylgard bottom	
Magnetic stand & camera	

Test I. Proboscis Extension Reflex (PER) Assay



The **proboscis extension** is the reflexive extension of the mouthpart (proboscis) in response to taste stimuli. It is a fundamental feeding behavior that can be stimulated by exposing the antennae, labellar hairs, or tarsi (last segments of the leg) to an appetitive taste stimulus.

While a solution with a low concentration of ethanol is appetitive to fruit flies (serving as an olfactory cue that plant-material fermentation has begun), higher concentrations (>10% EtOH) can be aversive and toxic to fruit flies. In this assay, you'll compare your wildtype flies to a knockout group across three solutions: sucrose (an appetitive stimulus), DI water (a neutral control), and 40% EtOH in sucrose (an aversive stimulus).

Immobilization

1. Place fly tubes on ice for 2–3 minutes until flies are immobile.

Note: Keep a close eye on your flies and do not leave them on ice too long! As soon as they're not moving, you can transfer them.

2. Place the bottom of the petri dish upside down on ice. Wipe away condensation before placing flies on the dish.
3. Using a brush, *very gently* isolate one fly onto the petri dish and *carefully* orient the fly so it is dorsal-side up (Figure E below). Be gentle — the flies are very fragile.
4. Prepare a coverslip: using the transfer pipette, place a tiny drop (1 μ l, <2 mm in diameter) of glue on a coverslip.



5. Gently lower the coverslip onto the dorsal side of the fly, until just submerging the back and wings, immobilizing the fly in the glue (Figures F–G above).

Note: Only the back should be submerged in the glue. The head, proboscis, and at least one leg should be exposed — if any one of those parts is submerged, try again with another fly. If you've only submerged the wings, try flipping the coverslip over and using forceps to gently lower the back into the glue.

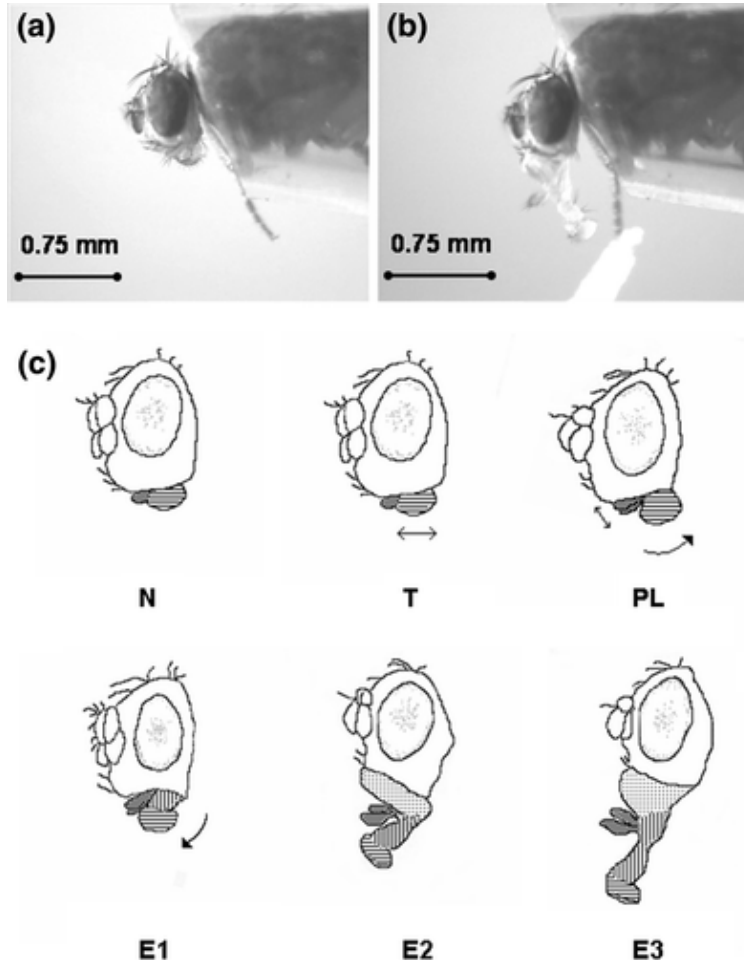
6. Place one damp paper towel in the plastic container on your lab bench. Ensure the box is dry. Place the coverslip in the box and close the container to regulate humidity.
7. Repeat with the remaining flies. You should prepare **5 wildtype** and **5 knockout** flies.
8. Allow the flies to recover for at least **1.5 hours** before testing. Complete the other behavioral tests during the recovery period.

Note: Omit flies that do not show movement after 1.5 hours of recovery.

Running the PER assay

9. Choose one of your responsive, immobilized flies with accessible head and legs, and place it under the dissecting microscope. Set up your lighting so you can see the legs and proboscis.

10. Dip the corner of a KimWipe in the sucrose solution and gently touch the tarsi with the KimWipe for 3 seconds.



- a. Score the response in **Data Table 1**. Assign a **1** for proboscis extension (right image above) and a **0** for no extension (left image above).
- b. Wait 3 seconds or until the proboscis retracts.
- c. Repeat for **10 trials**.
11. Repeat using **DI water**.
12. Repeat using **40% EtOH in sucrose**.

Data Table 1: PER Responses

Genotype	Trial	Sucrose	DI Water	40% EtOH in Sucrose
Wildtype	1			
Wildtype	2			
Wildtype	3			
Wildtype	4			
Wildtype	5			
Wildtype	6			
Wildtype	7			
Wildtype	8			
Wildtype	9			
Wildtype	10			
Knockout	1			
Knockout	2			
Knockout	3			
Knockout	4			
Knockout	5			
Knockout	6			
Knockout	7			
Knockout	8			
Knockout	9			
Knockout	10			

Reflection Question: Identify the positive and negative controls in this experiment. With these controls in mind, are your findings as expected?

Test II. Ethological approach to behavior

It's possible your flies do something different that's *not* captured by a standard behavioral assay. The best way to figure this out is to watch the flies, choose several behaviors to quantify, and see if those behaviors differ between your experimental and wildtype flies.

Choosing behaviors to quantify

Your next goal for today will be to quantify **three** different behaviors that your flies perform. It is up to you to define and operationalize these behaviors.

- 1. Freeze both groups of flies for 2–3 minutes until they stop moving.
- 2. Isolate three flies of each type in separate petri dishes.
Note: Make sure the flies are completely awake and moving before moving on to the next step. This could take about 10 minutes.
- 3. Take a few minutes to observe both batches of flies.
- 4. Choose three different behaviors that the flies do, and record them below. You also need to provide a definition for each behavior.

Table 1.

Behavior name	Definition	# Instances in WT	# Instances in Mutant

- 5. For 5 minutes, each person in your group should observe one behavior and record the number of instances those behaviors happen in the WT flies. Record the number of instances in the table above.
- 6. Do the same for the experimental group.

Test III. Recording locomotor behavior

For the final test, you'll record the movement of your two different types of flies in a petri dish and use an automated tracking program to track the velocity and heading of your flies. Your mutants might have altered locomotor behavior — this will help you test that.

Set up your camera

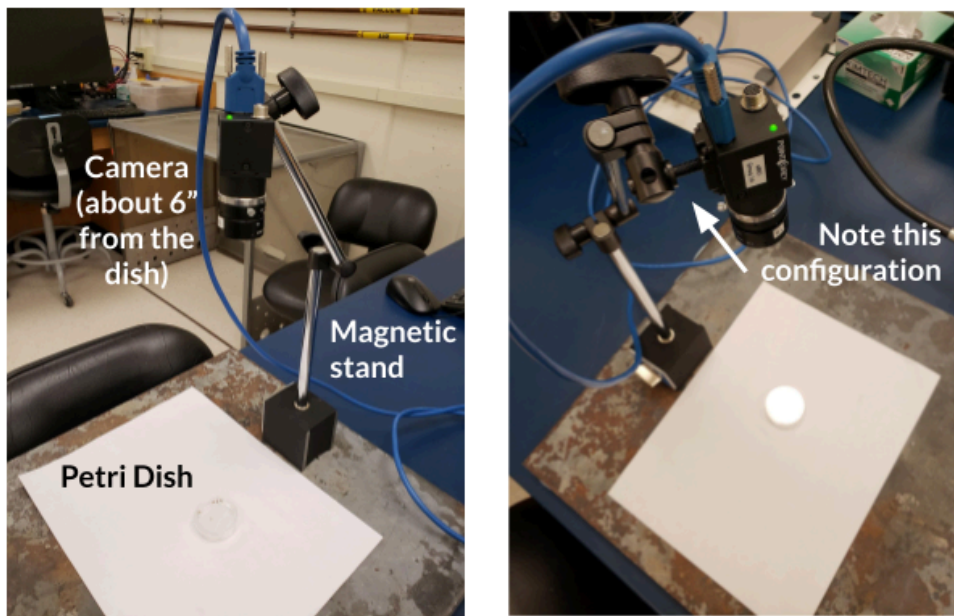


Figure 1. Fly locomotor recording setup.

First, we need to arrange the camera and petri dish so that you can get everything in frame, in focus, and well lit.

1. Ensure that your camera is connected via the USB cord to the computer. The camera should be plugged into a USB 3.0 slot (the blue ones).
2. Set up your camera and petri dish as in the figure above. You want your camera to be about 6 inches from the petri dish.
3. On the computer, open **Point Grey FlyCap2**.
4. Make sure that it recognizes a "Flea3" camera, and press OK.
5. Adjust the **focal length** knob on your camera (closest to end, ranges from infinity to 0.1) in order to get the image in focus.
6. Adjust the image on your screen using the toolbar at the top:

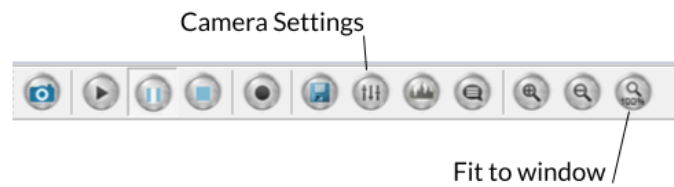
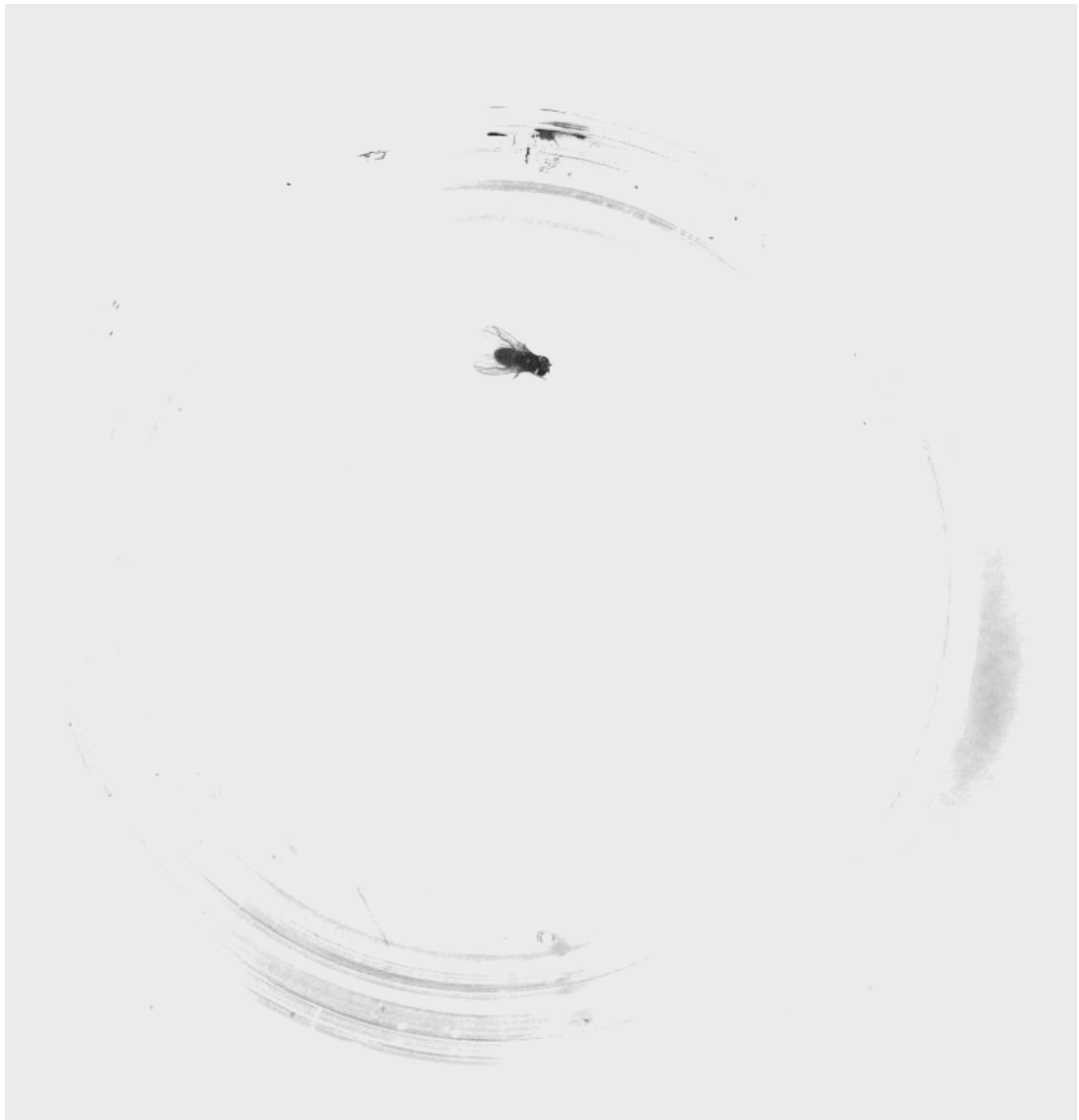


Figure 2. FlyCap menu bar.

7. Isolate one fly in your petri dish.
8. Click on the button to open up the Camera settings (e.g., exposure).
 - a. Uncheck *auto* for brightness, exposure, and shutter.
 - b. Increase the exposure almost to the point where your image is overexposed. You want the flies to stand out very clearly against the white background, and the edges of the petri dish to be invisible:



9. Ensure that you can easily see your fly throughout all areas of the arena.

Note: Optimizing the way your fly recording looks will make the tracking *much* easier. Make sure there are no glares from lights in the room, no dark spots on the petri dish, and clean off any fingerprints on the glass. **Every light and dark spot matters, and anything can throw off the tracking.**

Set up your flies & record

Recording Settings

Save Filename
 Filename: C:\Users\ajuavine\Videos\fly_test
 A timestamp, sequence number and file extension based on output type will be appended Browse

Saving Options
☐ Capture 0 frames*
☐ Capture 1 out of every frames from a run of frames*
☒ Capture for 60000 ms
☐ Capture once out of every ms from a run of ms*
 * Specify zero to capture until manually stopped

Recording Mode
☒ **Buffered:** Frames are buffered in physical memory while recording. Recording will stop if physical memory usage hits 95%
☐ **Streaming:** Frames are not buffered in system memory while recording. Dropped frames may occur depending on the speed of the PC and the current camera/recording settings.

☐ Only save corrupt frames when capturing

Images Videos

Video Recording Type: M-JPEG
 Frame Rate: Use Camera Frame Rate
 AVI Split Size: 0 MB (0 indicates no file split)
 M-JPEG Options
 JPEG Compression Quality (1-100): 75

Good Frames:	1198	Available Physical Memory:	12522.50 MB
Corrupt Frames:	0	Total Physical Memory:	16304.59 MB
Buffered Frames:	0	Usage:	23%
Saved Frames:	1198		

Hide Recording is stopped Start Recording

1. When you're ready to record, go to **File > Capture Image or Video**.
2. A dialogue will pop up. Set up a 60 second recording with the following settings:
 - **Filename:** browse to create your own (make sure you label it as WT or Knockout). **SAVE YOUR FILE LOCALLY**, not on your username server. **Your video file will not save correctly if you do not do this.**
 - ☒ Capture for 60000 ms
 - ☒ Buffered
 - (Click on **Videos** tab)
 - **Video Recording Type:** M-JPEG
 - **Frame Rate:** 15.0
 - **AVI Split Size:** 0
 - **JPEG Compression Quality:** 75

3. Press **Start Recording**, and wait while the program records the video.
Note: This window will not close, but the message at the bottom will change.
4. When it is done, **check that the video recorded correctly by finding the file and opening it.**
Note: If it didn't record correctly, close FlyCap, unplug the camera, and retry.
5. For the first fly you record, **move on to the next step to ensure that your video works well for tracking. If the tracking doesn't work well, you'll need to adjust your lighting and/or image acquisition.**
6. When you're confident the tracking works well with your videos, obtain videos for three flies from each group (you've already done one).
7. When you're done, save your videos in a clouded location for safekeeping.
8. Close FlyCap.

Quantify your locomotor behavior

Now that you have videos of your specimen, you need to quantify the locomotor behavior. You can do so using this notebook, which will generate means and standard deviations for each video:

<https://bipn145.github.io/FlyTracking/ColabNotebook.html>

Record those values in the table below (you'll ultimately want to plot these).

Table 2. Comparison of mean velocities for control and mutant flies.

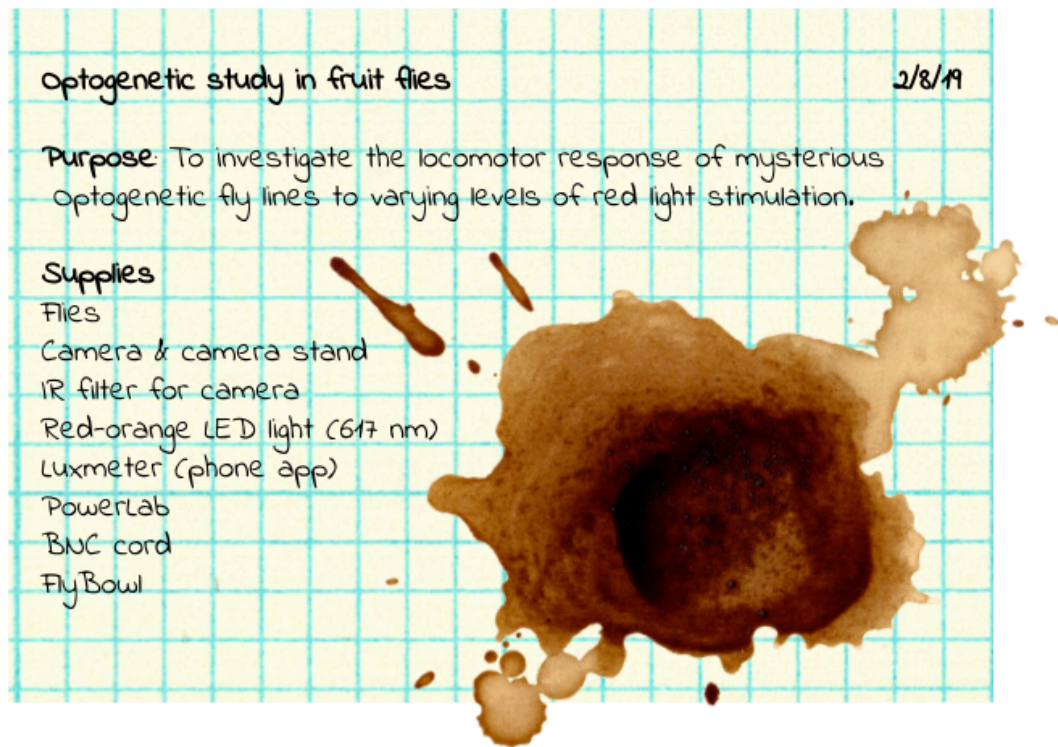
	Control	Mutant
Mean		
SEM		

Troubleshooting

Observation	Likely issue(s)	Possible solution
The camera output is black	Lens cap is on	Remove the lens cap
Camera not recognized	Camera not plugged in	Plug in the camera
Video file is not recognized in MATLAB or the movie is the wrong length/size	You wrote the video file to the server	You need to re-record your video but with the file saved directly to the computer locally
Fly does not extend proboscis to sucrose	Fly is dehydrated or improperly mounted	Check humidity in recovery box; verify proboscis and at least one leg are not stuck in glue
All flies look the same in tracking video	Lighting/exposure issues	Increase exposure so flies stand out as dark spots on a fully white background

Day 2: The Case of the Missing Methods

Just when you thought everything was back to normal in the lab, a remarkably inconveniently placed coffee spill completely drenches several pages of the notes given to you from your mentor. Unfortunately, those pages detailed all of the methods that you needed in order to perform your experiment. You've managed to save the supplies list, at least:



Hope is not lost. You've got some supplies, and you suspect that this study had something to do with the role of the genetically-modified neurons. Certain neurons have optogenetic channels, but what do those neurons actually control?

It might take a little bit of research to refresh your memories about those transgenic flies, but that's easy enough. And you've done at least some fly behavior before — you're pretty much the fly whisperer.

You can re-write the protocol and get some interesting results, can't you?

Tips for designing your experiment

You can control the intensity of the light source through LabChart by following these steps:

1. Plug the BNC input on the light into the **OUTPUT** of the PowerLab using a BNC cable.
2. Program the stimulator (**Setup > Stimulator**) to send different pulses with heights from 2V – 10V. You should notice that the intensity of the light changes with more voltage.

Do not stimulate your flies at higher than 1 Hz frequency or with stimulus pulses longer than 0.5 seconds.

3. Measure the intensity of the light using a phone app luxmeter (we recommend *Lux Light Meter Free*). You should measure the light intensity **from the same distance** as you are using it in the experiment.

Don't forget about control groups! It may help to compare these flies to each other, as well as to wildtype flies. We have wildtype flies if you need them.

You will need to put the red light **within several inches** of your flies in order to see an effect of the stimulation.

Fly Tracker (Local)

In this lab, you'll use a Python script to track the position and velocity of a fruit fly from video recordings. The script will detect the fly against a light background, track its movement frame-by-frame, and output position and velocity data as CSV files, along with plots.

At the end of this notebook, you'll be able to:

- Run the fly tracker script on your own video recordings
- Load and visualize tracking and velocity data in Python
- Customize your plots for your lab report

Part 1: Running the Fly Tracker Script

The fly tracker is a Python script that you'll run from the command line. Follow the steps below to get set up.

Step 1: Open Anaconda Prompt

1. Click the **Windows search bar** (bottom-left of your screen) and type **Anaconda**.
2. Click **Anaconda Prompt** to open it. You should see a black terminal window.

Note

This is intended to be run on the BIPN 145 Windows lab computers in York 1310. If you are running this on your own Mac computer, you can search for Python and run it that way. If you're running it on your own Windows computer, you'll need to download Python or Anaconda.

Step 2: Download the Scripts

In the Anaconda Prompt, first use the following to move into *your* folder. Replace USER with the UCSD username (not including @ucsd.edu) for the person that is logged into the computer.

Tip

Copying and pasting into the Anaconda Prompt: You can copy commands from this page by clicking the copy button () at the top-right of each code block and using Ctrl+V to paste into the command line.

```
cd C:\Users\USER
```

Then, run the following commands to download the setup and fly tracker scripts:

```
curl -o setup.py https://raw.githubusercontent.com/ajuavinett/fly_tracker/master/set
```

```
curl -o BIPN145_flytrack.py https://raw.githubusercontent.com/ajuavinett/fly_tracker
```

If you already have older versions of these scripts, this will overwrite them with the latest versions. You can also download the scripts manually from the [fly_tracker GitHub page](#).

Step 3: Install Required Packages

Run the setup script to install the required Python packages. You only need to do this **once** per computer:

```
python setup.py
```

You should see a message confirming that each package is installed. Once you see "Setup complete!", you're ready to go.

Step 4: Know Where Your Video File(s) Are

Make sure you know where your fly video file(s) (`.avi`, `.mp4`, etc.) are saved on your computer. When you run the script, a file picker window will pop up and you can navigate to them from there.

Step 5: Run the Script

In the Anaconda Prompt, type:

```
python BIPN145_flytrack.py
```

The script will walk you through everything interactively:

1. **Select your video(s)** — A file picker dialog will pop up. Navigate to your video file(s), select them, and click Open.
2. **Draw the ROI** — A window will pop up showing the first frame of your video. **Click and drag** to draw a rectangle around the dish/arena (and *only* around the dish – make it as snug as possible!), then press **Enter** or **Space** to confirm. Press **C** to cancel and redraw.
3. **Wait for tracking** — The script will process each frame. You'll see progress updates (10%, 20%, etc.). This will take a few minutes to run per video.
4. **View results** — When done, plots will appear will save automatically along with CSV files containing the tracking results. You can either use the plots that are saved or modify them (see below).

Step 6: Find Your Output Files

After the script finishes, you'll find two CSV files for each video in the same folder as your video:

File	Contents
<code>yourvideo_tracking.csv</code>	Time (s), X position (cm), Y position (cm)
<code>yourvideo_velocity.csv</code>	Time (s), Velocity (mm/s)

If you need to edit them, you can open these in Excel, Google Sheets, or load them into Python (see below).

Part 2: Working with Your Data in Python (Optional)

The cells below will help you load your CSV output files and create customized plots in case you need to adjust anything after running the tracking. This is useful if you want to adjust axis labels, colors, or combine data from multiple files for your lab report.

Task: Run the cell below to import the packages we'll need.

```
import numpy as np
import matplotlib.pyplot as plt
from matplotlib.collections import LineCollection

print('Packages imported!')
```

Packages imported!

Upload your CSV files

If you're running this notebook in **Google Colab**, run the cell below to upload your tracking and velocity CSV files. If you're running locally, you can skip this cell and just set the file paths directly.

Task: Run the cell below and upload your `_tracking.csv` and `_velocity.csv` files.

```
try:
    from google.colab import files
    uploaded = files.upload()
    filenames = list(uploaded.keys())
    tracking_file = [f for f in filenames if 'tracking' in f][0]
    velocity_file = [f for f in filenames if 'velocity' in f][0]
    print(f'Tracking file: {tracking_file}')
    print(f'VeLOCITY file: {velocity_file}')
except ImportError:
    # Running locally - set your file paths here
    tracking_file = 'yourvideo_tracking.csv' # <-- change this
    velocity_file = 'yourvideo_velocity.csv' # <-- change this
    print(f'Using local files: {tracking_file}, {velocity_file}')
```

Using local files: yourvideo_tracking.csv, yourvideo_velocity.csv

Load the data

Now let's load both CSV files using `numpy`. Each file has a header row, so we skip it with `skiprows=1`.

Task: Run the cell below to load your data.

```
# Load tracking data: columns are Time (s), X (cm), Y (cm)
tracking = np.loadtxt(tracking_file, delimiter=',', skiprows=1)
time_pos = tracking[:, 0]
x_pos = tracking[:, 1]
y_pos = tracking[:, 2]

# Load velocity data: columns are Time (s), Velocity (mm/s)
vel_data = np.loadtxt(velocity_file, delimiter=',', skiprows=1)
time_vel = vel_data[:, 0]
velocity = vel_data[:, 1]

print(f'Tracking data: {len(time_pos)} frames')
print(f'Velocity data: {len(time_vel)} time bins')
print(f'Mean velocity: {np.nanmean(velocity):.2f} mm/s')
print(f'SD velocity: {np.nanstd(velocity):.2f} mm/s')
```

```

-----
FileNotFoundError                                Traceback (most recent call last)
Cell In[3], line 2
      1 # Load tracking data: columns are Time (s), X (cm), Y (cm)
----> 2 tracking = np.loadtxt(tracking_file, delimiter=',', skiprows=1)
      3 time_pos = tracking[:, 0]
      4 x_pos = tracking[:, 1]

File /opt/miniconda3/lib/python3.12/site-packages/numpy/lib/_npio_impl.py:1395, in
1392 if isinstance(delimiter, bytes):
1393     delimiter = delimiter.decode('latin1')
-> 1395 arr = _read(fname, dtype=dtype, comment=comment, delimiter=delimiter,
1396               converters=converters, skiplines=skiprows, usecols=usecols,
1397               unpack=unpack, ndmin=ndmin, encoding=encoding,
1398               max_rows=max_rows, quote=quotechar)
1400 return arr

File /opt/miniconda3/lib/python3.12/site-packages/numpy/lib/_npio_impl.py:1022, in
1020     fname = os.fspath(fname)
1021 if isinstance(fname, str):
-> 1022     fh = np.lib._datasource.open(fname, 'rt', encoding=encoding)
1023     if encoding is None:
1024         encoding = getattr(fh, 'encoding', 'latin1')

File /opt/miniconda3/lib/python3.12/site-packages/numpy/lib/_datasource.py:192, in o
155 """
156 Open `path` with `mode` and return the file object.
157
(...)    188
189 """
191 ds = DataSource(destpath)
--> 192 return ds.open(path, mode, encoding=encoding, newline=newline)

File /opt/miniconda3/lib/python3.12/site-packages/numpy/lib/_datasource.py:529, in D
526     return _file_openers[ext](found, mode=mode,
527                               encoding=encoding, newline=newline)
528 else:
--> 529     raise FileNotFoundError(f"{path} not found.")

FileNotFoundError: yourvideo_tracking.csv not found.

```

Plot the fly path

The plot below shows the fly's path through the arena, color-coded by time. Earlier positions are shown in purple/blue, and later positions in yellow/green.

Task: Run the cell below to plot the fly path. Edit the axis labels, title, or colormap (`cmap`) if you'd like to customize it.


```

# Set the diameter of your dish (should match what you used in the script)
diameter = 4 # cm

fig, ax = plt.subplots(figsize=(6, 6))

# Create color-coded path
valid = ~np.isnan(x_pos) & ~np.isnan(y_pos)
points = np.array([x_pos[valid], y_pos[valid]]).T.reshape(-1, 1, 2)
segments = np.concatenate([points[:-1], points[1:]], axis=1)

lc = LineCollection(segments, cmap='viridis', linewidth=2)
lc.set_array(time_pos[valid][:-1])
ax.add_collection(lc)

ax.set_xlim(0, diameter)
ax.set_ylim(diameter, 0) # Inverted to match video orientation
ax.set_aspect('equal')

# --- Customize these labels for your report ---
ax.set_xlabel('X-coordinate (cm)', fontsize=12)
ax.set_ylabel('Y-coordinate (cm)', fontsize=12)
ax.set_title('Fly Path', fontsize=14)

cbar = fig.colorbar(lc, ax=ax, orientation='horizontal', pad=0.1)
cbar.set_label('Time (s)')

plt.tight_layout()
plt.show()

```

Plot velocity over time

The plot below shows how the fly's velocity changes over the course of the recording.

Task: Run the cell below to plot the velocity trace. Edit the labels, colors, or axis limits as needed.

```

fig, ax = plt.subplots(figsize=(10, 4))

ax.plot(time_vel, velocity, linewidth=1.5, color='steelblue')

# --- Customize these for your report ---
ax.set_xlabel('Time (s)', fontsize=12)
ax.set_ylabel('Velocity (mm/s)', fontsize=12)
ax.set_title('Fly Velocity Over Time', fontsize=14)

# Uncomment the line below to set custom axis limits:
# ax.set_xlim(0, 60) # e.g., first 60 seconds
# ax.set_ylim(0, 20) # e.g., max 20 mm/s

plt.tight_layout()
plt.show()

print(f'Mean velocity: {np.nanmean(velocity):.2f} mm/s')
print(f'SD velocity: {np.nanstd(velocity):.2f} mm/s')

```

Compare multiple flies *(optional)*

If you have velocity data from multiple flies, you can plot them together and compute summary statistics.

Task: Upload additional velocity CSV files and add their filenames to the list below.

```
# Add your velocity CSV filenames here
velocity_files = [
    'fly1_velocity.csv', # <-- change these
    'fly2_velocity.csv',
    # 'fly3_velocity.csv',
]

fig, ax = plt.subplots(figsize=(10, 5))
all_means = []

for i, vf in enumerate(velocity_files):
    data = np.loadtxt(vf, delimiter=',', skiprows=1)
    t = data[:, 0]
    v = data[:, 1]
    ax.plot(t, v, linewidth=1.5, label=f'Fly {i + 1}')
    all_means.append(np.nanmean(v))

ax.set_xlabel('Time (s)', fontsize=12)
ax.set_ylabel('Velocity (mm/s)', fontsize=12)
ax.set_title('Velocity Comparison', fontsize=14)
ax.legend()
plt.tight_layout()
plt.show()

print(f'\nMean velocity across flies: {np.mean(all_means):.2f} mm/s')
print(f'SD across flies: {np.std(all_means):.2f} mm/s')
```

About this Notebook

This notebook was created by [Ashley Juavinett](#) for classes at UC San Diego. The fly tracker script is based on MATLAB code by Jeff Stafford.

Fly Tracker (Colab)

This notebook tracks a fruit fly against a light background in a video, calculates its position and velocity over time, and produces path and velocity plots.

Notebook steps

1. Run the **Setup** cell to import packages.
2. **Mount your Google Drive** and copy the path to your video file.
3. Create the **Helper Functions**.
4. Run the ROI cell and **draw your ROI** on the first frame, then confirm it.
5. Run the **Track Fly** cell and wait for it to finish.
6. Run the **Plot** and **Velocity** cells to view results.
7. Re-run steps 3-7 if you would like to track an additional video.

Setup

```
#@title Install & import packages { display-mode: "form" }
import numpy as np
import cv2
import matplotlib.pyplot as plt
import matplotlib.colors as mcolors
from matplotlib.collections import LineCollection
from scipy.spatial.distance import euclidean
from google.colab import files, output
from IPython.display import display, clear_output, HTML, Image as IPImage
import ipywidgets as widgets
import os
import base64

# Initialize result containers – each is created independently so adding
# new lists in future won't require a full session restart
# You can uncomment the following to reset all results if needed:
#all_corrected = []
#all_velocity = []
#all_fps = []
#all_video_paths = []
#print('Results cleared – ready to process new videos.')

if 'all_corrected' not in dir(): all_corrected = []
if 'all_velocity' not in dir(): all_velocity = []
if 'all_fps' not in dir(): all_fps = []
if 'all_video_paths' not in dir(): all_video_paths = []
print(f'Ready. {len(all_corrected)} video(s) processed so far.')

print('All packages loaded successfully!')
```

```
-----
ModuleNotFoundError                                Traceback (most recent call last)
Cell In[1], line 3
      1 #@title Install & import packages { display-mode: "form" }
      2 import numpy as np
----> 3 import cv2
      4 import matplotlib.pyplot as plt
      5 import matplotlib.colors as mcolors

ModuleNotFoundError: No module named 'cv2'
```

Mount your drive

Run the cell below to mount your drive, which should contain your video files.

If you'd prefer to upload your videos, you can do that instead (and do not need to run the cell below. See instructions [here](#).)

```
from google.colab import drive
drive.mount('/content/drive')
```

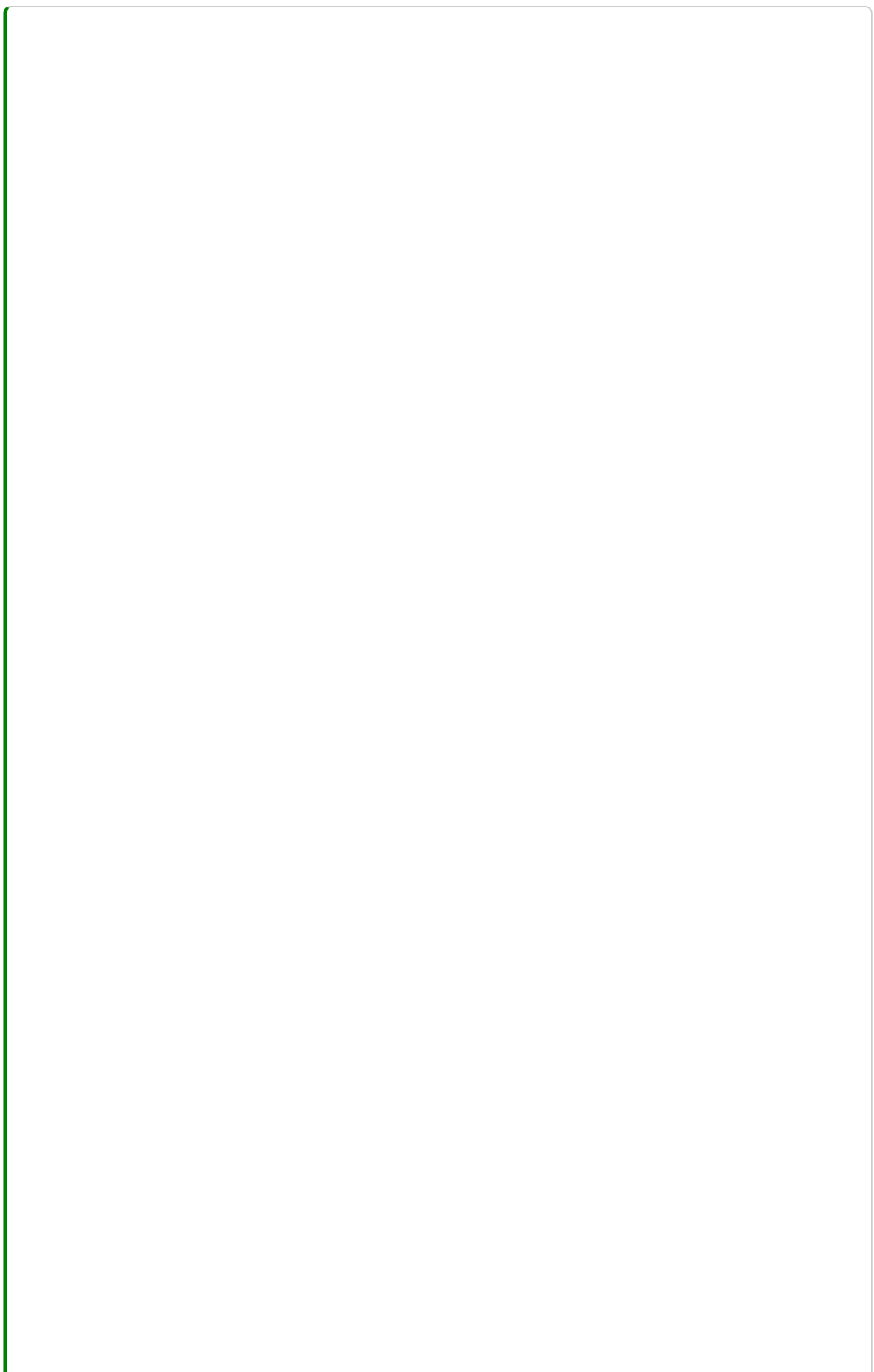
Select Video File

Update the path below to your video file. You'll process one video at a time, and results will accumulate in `all_corrected` and `all_velocity`.

Tip: In the window at left, you can use the three vertical dots to copy and paste the path to your video file.

```
# Set the current video file to process
current_video = '/content/drive/MyDrive/video1.avi'

if os.path.exists(current_video):
    size_mb = os.path.getsize(current_video) / 1e6
    print(f'File found: {os.path.basename(current_video)} ({size_mb:.1f} MB)')
else:
    print(f'File not found: {current_video}')
    print()
    print('Troubleshooting tips:')
    print('  1. Make sure you have run the "Mount your drive" cell above.')
    print('  2. In the left sidebar, click the folder icon to open the file browser.')
    print('  3. Navigate to your video file, right-click it, and select "Copy path".')
    print('  4. Paste the copied path above to replace the current_video value.')
    print()
    # Show video files in the same folder if it exists, to help spot typos
    folder = os.path.dirname(current_video)
    if os.path.exists(folder):
        videos = [f for f in os.listdir(folder)
                  if f.lower().endswith(('.avi', '.mp4', '.mov', '.mkv', '.mj2', '.m
        if videos:
            print(f'Video files found in {folder}:')
            for v in sorted(videos):
                print(f' {os.path.join(folder, v)}')
        else:
            print(f'No video files found in {folder} – check that you are looking in
    else:
        print(f'Folder not found: {folder}')
        print(' Make sure your Drive is mounted and the path is correct.')
```



```

#@title Define helper functions (fly_finder, dist_filter, interpolate_pos) { display
# --- Parameters ---
frame_rate = None # auto-detect from video (set e.g. frame_rate = 30 to override)
diameter = 4      # dish diameter in centimeters
search_size = 20   # fly search window size in pixels
per_pixel_threshold = 1.5
bin_size = 1       # velocity bin size in seconds
# -----

def fly_finder(roi_image, half_search, threshold, flip=True):
    """
    Find a fly (dark region) in a grayscale image.
    Locates the darkest pixel, retrieves a search area around it,
    and finds the center of pixel intensity.

    Returns (x, y) position or (NaN, NaN) if not found.
    """
    if flip:
        val = np.nanmin(roi_image)
    else:
        val = np.nanmax(roi_image)

    ys, xs = np.where(roi_image == val)
    xpos = np.mean(xs)
    ypos = np.mean(ys)

    h, w = roi_image.shape
    left = max(int(round(xpos) - half_search), 0)
    right = min(int(round(xpos) + half_search), w - 1)
    top = max(int(round(ypos) - half_search), 0)
    bottom = min(int(round(ypos) + half_search), h - 1)

    search_area = roi_image[top:bottom+1, left:right+1].astype(np.float64)

    if flip:
        search_area = 255.0 - search_area

    total = np.sum(search_area)

    if total >= threshold:
        x_indices = np.arange(search_area.shape[1])
        y_indices = np.arange(search_area.shape[0])
        x = np.sum(search_area @ x_indices) / total + left
        y = np.sum(search_area.T @ y_indices) / total + top
        return x, y
    else:
        return np.nan, np.nan

def dist_filter(array, tele_dist_threshold, num_avg=5):
    """
    Teleport filter: removes spurious points where fly position
    jumps far from the mean of surrounding frames.

    array: Nx3 array [time, x, y]
    """
    filtered = array.copy()
    n = len(filtered)
    x = filtered[:, 1]
    y = filtered[:, 2]

    def rolling_nanmean(arr, window, forward=False):

```

```

        """Rolling mean over 'window' preceding (or following) valid elements."""
        valid = (~np.isnan(arr)).astype(np.float64)
        filled = np.where(np.isnan(arr), 0.0, arr)
        cs_val = np.concatenate([[0.0], np.cumsum(filled)])
        cs_cnt = np.concatenate([[0.0], np.cumsum(valid)])
        idx = np.arange(n)
        if forward:
            starts = idx + 1
            ends = np.minimum(n, idx + 1 + window)
        else:
            starts = np.maximum(0, idx - window)
            ends = idx
        sum_v = cs_val[ends] - cs_val[starts]
        cnt_v = cs_cnt[ends] - cs_cnt[starts]
        with np.errstate(invalid='ignore', divide='ignore'):
            return np.where(cnt_v > 0, sum_v / cnt_v, np.nan)

    last_x = rolling_nanmean(x, num_avg)
    last_y = rolling_nanmean(y, num_avg)
    next_x = rolling_nanmean(x, num_avg, forward=True)
    next_y = rolling_nanmean(y, num_avg, forward=True)

    dist_last = np.sqrt((x - last_x) ** 2 + (y - last_y) ** 2)
    dist_next = np.sqrt((x - next_x) ** 2 + (y - next_y) ** 2)

    valid_pts = ~np.isnan(x)
    tele_mask = valid_pts & (
        (dist_last > tele_dist_threshold) | (dist_next > tele_dist_threshold)
    )
    tele_mask[:num_avg] = False
    tele_mask[n - num_avg:] = False

    filtered[tele_mask, 1:3] = np.nan
    tele_count = int(tele_mask.sum())

    edge_indices = list(range(0, min(5, n - 1))) + list(range(max(0, n - 6), n - 1))
    for idx in edge_indices:
        if np.any(np.isnan(filtered[idx, 1:3])) or np.any(np.isnan(filtered[idx + 1,
            continue
        if euclidean(filtered[idx, 1:3], filtered[idx + 1, 1:3]) > tele_dist_threshol
            filtered[idx, 1:3] = np.nan
            tele_count += 1

    print(f'{tele_count} points removed by the teleportation filter.')
    return filtered

def interpolate_pos(array, inter_dist_threshold):
    """
    Linearly interpolate fly position between NaN gaps,
    as long as the gap endpoints are within inter_dist_threshold.

    array: Nx3 array [time, x, y]
    """
    result = array.copy()
    interp_count = 0

    x = result[:, 1]
    y = result[:, 2]
    nan_mask = np.isnan(x)

    if not nan_mask.any():
        print('0 points recovered through interpolation.')
        return result

```

```

padded = np.concatenate([[False], nan_mask, [False]])
diff = np.diff(padded.astype(np.int8))
starts = np.where(diff == 1)[0]
ends = np.where(diff == -1)[0] - 1

for s, e in zip(starts, ends):
    last_idx = s - 1
    next_idx = e + 1
    if last_idx < 0 or next_idx >= len(result):
        continue
    last_pt = result[last_idx, 1:3]
    next_pt = result[next_idx, 1:3]
    if np.any(np.isnan(last_pt)) or np.any(np.isnan(next_pt)):
        continue
    if euclidean(last_pt, next_pt) <= inter_dist_threshold:
        gap = next_idx - last_idx
        fracs = np.arange(1, gap) / gap
        result[s:e + 1, 1] = last_pt[0] + fracs * (next_pt[0] - last_pt[0])
        result[s:e + 1, 2] = last_pt[1] + fracs * (next_pt[1] - last_pt[1])
        interp_count += e - s + 1

print(f'{interp_count} points recovered through interpolation.')
return result

def boundary_filter(array, diameter):
    """
    Remove points that fall outside the circular dish.
    """
    filtered = array.copy()
    cx, cy = diameter / 2, diameter / 2
    radius = diameter / 2
    x = filtered[:, 1]
    y = filtered[:, 2]
    dist_from_center = np.sqrt((x - cx) ** 2 + (y - cy) ** 2)
    outside = dist_from_center > radius
    outside = outside & ~np.isnan(x)
    filtered[outside, 1:3] = np.nan
    count = np.sum(outside)
    if count > 0:
        print(f' {count} points removed by the boundary filter (outside dish).')
    return filtered

def displacement_filter(array, multiplier=5):
    """
    Remove points where the frame-to-frame displacement is much larger
    than the median displacement. This catches sudden jumps that the
    teleport filter might miss (e.g., runs of bad frames).
    """
    filtered = array.copy()
    x = filtered[:, 1]
    y = filtered[:, 2]

    dx = np.diff(x)
    dy = np.diff(y)
    displacements = np.sqrt(dx ** 2 + dy ** 2)

    valid_disp = displacements[~np.isnan(displacements)]
    if len(valid_disp) == 0:
        return filtered
    median_disp = np.nanmedian(valid_disp)

    if median_disp == 0:
        return filtered

```



```

threshold = median_disp * multiplier
count = 0

for i in range(len(displacements)):
    if np.isnan(displacements[i]):
        continue
    if displacements[i] > threshold:
        filtered[i + 1, 1:3] = np.nan
        count += 1

if count > 0:
    print(f' {count} points removed by the displacement filter (>{multiplier}x
return filtered

def process_video(video_path, roi, diameter, frame_rate, search_size, per_pixel_thre
"""
Process a single fly video and return the corrected position array.
Returns (Nx3 array [time_s, x_cm, y_cm], frame_rate).
"""
height = diameter
width = diameter
roi_x, roi_y, roi_w, roi_h = roi

cap = cv2.VideoCapture(video_path)
fps = cap.get(cv2.CAP_PROP_FPS)
total_frames = int(cap.get(cv2.CAP_PROP_FRAME_COUNT))
nfrm = total_frames - 1

# Auto-detect frame rate from video if not specified
if frame_rate is None:
    if fps > 0:
        frame_rate = round(fps)
        print(f"\nProcessing: {os.path.basename(video_path)}")
        print(f" Auto-detected frame rate: {frame_rate} fps ({total_frames} fra
    else:
        frame_rate = 30
        print(f"\nProcessing: {os.path.basename(video_path)}")
        print(f" WARNING: Could not detect frame rate, using default: {frame_ra
        print(f" Override by setting frame_rate = 30 (or your value) in the S
else:
    print(f"\nProcessing: {os.path.basename(video_path)}")
    print(f" Video FPS: {fps}, using override: {frame_rate} fps ({total_frames}

# Sanity check: warn if video duration seems unusual
if frame_rate > 0:
    duration = total_frames / frame_rate
    print(f" Estimated duration: {duration:.1f} seconds")
    if duration < 50 or duration > 70:
        print(f" WARNING: Expected ~60s video but got {duration:.1f}s.")
        print(f" Override by setting frame_rate = 30 (or your value) in the S

# --- Create background from 100 random frames ---
print(" Calculating background...")
bg_number = min(100, nfrm)
bg_indices = sorted(np.random.choice(nfrm, bg_number, replace=False))

cap.set(cv2.CAP_PROP_POS_FRAMES, 0)
ret, sample = cap.read()
gray_sample = cv2.cvtColor(sample, cv2.COLOR_BGR2GRAY)
bg_array = np.zeros((*gray_sample.shape, bg_number), dtype=np.uint8)

for idx, frame_num in enumerate(bg_indices):
    cap.set(cv2.CAP_PROP_POS_FRAMES, frame_num)

```

```

    ret, frame = cap.read()
    if ret:
        bg_array[:, :, idx] = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)

background = np.mean(bg_array, axis=2).astype(np.uint8)
bg_scale = 256.0 / (background.astype(np.float64) + 1)
# Pre-crop bg_scale to ROI so per-frame math runs on ROI pixels only
bg_scale_roi = bg_scale[roi_y:roi_y + roi_h, roi_x:roi_x + roi_w]

# --- Process each frame ---
print(" Tracking fly positions...")
threshold = (search_size ** 2) * per_pixel_threshold
half_search = round(search_size / 2)

pos_array = np.zeros((nfrm, 3))

# Seek once; sequential cap.read() is far faster than per-frame seeking
# (compressed formats must decode from the last keyframe on every seek)
cap.set(cv2.CAP_PROP_POS_FRAMES, 0)
for nofr in range(nfrm):
    ret, frame = cap.read()
    if not ret:
        pos_array[nofr] = [nofr, np.nan, np.nan]
        continue

    # Crop to ROI before float conversion so background subtraction
    # operates on ROI pixels only, not the full frame
    frame_roi = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)[
        roi_y:roi_y + roi_h, roi_x:roi_x + roi_w
    ].astype(np.float64)
    frame_div = np.clip(frame_roi * bg_scale_roi, 0, 255).astype(np.uint8)

    # Find fly
    fx, fy = fly_finder(frame_div, half_search, threshold, flip=True)
    pos_array[nofr] = [nofr, fx, fy]

    # Progress update every 10%
    if (nofr + 1) % max(1, nfrm // 10) == 0:
        pct = (nofr + 1) / nfrm * 100
        print(f"    {pct:.0f}% complete ({nofr + 1}/{nfrm} frames)")

cap.release()

# --- Convert to real coordinates ---
xscale = width / roi_w
yscale = height / roi_h

corrected_array = np.column_stack([
    pos_array[:, 0] / frame_rate,
    pos_array[:, 1] * xscale,
    pos_array[:, 2] * yscale,
])

skipped = np.sum(np.isnan(corrected_array[:, 1]))
print(f"    {skipped} points skipped out of {nfrm}.")

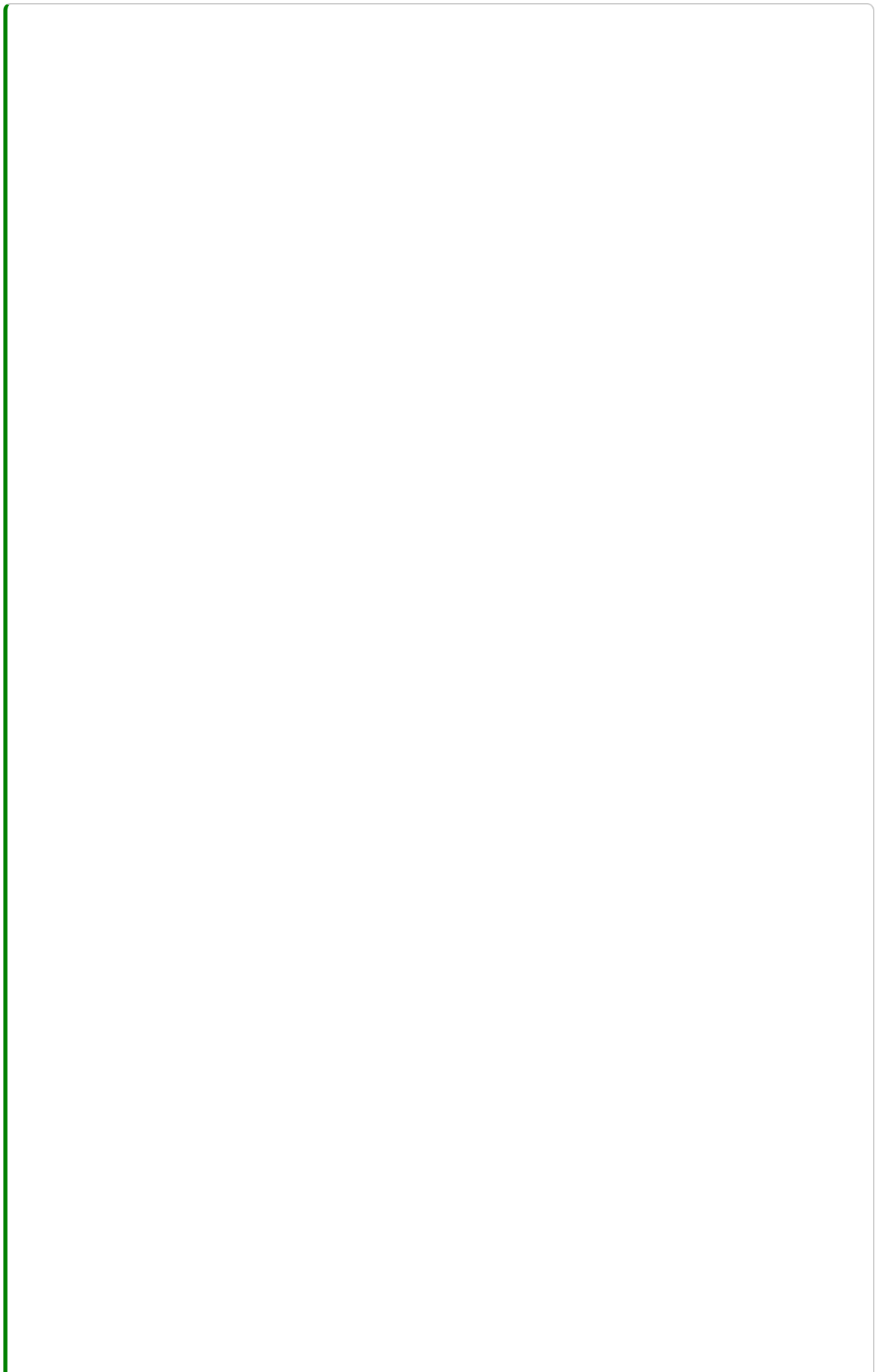
# Apply filters and interpolation
corrected_array = boundary_filter(corrected_array, diameter)
corrected_array = displacement_filter(corrected_array)
corrected_array = dist_filter(corrected_array, 2)
corrected_array = interpolate_pos(corrected_array, 2)

return corrected_array, frame_rate

```

```
print('Helper functions defined.')
```

The cell below is going to prompt you to select an ROI using a square. You want to select **the entire petri dish** (you may need to guess where the edges are if they are hard to see).



```

#@title Interactive ROI selector – click and drag on the image { display-mode: "form"

# Read first frame and encode as base64 for display
cap = cv2.VideoCapture(current_video)
ret, frame = cap.read()
cap.release()
if not ret:
    raise ValueError(f'Could not read first frame of {current_video}')

frame_rgb = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
img_h, img_w = frame_rgb.shape[:2]

# Encode frame as PNG base64
_, buf = cv2.imencode('.png', cv2.cvtColor(frame_rgb, cv2.COLOR_RGB2BGR))
img_b64 = base64.b64encode(buf).decode('utf-8')

print(f'Image size: {img_w} x {img_h} pixels')
print('Click and drag on the image below to select the ROI, then click "Confirm ROI"')

# JavaScript/HTML interactive ROI selector
html_code = f"""
<div id="roi-container" style="position:relative; display:inline-block; cursor:crosshair"
  <canvas id="roi-canvas" width="{img_w}" height="{img_h}"
    style="max-width:67%; height:auto; border:1px solid #ccc;"></canvas>
  </div>
<br>
<button id="roi-confirm-btn" style="font-size:16px; padding:8px 20px; background:#4CAF50;
  color:white; border:none; border-radius:4px; cursor:pointer; margin-top:8px;">
  Confirm ROI
</button>
<p id="roi-info" style="font-size:14px; color:#333;"></p>

<script>
(function() {{
  var canvas = document.getElementById('roi-canvas');
  var ctx = canvas.getContext('2d');
  var img = new window.Image();
  img.src = 'data:image/png;base64,{img_b64}';

  var drawing = false;
  var x0 = 0, y0 = 0, x1 = 0, y1 = 0;
  var hasROI = false;

  // Scale factor: canvas CSS size vs actual pixel size
  function getScale() {{
    var rect = canvas.getBoundingClientRect();
    return {{ sx: canvas.width / rect.width, sy: canvas.height / rect.height }};
  }}

  img.onload = function() {{
    ctx.drawImage(img, 0, 0);
  }};

  function redraw() {{
    ctx.clearRect(0, 0, canvas.width, canvas.height);
    ctx.drawImage(img, 0, 0);
    if (hasROI) {{
      var rx = Math.min(x0, x1);
      var ry = Math.min(y0, y1);
      var rw = Math.abs(x1 - x0);
      var rh = Math.abs(y1 - y0);
      ctx.strokeStyle = 'red';
      ctx.lineWidth = 3;
      ctx.strokeRect(rx, ry, rw, rh);
    }}
  }}
  window.addEventListener('click', function(e) {{
    if (e.target === canvas) {{
      if (!drawing) {{
        x0 = e.offsetX, y0 = e.offsetY;
        drawing = true;
      }} else {{
        x1 = e.offsetX, y1 = e.offsetY;
        if (x0 === x1 && y0 === y1) {{
          drawing = false;
        }} else {{
          redraw();
        }}
      }}
    }}
  }});
  window.addEventListener('mousedown', function(e) {{
    if (e.target === canvas && !drawing) {{
      x0 = e.offsetX, y0 = e.offsetY;
      drawing = true;
    }}
  }});
  window.addEventListener('mouseup', function(e) {{
    if (e.target === canvas && drawing) {{
      drawing = false;
      redraw();
    }}
  }});
  window.addEventListener('mousemove', function(e) {{
    if (e.target === canvas && drawing) {{
      redraw();
    }}
  }});
  window.addEventListener('keydown', function(e) {{
    if (e.key === 'Enter' && document.getElementById('roi-confirm-btn') === document.activeElement) {{
      document.getElementById('roi-confirm-btn').click();
    }}
  }});
})();

```

```

        document.getElementById('roi-info').innerText =
            'ROI: x=' + Math.round(rx) + ', y=' + Math.round(ry) +
            ', w=' + Math.round(rw) + ', h=' + Math.round(rh);
    }}
}}

canvas.addEventListener('mousedown', function(e) {{
    var rect = canvas.getBoundingClientRect();
    var scale = getScale();
    x0 = (e.clientX - rect.left) * scale.sx;
    y0 = (e.clientY - rect.top) * scale.sy;
    drawing = true;
    hasROI = false;
}});

canvas.addEventListener('mousemove', function(e) {{
    if (!drawing) return;
    var rect = canvas.getBoundingClientRect();
    var scale = getScale();
    x1 = (e.clientX - rect.left) * scale.sx;
    y1 = (e.clientY - rect.top) * scale.sy;
    hasROI = true;
    redraw();
}});

canvas.addEventListener('mouseup', function(e) {{
    drawing = false;
}});

document.getElementById('roi-confirm-btn').addEventListener('click', function() {{
    if (!hasROI) {{
        document.getElementById('roi-info').innerText =
            'No ROI drawn yet! Click and drag on the image first.';
        return;
    }}
    var rx = Math.round(Math.min(x0, x1));
    var ry = Math.round(Math.min(y0, y1));
    var rw = Math.round(Math.abs(x1 - x0));
    var rh = Math.round(Math.abs(y1 - y0));

    // Send ROI back to Python via Colab's kernel eval
    google.colab.kernel.invokeFunction('set_roi', [rx, ry, rw, rh], {{}});
    document.getElementById('roi-info').innerText =
        'ROI CONFIRMED: x=' + rx + ', y=' + ry + ', w=' + rw + ', h=' + rh;
    document.getElementById('roi-confirm-btn').style.background = '#888';
    document.getElementById('roi-confirm-btn').innerText = 'ROI Confirmed!';
    }});
}}})();
</script>
"""

# Register a callback so JavaScript can send the ROI to Python
roi_result = {'value': None}

def set_roi(x, y, w, h):
    roi_result['value'] = (int(x), int(y), int(w), int(h))
    print(f'ROI received: x={int(x)}, y={int(y)}, w={int(w)}, h={int(h)}')

output.register_callback('set_roi', set_roi)

display(HTML(html_code))

```

```
# Run this cell AFTER you have confirmed your ROI above
roi = roi_result['value']

if roi is None:
    raise ValueError("No ROI selected! Go back and draw a rectangle on the image, th

print(f'Using ROI: x={roi[0]}, y={roi[1]}, width={roi[2]}, height={roi[3]}')
```

Track Fly in Video(s)

This processes each video frame-by-frame:

1. Creates a background image from 100 random frames
2. Subtracts the background from each frame
3. Finds the fly position using center-of-mass of dark pixels
4. Applies teleportation filter and interpolation
5. Converts pixel positions to centimeters

```
# --- Process the current video ---
corrected, detected_fps = process_video(current_video, roi, diameter, frame_rate,
                                         search_size, per_pixel_threshold)

all_corrected.append(corrected)
all_fps.append(detected_fps)
all_video_paths.append(current_video)

print(f'\nDone! Total videos processed: {len(all_corrected)}')
print('To process another video, go back to "Select Video File" cell and change `cur
```

Plot Fly Path(s)

Color-coded by time (light blue = start, dark purple = end).

```

for idx, corrected in enumerate(all_corrected):
    x = corrected[:, 1]
    y = corrected[:, 2]
    t = corrected[:, 0]

    fig, ax = plt.subplots(figsize=(6, 6))

    # Plot path colored by time
    valid = ~np.isnan(x) & ~np.isnan(y)
    points = np.array([x[valid], y[valid]]).T.reshape(-1, 1, 2)
    segments = np.concatenate([points[:-1], points[1:]], axis=1)

    lc = LineCollection(segments, cmap='BuPu', linewidth=2)
    lc.set_array(t[valid][:-1])
    ax.add_collection(lc)

    ax.set_xlim(0, diameter)
    ax.set_ylim(diameter, 0) # Invert y to match video
    ax.set_aspect('equal')
    ax.set_xlabel('X-coordinate (cm)', fontsize=11)
    ax.set_ylabel('Y-coordinate (cm)', fontsize=11)
    ax.set_title(f'Fly Path - Video {idx + 1}')

    cbar = fig.colorbar(lc, ax=ax, orientation='horizontal', pad=0.1)
    cbar.set_label('Time (s)')

    plt.tight_layout()
    plt.show()

```


Calculate & Plot Velocity



```

# Calculate velocity for videos that haven't been processed yet
start_idx = len(all_velocity) # Only calculate for newly added videos

for idx in range(start_idx, len(all_corrected)):
    corrected = all_corrected[idx]
    x = corrected[:, 1]
    y = corrected[:, 2]

    fps = round(1.0 / corrected[1, 0]) if corrected[1, 0] > 0 else 30
    total_time = len(x) / fps
    total_bins = int(np.floor(total_time / bin_size))

    # Calculate velocity per bin
    data_rate = round(1.0 / corrected[1, 0]) * bin_size if corrected[1, 0] > 0 else
    data_rate = int(data_rate)

    if data_rate < 1:
        raise ValueError('bin_size is smaller than the minimum data rate.')

    velocity = np.zeros(total_bins)
    for row in range(0, len(corrected) - data_rate, data_rate):
        bin_idx = row // data_rate
        if bin_idx >= total_bins:
            break
        p1 = corrected[row, 1:3]
        p2 = corrected[row + data_rate, 1:3]
        if np.any(np.isnan(p1)) or np.any(np.isnan(p2)):
            velocity[bin_idx] = np.nan
        else:
            # 10x converts cm to mm
            velocity[bin_idx] = 10.0 * euclidean(p1, p2)

    # Convert from mm/bin to mm/s
    velocity = velocity / bin_size

    all_velocity.append(velocity)

    # Warn about absurd velocities
    if np.nanmax(velocity) > 30:
        print(f'WARNING (Video {idx + 1}): Absurdly high velocities detected.')
        print('  Consider changing the ROI or re-recording the video.')

    mean_vel = np.nanmean(velocity)
    std_vel = np.nanstd(velocity)
    print(f'\n--- Video {idx + 1} (from: {os.path.basename(current_video)}) ---')
    print(f'  Mean velocity: {mean_vel:.2f} mm/s')
    print(f'  Std deviation: {std_vel:.2f} mm/s')

    # Plot velocity over time
    time_axis = np.arange(total_bins) * bin_size
    fig, ax = plt.subplots(figsize=(10, 4))
    ax.plot(time_axis, velocity, linewidth=1.5)
    ax.set_xlim(0, time_axis[-1] + bin_size if len(time_axis) > 0 else 1)
    ax.set_ylim(0, np.nanmax(velocity) * 1.5 if np.nanmax(velocity) > 0 else 1)
    ax.set_xlabel('Time (s)', fontsize=11)
    ax.set_ylabel('Velocity (mm/s)', fontsize=11)
    ax.set_title(f'Fly Velocity - Video {idx + 1}')
    plt.tight_layout()
    plt.show()

print(f'\nVelocity analysis complete. Total videos analyzed: {len(all_velocity)}')

```

Summary Across All Videos

```
num_files = len(all_velocity)

if num_files > 1:
    # Pad velocity arrays to the same length for comparison
    max_len = max(len(v) for v in all_velocity)
    velocity_matrix = np.full((num_files, max_len), np.nan)
    for i, v in enumerate(all_velocity):
        velocity_matrix[i, :len(v)] = v

    # Plot all velocities together
    fig, ax = plt.subplots(figsize=(10, 5))
    time_axis = np.arange(max_len) * bin_size
    for i in range(num_files):
        ax.plot(time_axis, velocity_matrix[i], linewidth=2,
                label=f'Fly {i + 1}')
    ax.set_xlabel('Time (s)', fontsize=11)
    ax.set_ylabel('Velocity (mm/s)', fontsize=11)
    ax.set_title('All Fly Velocities')
    ax.legend()
    plt.tight_layout()
    plt.show()

    # Summary stats across videos
    per_video_means = [np.nanmean(v) for v in all_velocity]
    mean_across = np.mean(per_video_means)
    sd_across = np.std(per_video_means)
    print(f'\n=== Summary Across {num_files} Videos ===')
    print(f'Mean velocity across videos: {mean_across:.2f} mm/s')
    print(f'SD of mean velocity across videos: {sd_across:.2f} mm/s')
else:
    mean_vel = np.nanmean(all_velocity[0])
    sd_vel = np.nanstd(all_velocity[0])
    print(f'\n=== Summary (1 Video) ===')
    print(f'Mean velocity: {mean_vel:.2f} mm/s')
    print(f'SD of velocity: {sd_vel:.2f} mm/s')
```

Prepare for Another Video (Optional)

If you want to process another video with the same ROI:

1. Go back to the **Select Video File** cell and update `current_video` to the new file path
2. Re-run the video selection cell
3. Skip the ROI selection (you can reuse the same ROI with `roi = roi` or draw a new one)
4. Run the **Track Fly** cell again — results automatically append to `all_corrected`

Or, to apply a **different ROI** for the next video, re-run the ROI selection cells before tracking.

Download Results (Optional)

Download the tracking data as CSV files.

```
for idx, (corrected, vel) in enumerate(zip(all_corrected, all_velocity)):
    base = os.path.splitext(os.path.basename(all_video_paths[idx]))[0]

    csv_name = f'{base}_tracking.csv'
    np.savetxt(csv_name, corrected, delimiter=',',
               header='Time_s,X_cm,Y_cm', comments='')
    files.download(csv_name)
    print(f'Downloaded {csv_name}')

    time_axis = np.arange(len(vel)) * bin_size
    vel_csv = f'{base}_velocity.csv'
    np.savetxt(vel_csv, np.column_stack([time_axis, vel]), delimiter=',',
               header='Time_s,Velocity_mm_per_s', comments='')
    files.download(vel_csv)
    print(f'Downloaded {vel_csv}')
```

Measuring Alpha Wave Activity Using EEG

Background & Goals

The cerebral cortex contains huge numbers of neurons. Activity of these neurons is — to some extent — synchronized in regular firing rhythms. These are referred to as brain waves. Electrodes placed in pairs on the scalp can pick up variations in electrical potential that derive from this underlying cortical activity. This recording of the electrical activity in the cortex is called an electroencephalogram (EEG). EEG signals are strongly affected by different states of arousal and show characteristic changes in different stages of sleep. EEG signals are also affected by stimulation from the external environment and brain waves can become entrained to external stimuli. Electroencephalography is used, among other things, in the diagnosis of epilepsy and the diagnosis of brain death.

Recording EEG signals

EEG recording is technically difficult, mainly because of the small size of the voltage signals, which are typically 50 μV **peak-to-peak**. The signals are small because the recording electrodes are separated from the brain's surface by the scalp, the skull, and a layer of cerebrospinal fluid. A specially designed amplifier, such as the Bio Amp built into the PowerLab, is essential to record EEGs. It is also important to use electrodes made of the right material and to connect them properly. Even with these precautions, recordings may be spoiled by a range of unwanted interfering influences, known as artifacts.

In this lab, you will record EEG activity with an occipital electrode on the scalp at the back of the head. A third (ground or earth) electrode is also attached, to reduce electrical interference. In clinical EEG, it is usual to record many channels of activity from multiple recording electrodes placed in an array over the head.

Origins of the EEG signals

The EEG results from slow changes in the membrane potentials of cortical neurons, especially the excitatory and inhibitory post-synaptic potentials (EPSPs and IPSPs). Very little contribution normally comes from action potentials propagated along nerve axons. The EEG reflects the sum of the electrical potential changes occurring from large populations of cells. Therefore, large amplitude waves require the synchronous activity of a large number of neurons. The rhythmic events that these waves reflect often arise in the thalamus whose activity is in turn affected by a variety of inputs including structures in the brainstem reticular formation.

Components of the EEG waveform

The EEG waveform contains component waves of different frequencies. These can be extracted and provide information about different brain activities. The types of brain waves are:

- **alpha** (between 8 to 13 Hz; average amplitudes of 30 to 50 μV peak-to-peak), which will be studied in this experiment. Alpha rhythm is seen when the eyes are closed and the volunteer relaxed. It is abolished by eye opening and by mental effort such as doing calculations or concentrating on an idea. It is thus thought to indicate the degree of cortical activation. The greater the activation, the lower the alpha activity. Alpha waves are strongest over the occipital (back of the head) cortex and also over the frontal cortex.
- **beta** (13 to 30 Hz; <20 μV peak-to-peak), which are prominent in alert individuals with their eyes open. The beta rhythm may be absent or reduced in areas of cortical damage and can be accentuated by sedative-hypnotic drugs such as benzodiazepines and barbiturates.
- **theta** (4 to 8 Hz; <30 μV peak-to-peak), which are seen in awake children but not adults. The theta rhythm is normal during sleep at all ages. However, some researchers separate this frequency band into two components: low theta (4–5.45 Hz) activity correlated with

decreased arousal and increased drowsiness, and high theta (6–7.45 Hz) activity that is claimed to be enhanced during tasks involving working memory.

- **delta** (0.5 to 4 Hz; up to 100–200 μ V peak-to-peak), which is the dominant rhythm in sleep stages three and four but not seen in conscious adults. The delta rhythm tends to have the highest amplitude of any of the component EEG waves. EEG artifacts caused by movements of jaw and neck muscles can produce waves in the same frequency band.
- **gamma** (30 to 50 Hz). Most people recognize gamma rhythm, but its importance is controversial. It may be associated with higher mental activity, including perception and consciousness, and it disappears under general anesthesia. One suggestion is that the gamma rhythm reflects the mental activity involved in integrating various aspects of an object (color, shape, movement, etc.) to form a coherent picture.

Written by staff of ADInstruments & A. Juavinett. Copyright © 2015 ADInstruments Pty Ltd. All rights reserved. PowerLab® and LabChart® are registered trademarks of ADInstruments Pty Ltd.

Lab Protocol

Supplies

- LabChart software & computer
- PowerLab 26T
- 5 Lead Shielded Bio Amp Cable
- EEG Flat Electrodes (gold contacts)
- Electrode paste
- Abrasive Gel (NuPrep)
- Alcohol swabs
- Medical tape
- Elastic bandage
- Cotton swab

Part I. Spectral Analysis Tutorial

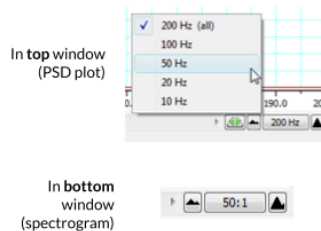
A spectrum is a representation of data based on the frequency distribution of its component sine waves. Spectra indicate the strength of the various frequencies in a time-varying waveform. Spectrum View allows you to observe the frequency distribution of data that might not otherwise be easily seen. For example, it could be used to break down an EEG waveform into its various components: beta waves, alpha waves, theta waves and delta waves. A mathematical technique known as the Fast Fourier Transform is applied to the raw data. The results of this analysis can be displayed as a plot of the power (vertical axis) of different frequencies (horizontal axis) relative to each other in the input signal. This is called a **Power Spectrum Density (PSD) plot**. The data can also be displayed as a 3-dimensional color plot of spectral power, frequency, and time, called a **Spectrogram**.

Note: The questions here are written to ensure that you are understanding the steps of the tutorial. You do not need to answer them, and they do not directly relate to your lab report. You *should*, however, understand how to interpret a spectrogram.

1. Open the EEG Spectral Analysis Tutorial settings file (<http://bit.ly/labchart>).
2. Examine the Chart View. Use the **View Buttons** to view each block. You should see five blocks of data. The first record is a slowly oscillating sine wave.
3. Open Spectrum view by clicking on the Spectrum View button in the Toolbar:



4. Click the **Smart Tile** button in the LabChart Toolbar to display both windows in full screen mode.
5. In Chart View, **select** the first record by double-clicking in the Time axis. This will perform a spectral analysis for this record and display the result in the Spectrum view.
6. Adjust the horizontal scaling of plots to view the results:
 - Set the **horizontal scaling** for the **Power Spectrum Density (PSD)** plot to 50 Hz.
 - Use the horizontal scroll bar to display the 0 Hz to 50 Hz region of the plot.
 - You may need to Autoscale the axes.
 - Set the **horizontal scaling** for the Spectrogram to 50:1.



7. Examine the PSD plot and then the first section of the Spectrogram. Expand the vertical axes if necessary. Use the waveform cursor to identify the frequency in Hertz (Hz) of the peak in the PSD plot and the band in the Spectrogram. Values are displayed at the top of each plot.
8. Select the second record and again view the result in the Spectrum view.
9. Select the third record and again view the result in the Spectrum view. You should now see two prominent peaks (PSD plot) and bands (Spectrogram) in the result.
10. Select the fourth record and again view the result in the Spectrum view.

Questions for reflection

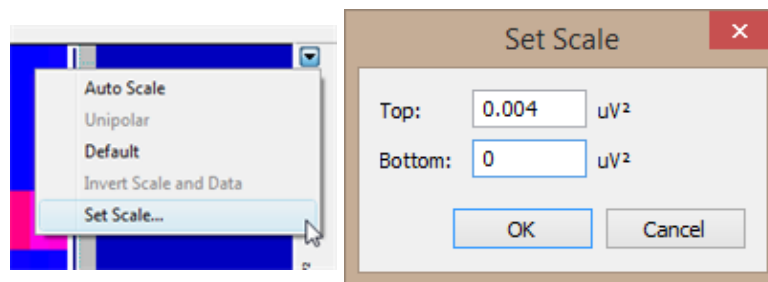
- What is the frequency in Hertz (Hz) of the first sine wave?
- What is the frequency in Hertz (Hz) of the second sine wave?
- Are the two peaks/bands you see in record 3 the same as for the first two records? How is this possible?
- Is there any regular signal within record 4?

11. In Chart View compare the signal amplitudes of the fourth and fifth records. Note that the fifth record has lower amplitude compared with the fourth record (you may need to Autoscale to see a peak at all).
12. Select the fourth record again. In the Spectrum view examine the PSD plot. Move the **Waveform Cursor** to the prominent peak.
13. Select the fifth record and examine the PSD plot. Move the **Waveform Cursor** to the peak.

Questions for reflection

- What is the frequency (Hz) of the signal in record 4?
- What is the power (mV^2) of the signal in record 4?
- What is the frequency (Hz) of the signal in record 5?
- What is the power (mV^2) of the signal in record 5?

14. Examine the Spectrogram. Note that the band corresponding to the signal's frequency appears to be missing. This is because the power of the signal is small compared with the previous four records. Expand the scale on the right-hand side of the Spectrogram by setting the scale closer to the power of the signal (e.g., right-click **Set Scale** to 0 to 0.004 V^2). Note that the band is now visible at the expected frequency.



15. The fifth record is the same signal as the fourth record, except that the quality of the raw signal has been affected. Compare your features (amplitude, power, frequency) of the fourth and fifth records.

Questions for reflection

- How has the quality of the signal affected the wave features?

Part II. Detecting alpha waves

Equipment Setup and Electrode Attachment

1. Make sure the PowerLab is turned **off** and the USB cable is connected to the computer.
2. Connect the 5 Lead Shielded Bio Amp Cable to the Bio Amp Connector on the front panel of the PowerLab (Figure 1). The hardware needs to be connected **before** you open the settings file.

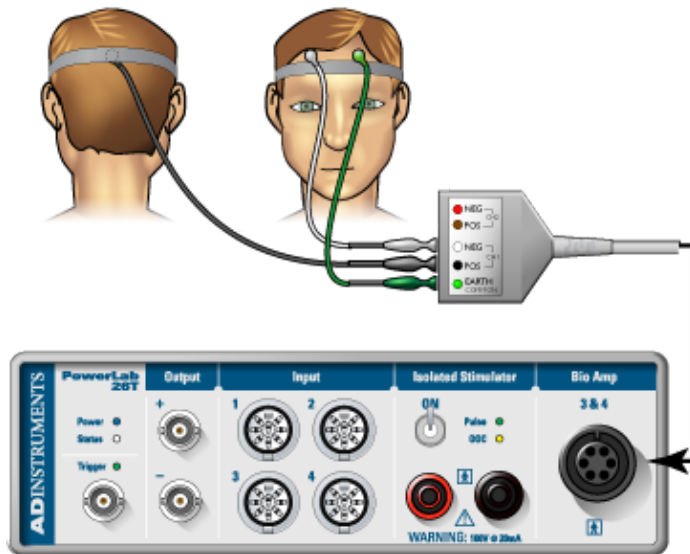


Figure 1. Equipment setup for PowerLab 26T. Image courtesy of ADInstruments ©

3. Attach the leads of the EEG Flat Electrodes to the Earth, CH1 NEG and POS pins closest to the labeled side on the Bio Amp Cable. Channel 1 "**positive**" will lead to theinion (the bump on the back of the head above the neck) and Channel 1 "**negative**" will lead to the forehead. Channel 2 will be empty and the Earth will lead to the temple. Refer to Figure 1 for proper placement, but do not attach them to the volunteer yet. Follow the color scheme on the Bio Amp Cable.
4. Remove any jewelry from the volunteer's face, ears, and neck.
5. With Figure 1 as a guide, locate where you will place the electrodes. Abrade the skin with the NuPrep Abrasive Gel.
Note: This is important as abrasion helps reduce the skin's resistance. Your recordings will be much better if you have abraded the skin well (but not too much — that will cause irritation).
6. After abrasion, clean the area with an alcohol swab to remove the dead skin cells.
7. While the skin is drying, scoop Electrode Paste into the EEG Flat Electrodes. When the skin is dry, stick the electrodes to the skin (Figure 1). Immediately hold the electrodes and wires in place with the medical tape. Make sure the tape firmly holds the electrodes against the head. Use the elastic bandage to wrap tightly around the head. This will help the electrodes maintain good contact with the skin.
8. Have the volunteer lie in a comfortable position on his/her back (or sit comfortably), with the head turned so that none of the electrodes are disturbed or compressed.
9. Check that all three electrodes are properly connected to the volunteer and the Bio Amp Cable before proceeding. Turn on the PowerLab.

Exercise 1: Recognizing Artifacts

In this exercise, you will learn to recognize some of the artifacts that can appear on an EEG.

1. Launch LabChart with the EEG Settings file (<http://bit.ly/labchart>).
2. Select **Bio Amp** from the EEG Channel Function pop-up menu. Make sure the settings are as follows: **Range 200 μ V, High Pass 0.5 Hz, and Low Pass 50 Hz.**



Questions for reflection

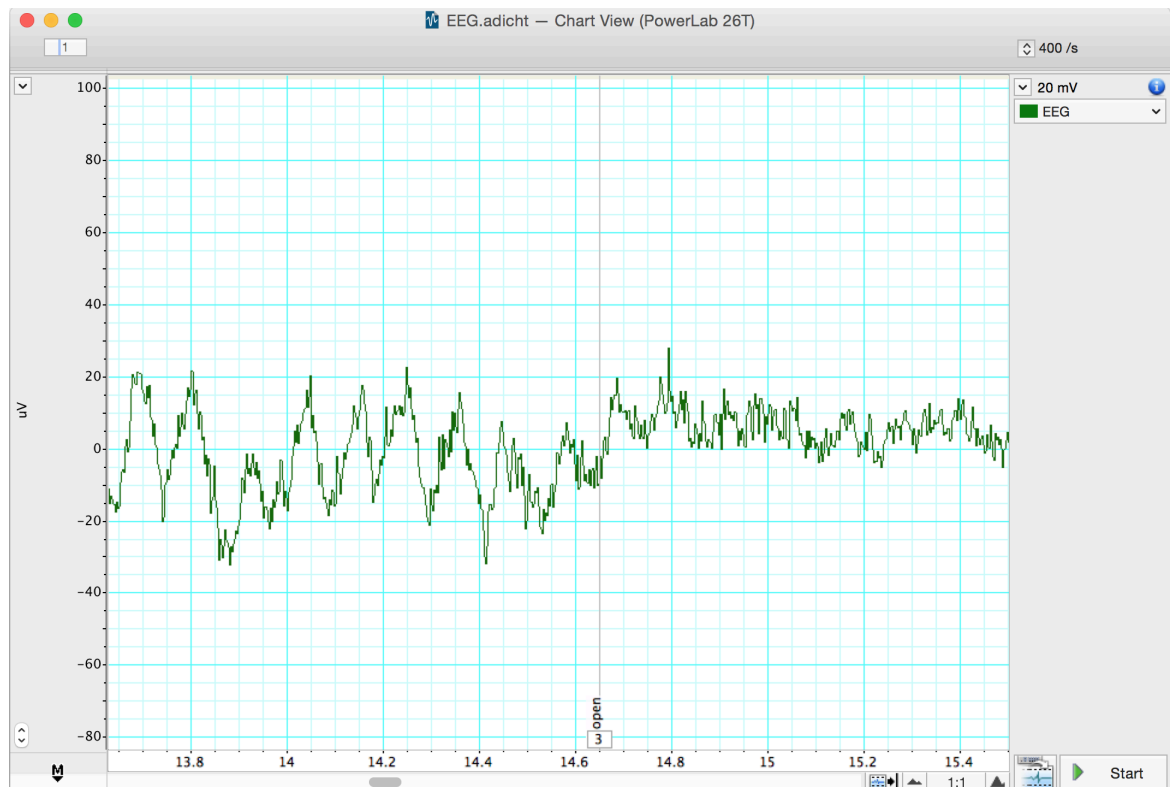
- Why are we applying high- and low-pass filters to our data?

3. **Start** recording. Add a **comment** "blinking," and have the volunteer blink repeatedly. **Stop** recording after 10 seconds.
4. Repeat step 3, this time, have the volunteer make eye movements. Add a **comment** "eye movements." Have the volunteer gaze up-and-down and left-and-right in a repeated pattern. Make sure the volunteer's head is still and only the eyes move.
5. Repeat step 3, this time, have the volunteer make head movements. Add a **comment** "head movements." Have the volunteer gently move his/her head in a repeated pattern.
6. Examine the vertical scale at the left of the Chart View, and note the positions corresponding to $+50\ \mu\text{V}$ and $-50\ \mu\text{V}$. At rest (when your volunteer is not blinking or moving their head), the EEG recording should not exceed these limits. It is okay if it goes beyond $-50\ \mu\text{V}$ or above $50\ \mu\text{V}$ when the person is blinking or moving. If your signal exceeds these limits, please ensure that the electrode on the back of the head is making electrical contact with the scalp and reposition if necessary.
7. **Save your data**, and open a new file with the same settings.

★ Exercise 2: Alpha Waves in the EEG

In this exercise, you will examine the effects of relaxation and eye movement on alpha waves in the EEG.

1. Make sure the volunteer is relaxed and comfortable. Have the volunteer close his/her eyes and remain quiet. Keep noise to a minimum and keep all distractions away from the volunteer.
2. **Start** recording. Prepare a comment with "open"; do not enter it yet.
3. Tell the volunteer to open both eyes. Immediately press Return/Enter to add the **comment**. Record with the volunteer's eyes open for 30 seconds. Do not stop recording.
4. Prepare a comment with "shut." When the 30 seconds are complete, tell the volunteer to close both eyes. Immediately press Return/Enter to add the **comment**.
5. Repeat steps 3 and 4 nine times to give you ten total repeats of each condition. **Save your data**.



EEG signal with eyes shut and open. Note alpha waves during eyes shut (left).

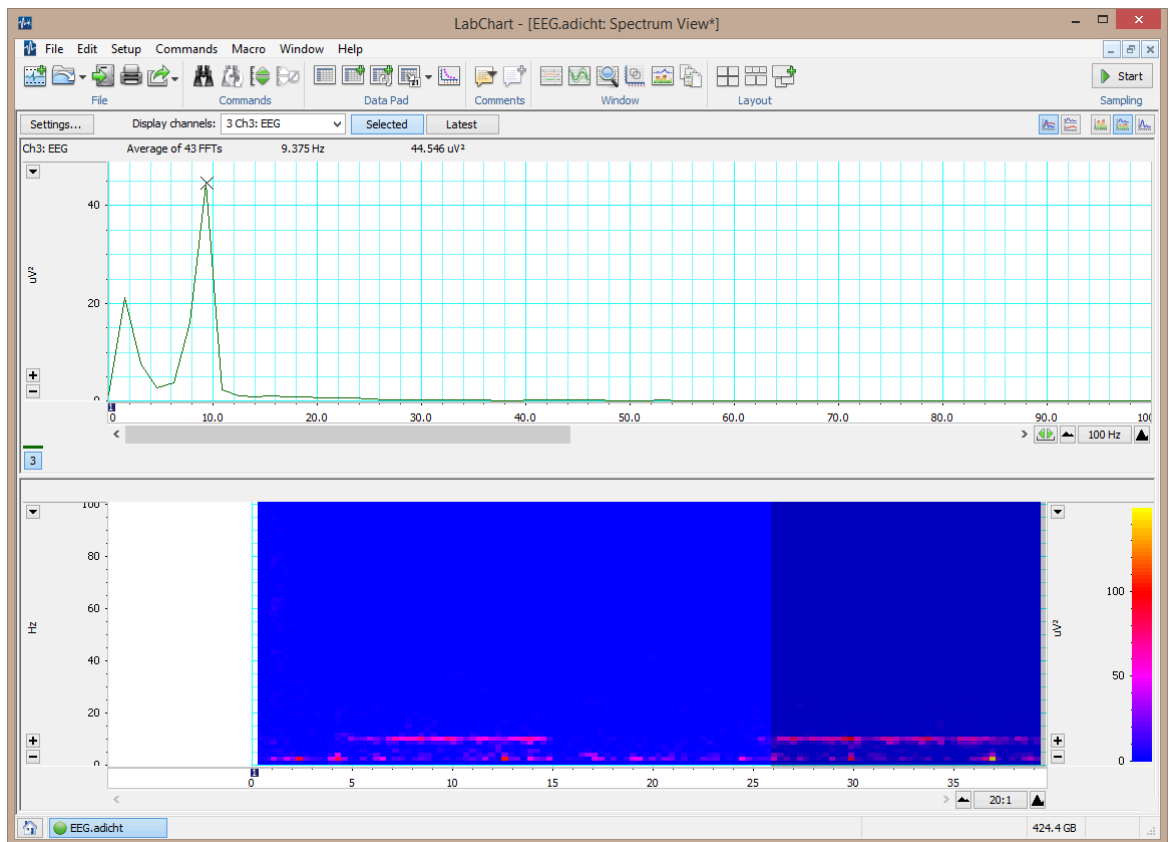
Wave Amplitude Analysis

1. Use the **View Buttons** to change the horizontal compression to see data with eyes open and shut. Make a data selection that includes some data from both eyes open and eyes shut conditions. View this selection in **Zoom View**. This should make it easier to see the alpha wave activity. **Autoscale**, if necessary.
2. In Chart View, scroll through the parts of the recording that were made with the volunteer's eyes shut to look for alpha waves. Use the **View Buttons** to change the horizontal compression if necessary. The alpha waves can be recognized by their amplitude (usually 30 to 50 μV peak-to-peak) and their frequency. Each cycle of an alpha wave lasts ~ 0.1 s.
3. Use the **Marker** and **Waveform Cursor** to measure the amplitude of the alpha waves. Place the **Marker** at the lowest point of the wave and move the **Waveform Cursor** to the peak of the wave. Measure the amplitudes of five waves from when the volunteer's eyes were closed. Record the values in **Table 1** of the Data Notebook.
4. Now measure wave amplitudes when the volunteer's eyes were open. Record these values in **Table 2** of the Data Notebook.

Spectral Analysis

Now you will use Spectral Analysis to examine the EEG you recorded.

1. Find the part of the recording when the volunteer had their eyes **shut**. Click-and-drag across this part of the data trace to **select** it.
2. From the **Window** menu, select **Spectrum**. In the Spectrum View, choose **Selected** (underneath the main toolbar at the top).



Spectrum of an EEG

3. Next to Display Channels, choose Ch 3: EEG.
4. Alpha activity should show up in the PSD plot as a clear peak in the 8–12 Hz range.
5. Alpha activity shows up in the Spectrogram as a band of color in the 8–12 Hz range. If you cannot see the alpha activity as a clear peak in the 8–12 Hz range, manually scale the horizontal and vertical axes. Note that Spectrogram displays all the recorded data and that the selection you have made is highlighted in a darker blue color.
6. Make a data selection of several seconds from when the volunteer had their eyes **open**. Select **Spectrum**. Note that in the PSD plot the peak in the alpha activity range of 8–12 Hz is small or absent, and in the Spectrogram the band of color in the alpha activity range of 8–12 Hz is weak or absent.
7. In the Spectrogram, scale the horizontal axis so that all the data is visible. Note the presence and absence of the band of color in the alpha activity range of 8–12 Hz, which corresponds with the eyes shut and eyes open conditions.
8. If you're happy with your spectrograms (e.g., you can clearly see alpha activity in closed conditions but not open conditions), you've completed the task for this lab. Save your data in a cloud location.



Questions for reflection

- Under what conditions did you see alpha waves more clearly?
- What are alpha waves thought to indicate?
- Look at your Spectral Analysis from when the volunteer had their eyes closed. At what range do you have the greatest peak? What type of brain waves does the Spectral Analysis suggest is active?

Analyzing your EEG data

1. Export 30 seconds of one closed trial and one open trial for your analysis. [See this page if you need a reminder.](#)
2. Using this notebook, you can generate plots from your EEG data: <https://bipn145.github.io/Python/SpectralAnalysis.html>

Data Notebook

Table 1. Alpha Waves with Eyes Open and Closed

	Eyes Open Amplitude (μV)	Eyes Closed Amplitude (μV)
Wave #1		
Wave #2		
Wave #3		
Wave #4		
Wave #5		
Wave #5		
Wave #6		
Wave #7		
Wave #8		
Wave #9		
Wave #10		

Working with LabChart Files

Saving recordings from lab computers

After finishing your experiment, save the raw data (.adicht) file. I'd recommend using either Google Drive or Dropbox to save your data so that you may access it from anywhere. Your files will rarely be bigger than 5 MB.

Viewing LabChart files from your Mac or PC

If you'd like to view your files on your personal computer without LabChart 8, you can download [LabChart 8 Reader](#).

If you're using LabChart Reader on PC, you can simply click & drag to select the data you'd like to export, copy, and then paste this into Excel. Unfortunately, LabChart Reader on Mac does not allow you to export your data.

To export your data from LabChart on a PC (or the computers in lab), proceed to the next page.

Exporting LabChart Files

This page will help you **export** the raw values from your LabChart recordings as a text file that we can read into either Excel or Python.

Note

This will work for LabChart8 but not LabChart Reader. In other words, you should do this step **before** you leave lab!

Video Tutorial

Written Tutorial

1. Select the data you would like to save by clicking and dragging in the window with your recording.
2. In LabChart, go to **File > Export**.
3. By default, LabChart will offer to export it as a Text File. In the first window that pops up, choose a name and location for your file and press Save.
4. In the following window, you have options for how your exported file will look. In the drop down Channels window, be sure that your data channel is checked. If applicable, you may also want to save your stimulus channel, so that you can also plot when the stimulus came on (and at what amplitude). If you're planning on using the Python script, do not export more than one channel.
5. Choose the bubble for "Current selection."
6. You *may* want to Downsample your data (e.g., take fewer data points than were collected), especially if you have issues plotting it later because the dataset is too large. However, if you don't have plotting issues, don't bother with Downsampling.

Once you are done exporting your data, you can import it using Excel or Python.

Importing LabChart Files (Excel)

The steps below will help you import and plot your LabChart .txt files using Microsoft Excel.

Note

If all of the steps below look a lot of work to simply replot a recording trace — you're right, it is. And so, we have a Python script for that! Check the other notebook, Importing in Python.

On Windows

1. Open a New Sheet in Excel.
2. Go to **File > Open > Browse** and navigate to where your text file is saved.
3. Change the file type to "All files" and open your .txt file.
4. In the next window(s), hit Next. In the following (screenshot below) make sure that the Delimiter is set to Tab, and click Finish. The data preview should have multiple separated

columns and look something like this:

Text Import Wizard - Step 2 of 3

This screen lets you set the delimiters your data contains. You can see how your text is affected in the preview below.

Delimiters

☒ Tab
☐ Semicolon
☐ Comma
☐ Space
☐ Other:

☐ Treat consecutive delimiters as one

Text qualifier:

Data preview

Interval=	2.5e-005 s	
ExcelDateTime=	4.3738531324010488e+004	9/30/2019 12:45:06.394507
TimeFormat=	StartOfBlock	
DateFormat=		
ChannelTitle=	BioAmp Recording	

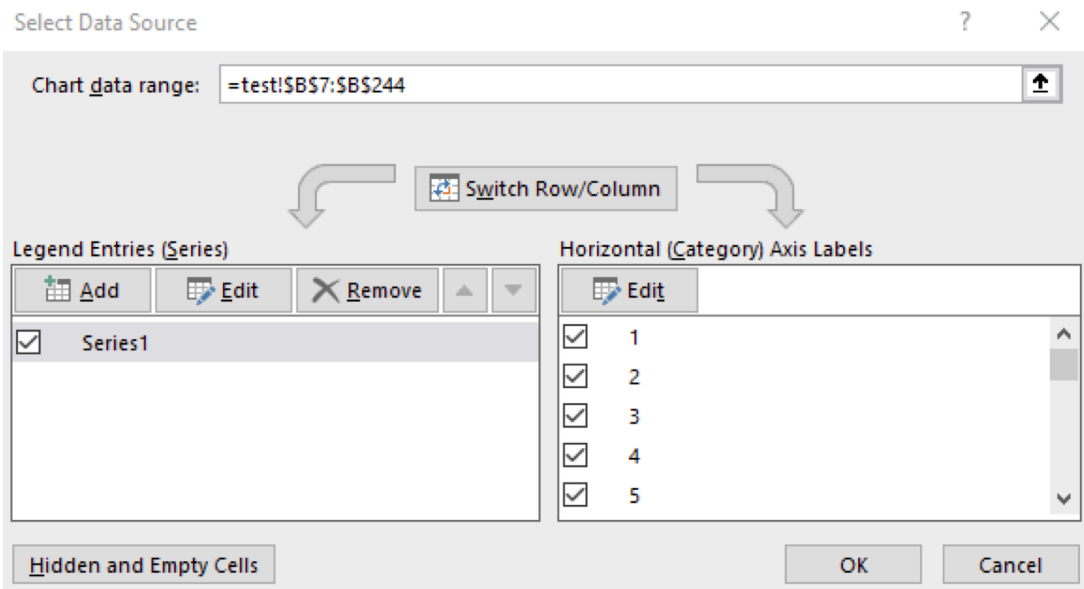
Cancel < Back Next > Finish

5. This file will have multiple columns. The first column will contain the timestamps for your experiment, in seconds. The second column will be the first channel of data that you've exported. In the example above, it is the stimulus channel. The third column will be the second channel of data, etc.

Note

If you have exported all of your data, each page of your experiment will be vertically concatenated. In other words, it will start at 0 and go to the end of Page 1, and then start at 0 again for the next page.

6. Select the cells containing the data you would like to plot (if you exported the stimulus channel, it's the third column; if you didn't export the stimulus channel, this should be the second column of your file).
7. Go to **Insert > Line Chart**.
8. To add the correct time axis labels, right click on the chart and choose "Select Data."
9. In the next window, under "Horizontal (Category) Axis Labels" choose "Edit." In the window that pops up, select all of the time values (the first column of your file):



10. From here, you might need to adjust the Horizontal Axis crossings in order to prevent the X axis from directly crossing your data (since your data will likely hover around zero, where the axis goes by default). For example, you might want to have your X axis cross at -100 mV, so that it doesn't overlap with your data.
11. Format your plot with axes labels and a title, and make any additional changes as necessary.

On Mac

1. Open a New Sheet in Excel.
2. Go to **File > Open > Browse** and navigate to where your text file is saved.
3. Change the file type to "All files" and open your .txt file.
4. In the next window(s), hit **Next**. In the following (screenshot below) make sure that the Delimiter is set to Tab, and click **Finish**. The data preview should have multiple separated

columns and look something like this:

Text Import Wizard - Step 2 of 3

This screen lets you set the delimiters your data contains.

Delimiters

☒ Tab ☐ Treat consecutive delimiters as one

☐ Semicolon

☐ Comma

☐ Space

☐ Other:

Text qualifier: "

Preview of selected data:

0.00005	-2.40625
0.000075	-5.6875
0.0001	-10.625
0.000125	-15.25
0.00015	-16.34375
0.000175	-10.03125
0.0002	7.25
0.000225	37.51562
0.00025	80.28125

Cancel < Back Next > Finish

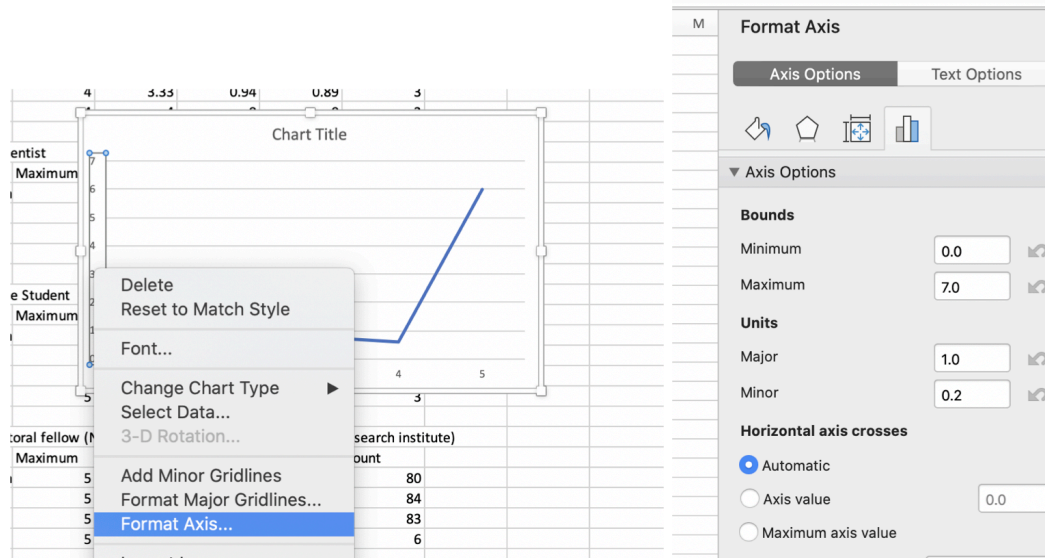
Note

This file will have multiple columns. The first column will contain the timestamps for your experiment, in seconds. The second column will be the first channel of data that you've exported. In the example above, it is the stimulus channel. The third column will be the second channel of data, etc.

If you (accidentally) exported all of your data, each page of your experiment will be vertically concatenated. In other words, it will start at 0 and go to the end of Page 1, and then start at 0 again for the next page.

5. Select the cells containing the data you would like to plot (if you exported the stimulus channel, it's the third column; if you didn't export the stimulus channel, this should be the second column of your file).
6. Go to Insert > Line Chart.
7. To add the correct time axis labels, click on **Select Data** under the Chart Design menu.
8. Where it says "Horizontal (Category) Axis labels, click the button to the right with the tiny red arrow.
9. Select all of your time stamps in the Excel file (the first column) and press Okay. Your x axis should now be updated with the time, instead of the samples.
10. From here, you might need to adjust the Horizontal Axis crossings in order to prevent the X axis from directly crossing your data (since your data will likely hover around zero, where the axis goes by default). For example, you might want to have your X axis cross at -100 mV, so that it doesn't overlap with your data. You can do this by clicking on an Axis, and modifying the "Horizontal axis crosses" in the "Format Axis" menu on the right (under the tab with the). Alternatively, you can right click on the axis, and choose "Format Axis", which will open the

same menu.



11. Format your plot with axes labels and a title, and make any additional changes as necessary.

Importing LabChart (Python)

This notebook will help you import and plot *any* exported .txt file from LabChart. Once you know how to use this notebook, you can use this to plot *any* exported text file from LabChart, from earthworms, to leeches, and beyond! It will also allow you to plot traces on top of each other.

To run this, you will need to upload an exported .txt file from LabChart into Colab. First, [export the channels you'd like to plot](#). Then [upload the exported file it to Colab](#).

Step 1. Setup

Task: Run the cell below to import our necessary packages and configure the notebook environment. You do not need to change anything in this cell.

```

# Import our packages
import matplotlib.pyplot as plt
import numpy as np

# Run this cell to set up the notebook – no need to change anything here!
def load_recording(filename):
    """Load a LabChart .txt export and return (time, recording) arrays."""
    try:
        data = np.genfromtxt(filename, dtype=float, skip_header=6, delimiter='\t', e
    except FileNotFoundError:
        print(f'File "{filename}" was not found. Follow directions above for uploadi
        return None, None
    except ValueError:
        print('It looks like your data is the wrong shape, did you export the CURREN
        return None, None

    if data.shape[1] == 2:
        print(f'Loaded "{filename}" successfully.')
        return data[:,0], data[:,1]
    elif data.shape[1] == 3:
        print(f'Loaded "{filename}" successfully. Using the third channel (column 3)
        return data[:,0], data[:,2]
    else:
        print('Invalid data file. Please upload a .txt file exported from LabChart w
        return None, None

def align_recordings(time, recording, time2, recording2):
    """Trim two recordings (assumed same sampling rate) to the same length.
    Returns (time_common, recording_trimmed, recording2_trimmed)."""
    n = min(len(time), len(time2))
    print(f'Trimmed both recordings to {n} samples ({time[n][-1]:.3f} s).')
    return time[:n], recording[:n], recording2[:n]

print('Packages imported and helper functions defined.')

```

Packages imported and helper functions defined.

Step 2. Import data

Now that we have our notebook configured, we can import our data. We will import it as a numpy array.

Task: [Upload your file](#) into the same folder as this code and change the filename below to your filename.

```
# Change the filename to EXACTLY match your file
filename = 'recording.txt'

# (Optional) To overlay a second recording, change filename2 to your second filename
# Leave as None if you only have one file.
filename2 = None

og_time, recording = load_recording(filename)

if filename2 is not None:
    time2, recording2 = load_recording(filename2)
    time, recording_trimmed, recording2_trimmed = align_recordings(og_time, recording2, recording2_trimmed)
else:
    time, recording_trimmed, recording2_trimmed = og_time, recording, None
```

Loaded "recording.txt" successfully. Using the third channel (column 3) for analysis.

Modify axes units (*optional*)

LabChart data *should* export by volts and second by default. The cell below will ensure that you have a time vector, and if not, will generate one for you.

Note: Remember that if your data was collected at 40,000 Hz (40 kHz), this means that there is 1/40,000 (or 0.000025) seconds between each data point.

```
try:
    print('\nTime between each data point is:')
    print(time[1]-time[0], ' seconds')
except NameError:
    print('You do not currently have a known time axis.')
    length = input('What is the length of your recording in seconds?')
    time = np.linspace(0, float(length)+1, len(recording))
    print(time)
```

Time between each data point is:
2.499999999997247e-05 seconds

Instead of showing the axes in seconds, you might choose to show it in milliseconds. To do so, we should multiply the entire array by 1000, to convert from **s** to **ms**. You can multiply arrays in Python by taking the original array, and writing an expression to multiple it. For example:

```
array_s = np.array([0.000 , 0.0010 , 0.0020 , 0.0030 ])
array_ms = array_s * 1000
```

Task: Create an array of timestamps in milliseconds by multiplying `time` by 1000. Assign it to `timestamps_ms`. Check that this worked by printing `timestamps_ms`.

```
# Convert into ms here
timestamps_ms = ...
```

Now you should have a `timestamps_ms` variable that you can plot with below, if you'd like!

Step 3. Plot data

The code below will plot your data using `plt.plot()`. This function requires two inputs: x and y. When we're plotting recordings, typically this means x = time, and y = voltage. If you'd like to plot your timestamps in milliseconds, you'll need to replace `time` with `timestamps_ms`. You should also add x and y labels.

```
# Set up figure & plot
fig,ax = plt.subplots(figsize=(10,4))

# Plot the first recording
plt.plot(time, recording_trimmed, label=filename)

# Plot the second recording if one was loaded
if recording2_trimmed is not None:
    plt.plot(time, recording2_trimmed, label=filename2)
    plt.legend()

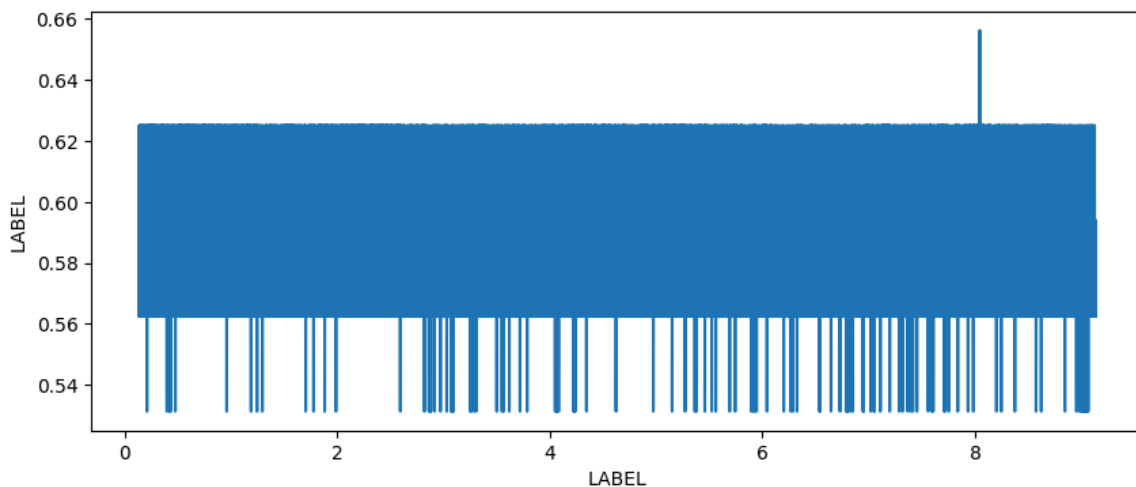
# You may need to change the x label!
plt.xlabel('LABEL')

# You may need to change the y label!
plt.ylabel('LABEL')

# This makes the axis labels print without scientific notation
# You can comment this line if you do not wish to use it
ax.ticklabel_format(useOffset=False, style='plain')

# You can uncomment the line below to restrict the x axis plotting
#plt.xlim([200,700])

plt.show()
```



You can right click on the plot above, or screenshot it, to save it.

Visualizing Data in Python

The notebooks under this header will help you visualize data in Python.

Uploading data to Colab

To work with exported LabChart data in Python, you'll either need to upload the file to Colab or put it in Google Drive & mount your drive.

Uploading directly

To upload a file, click the Directory icon on the left hand side of the window. Then, click the file upload button. There is a video of how to do this below:

Mounting your drive

1. To mount your drive, click the Directory icon on the left hand side of the window.
2. Click the Google Drive icon.
3. Colab will create a new cell in the notebook and prompt you to run it.
4. If it's the first time, a separate window will pop up to ask you for access to your Drive.
5. Click "Allow".
6. After a few seconds, a message will print that your Drive has been mounted.
7. Now, you can use the location of your file in a filepath that starts with `/content/drive`.

Saving work in Colab

You can save a Colab file that you've edited just like any other file! In Colab, go to `File > Save a copy in Drive`. These files will show up in a folder called "Colab Notebooks" in your Drive.

Introduction to Jupyter Notebooks

This first notebook will introduce you using Google Colab/Jupyter Notebooks, a handy coding environment for learning as well as sharing code with others.

At the end of this notebook, you'll be able to:

- Recognize the main features of Jupyter Notebooks
- Use notebooks to run Python Code
- Identify and edit simple Markdown code
- Create a list and a NumPy array
- Use matplotlib to create line and scatter plots

Saving your work

To save a copy of this notebook for yourself in Google Colab, go to **File > Save a copy** in Drive. These files will show up in a folder called "Colab Notebooks" in your Drive. Otherwise, your changes here will not be saved.

Part I. About Jupyter Notebooks

Jupyter notebooks are a way to combine executable code, code outputs, and text into one connected file. They run in a web browser, but don't require the internet (unless you're running it on Colab or a Jupyter Hub).

The **'kernel'** is the thing that executes your code. It is what connects the notebook (as you see it) with the part of your computer, or the DataHub computers, that runs code.

Types of Cells

Jupyter Notebooks have two types of cells, a **Markdown** (like this one) and **Code**. Most of the time you won't need to run the Markdown cells, just read through them. However, when we get to a code cell, you need to tell Jupyter to run the lines of code that it contains.

Code cells will be read by the Python interpreter. In other words, the Python kernel will run whatever it recognizes as code within the cell.

Task: Run the cell below! You can run a cell (this box) by pressing shift-enter or by pressing the run arrow.

```
# In Python, anything with a "#" in front of it is code annotation,  
# and is not read by the computer.  
#  
# Click in this cell and then press shift and enter simultaneously.  
# The print function below generates a message.  
print('Nice work!')
```

Nice work!

Part II. Using Markdown

Markdown is useful because it can be formatted using simple symbols. [Here's a full cheatsheet of markdown](#) for more tips, but the main syntax is below:

- You can create bulleted lists using asterisks.
- Similarly, you can create numbered lists using numbers.
- You can **bold** with two asterisks or underscores on either side (`**bold**`) or *italicize* with one asterisk or underscore (`*italicize*`)
- Pound signs (#) create headers. More pound signs means a smaller header.

Task: Edit the markdown cell below with a quick biography of yourself. You should have your name as a big header, a short quippy subtitle for yourself as a smaller header, and a three bullet points that use both **bold** and *italic*.

Edit this markdown cell!

Part III: Basic Python Syntax

Like any language, Python follows a set of rules, known as the language **syntax**. Below, we'll see how to write expressions in Python, create variables, and then manipulate those variables.

Python Expressions

We can perform various arithmetic operations in Python:

Symbol	Operation	Usage
+	Addition	10+2
-	Subtraction	10-2
*	Multiplication	10*2
/	Division	10/2
**	Exponent	10**2
%	Modulo	10%2

Notes:

- The default order of operations is the same as in mathematics! (PEMDAS)
- If you want a whole number from your division, use // instead

Task: Try each of the operators above. Before using the modulo operator, predict what it will output, and make sure it produces what you expect. You can use `print()` statements to see the outcome of multiple lines of code.

```
# Let's play with numbers weee
```

Variables

Variables enable us to store a value and come back to it later. They are defined with `name = value`. *Assignment is not the same thing as equality*, as in mathematics. Instead, think of this as storing a value in a jar (the variable).

Assigning variables

Task: Create two variables: `a` & `b`. Then, use an expression that combines `a` and `b`, and assign this to `c`. In the end, `c` should be equal to `6`.

```
# Complete the task above here  
a = ...
```

Note: Most code cells will not give you an output unless you ask for it. You can use `print( )` to output a variable or string. However, cells that *only* contain one variable will print its value.

Usefully, you can run your cells out of order. This is useful for testing and debugging.

Task: Change your equation for `c` in the cell above and then use `print(c)` to check its value. For example, if you originally multiplied two times three, change the equation to two plus three.

Part IV: Generate some plots

Step 1. Import packages

We can take advantage of pre-packaged code for many common functions in Python. But first, we need to tell Python to import it. This is a really common step for most Python code.

Below, we'll import a package called "numpy" and nickname it "np" and "matplotlib.pyplot" and nickname it "plt."

Task: Add `as plt` after `import matplotlib.pyplot` to instruct Python to import the [Matplotlib pyplot package](#) but nickname it `plt`. When you see `plt` in our script, it's actually calling scripts from this package.

After you import it, we'll use `%whos` to show all of the variables in the namespace, including our packages. Having printed messages like these is a really nice way to check that your cell actually ran, and that your packages are available.

```
# Add your code here  
import matplotlib.pyplot  
  
%whos
```

Variable	Type	Data/Info
a	ellipsis	Ellipsis
matplotlib	module	<module 'matplotlib' from<...>/matplotlib/__init__.py'>

Step 2: Enter some data to plot

Lists are a type of **data structure** in Python. They are defined using brackets `[ ]` with individual items separated by commas. For example, if you wanted to create a list, you could write:

```
my_list = [1,3,2,5]
```

Task: Below, generate a random list of 10 numbers for Python to plot. Assign it to `random_list`.

```
# Replace the ... with your list of numbers. Don't forget brackets!
random_list = ...
```

Task: Let's make sure Python did what we wanted it to do – store our random list as a variable called `random_list`. Check by typing the variable name into the next cell, and running it. If that worked, you should see a list of values.

Step 3: Plot your data

Let's pretend this is data from an awesome experiment we ran. We need to plot the data. Remember that we imported `matplotlib.pyplot` as `plt`. We can now use the `plt.plot()` function to plot our random list. `plot` is matplotlib pyplot's basic plotting function.

Task: In the cell below, run `plt.plot()`, giving it just one argument: `random_list`. In the following line, use `plt.show()` (without any arguments) to cleanly show the plot.

Tasks:

1. Add axes to your plots by adding `plt.xlabel('yourxlabelhere')` and `plt.ylabel('yourylabel')`. Add those above *before* `plt.show()`.
2. Add some markers to your line! Do this by adding an additional argument to `plt.plot(random_list)`, so that it says `plt.plot(random_list, marker="o")`. [You can add various markers of your choosing!](#)

Step 4: Add some more data

Let's create another random list and create a scatterplot of the data. To do so, we'll take a couple of steps.

1. Import another package, called `numpy` (numerical python). The convention is to import this as `np`.
2. Convert our list into a numpy array. This will allow us to perform numerical operations on it.
3. **Task:** Create a second array named `random_array_2` where each value is **equal to all of the values in `random_array` times two**. You can do this with the same operators as above. For example, if we wanted to divide all of the values in `random_array`, we would use `random_array/2`.

```
# 1 - Import numpy
import numpy as np

# 2 - Convert random_list to an array
random_array = np.array(random_list)

# 3 - Create random_array_2 below
random_array_2 = ...
```

Task: In the box below, make a scatterplot of your data using the matplotlib's scatter function. You should plot `random_array` on the x axis, and `random_array_2` on the y axis. Be sure to label your axes as well. You can find documentation on the scatter function [here](#) or [here \(for more fun\)](#).

Step 5: Celebrate

That's the Jupyter Notebook tutorial! You're ready to tackle more complex notebooks.

```
from IPython.display import HTML
HTML('<img src="https://media.tenor.com/images/99cff34bdcb675975b2b0cc661f2e4ce/tenor'...
```



Resources

For additional Jupyter Notebook information and practice, see [this tutorial](#) from DataQuest.

About this Notebook

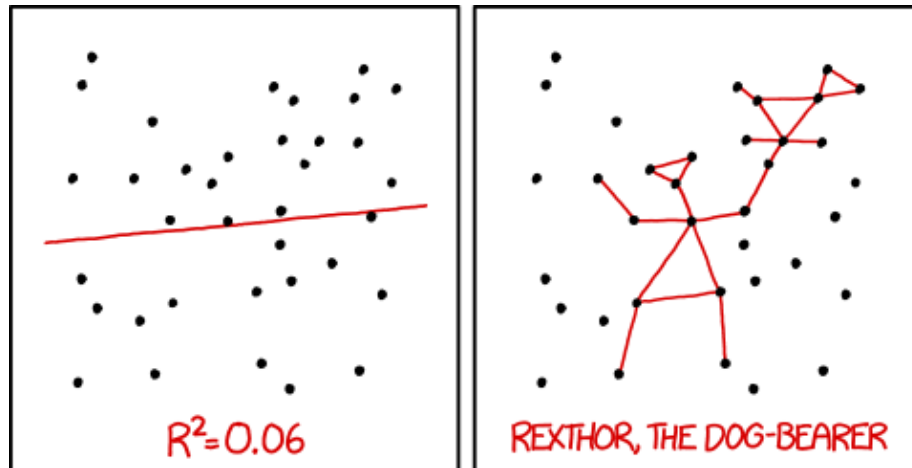
This notebook was created by [Ashley Juavinett](#) for classes at UC San Diego.

Plot scatter

This notebook will help you generate a scatterplot in Python. **Scatterplots** help us visually inspect the relationship between two continuous variables. This notebook should be used to plot your **resistance** vs. **time constant** observations from the RC circuit lab, for example.

After plotting, we'll also show how to generate a linear regression line for your data. **Linear regression** is a common tool to model a linear relationship between two variables.

If you're new to Jupyter Notebooks and/or Python, please be sure to work through the Introduction notebook before this one.



I DON'T TRUST LINEAR REGRESSIONS WHEN IT'S HARDER
TO GUESS THE DIRECTION OF THE CORRELATION FROM THE
SCATTER PLOT THAN TO FIND NEW CONSTELLATIONS ON IT.

Comic: XKCD <https://xkcd.com/1725/>

Setup

At the start of almost every coding notebook, we'll import the packages we need. To plot our scatter plot, we just need package: `matplotlib.pyplot`.

Task: Import the `matplotlib.pyplot` package as `plt`, just as you did in the introduction notebook.

```
# Import necessary packages here

# The line below makes the figures generate in higher resolution
%config InlineBackend.figure_format = 'retina'
```

Define values to plot

With matplotlib imported, we can now use the scatter function by calling `plt.scatter()`. However, we need to define what to plot first. One straightforward way to think about this is to define an x variable and a y variable. Below, there are **lists** of values (defined in brackets `[ ]`) assigned to `x` and `y`. Replace these with your own values, depending on what you'd like to plot on the x and y axis.

Note: Remember that in a scatterplot, each dot has both an x and a y value. Therefore, these lists should be the same length. The coordinate for each point will be the values at the same **index** in `x` and `y`. For example, the coordinate for the very first point will be `x[0],y[0]`.

```
# Replace the values below with your scatterplot values
x = [1,2,3,4,5,6]
y = [4,5,6,7,8,9]
```

Plot & label

We can now plot our values using the code below. Remember that you can add axis labels using `plt.xlabel()`. If you need a reminder for how to do this, refer to the Introduction notebook.

```
plt.scatter(x,y)

# Add labels here

plt.show()
```

Task: Create a cell below with the code above. Add code where it says `# Add labels here` to label your axes.

Add a linear regression line

If we have an a priori hypothesis about the relationship between our variables, or would like to predict additional data points, we can attempt to fit a linear regression line to our data. To do so, we will do the following:

1. Import two more packages: `numpy` (numerical python; the convention is to import this as `np`) and the `stats` package from `scipy` (scientific python).
2. Convert our x and y lists into arrays, so that we can perform math on them.
3. Perform a linear regression using `linregress()` from the `stats` library. The `linregress()` function calculates a **linear least-squares regression** for two sets of measurements. It returns several parameters, including the slope (`slope`), the y-intercept (`intercept`), the correlation coefficient (`r_value`), the two-tailed p-value (`p_value`), and the standard error of the estimate (`std_err`).
4. Plot the regression line, using the computed slope and intercept to construct it ($mx+b$).
5. Plot the original data points.

```
# 1 - Import additional packages
import numpy as np
from scipy import stats

# 2 - Convert x and y to numpy arrays
x_array = np.array(x)
y_array = np.array(y)

# 3 - Perform linear regression
slope, intercept, r_value, p_value, std_err = stats.linregress(x_array,y_array)

# 4 - Plot a regression line, using the slope & intercept
plt.plot(x_array, slope*x_array+intercept, color='gray',label='fitted line')

# 5 - Plot our original data points and show
plt.scatter(x_array,y_array,label='original data')
plt.legend()
plt.show()
```

```
-----
NameError                                Traceback (most recent call last)
Cell In[3], line 13
     10 slope, intercept, r_value, p_value, std_err = stats.linregress(x_array,y_arr
     12 # 4 - Plot a regression line, using the slope & intercept
-->  13 plt.plot(x_array, slope*x_array+intercept, color='gray',label='fitted line')
     15 # 5 - Plot our original data points and show
     16 plt.scatter(x_array,y_array,label='original data')

NameError: name 'plt' is not defined
```

It is also a good idea to print the computed statistics from our linear regression above. These are contained in `r_value` and `p_value`:

- `r_value`: This is the Pearson correlation coefficient. It measures the strength and direction of the linear relationship between the two variables. It ranges from -1 to 1, where a value of -1 indicates a strong negative linear relationship, a value of 0 indicates no linear relationship, and a value of 1 indicates a strong positive linear relationship.
- `p_value`: The p-value for a hypothesis test whose null hypothesis is that the slope is zero (in other words, that there is no relationship between x and y. A smaller p value (we typically use a 0.05 cutoff) suggests that it is unlikely the slope is zero.

It can also be helpful to look at `std_err`: the standard error of the estimate. It represents the standard deviation of the residuals (the differences between the observed y values and the predicted y values). A small standard error indicates that the fitted line is a good fit for the data.

Task: Print your values for `r_value`, `p_value`, and `std_err` in the cell below.

Finally, sometimes it is useful to have the fitted line equation. We can create a text string by adding variables together, like so:

```
text_string = 'this is ' + variable
```

However, Python will not allow us to add anything that isn't a string – we have to convert it to a string first using the `str()` function. To do so, we could write:

```
text_string = 'this is ' + str(variable)
```

Task: Below, create a text string for the fitted line equation, in the form of $Y = MX + B$, where M is your slope, and B is your intercept. In other words, it should look something like this:

```
Y = 1.0X + 3.0
```

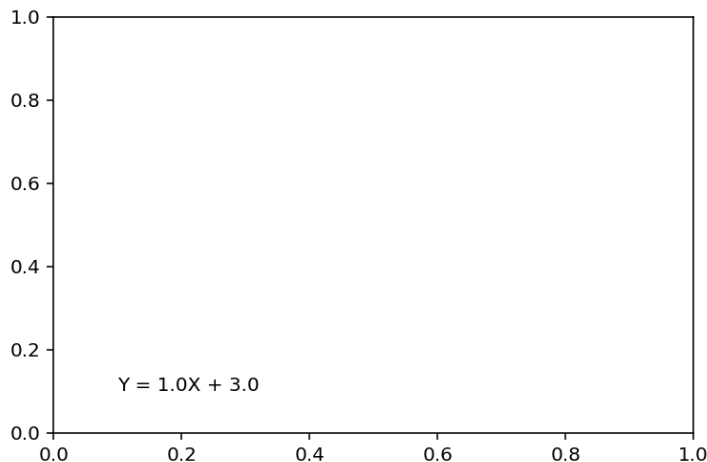
Assign the fitted line equation to a variable called `label`.

```
label = ...
```

Add text for your fitted line to the plot (*optional*)

It is useful to report the fitted line equation either on the plot itself or in the figure caption. The code below will show you how to add text using `plt.text()`, which takes three arguments: the x,y location of the text as well as the string itself (s). If you'd like, you can integrate this into your code above.

```
plt.figure()
plt.text(x = 0.1, y = 0.1, s = label)
plt.show()
```



Additional notes & resources

Another way to generate a scatterplot with a linear regression line is the [seaborn regplot function](#)!

About this Notebook

This notebook was created by [Ashley Juavinett](#) for classes at UC San Diego.

Fit a curved line

This notebook will help you plot the **strength-duration curve** for your earthworm.

Note: If you're new to Jupyter Notebooks and/or Python, please be sure to work through the Introduction notebook before this one.

Step 1. Define values to plot

With matplotlib imported, we can now use the scatter function by calling `plt.scatter()`. However, we need to define what to plot first. One straightforward way to think about this is to define an x variable and a y variable. Below, there are **lists** of values (defined in brackets `[ ]`) assigned to `x` and `y`. Replace these with your own values, depending on what you'd like to plot on the x and y axis of your strength duration curve.

Note: Remember that in a scatterplot, each dot has both an x and a y value. Therefore, these lists should be the same length. The coordinate for each point will be the values at the same **index** in `x` and `y`. For example, the coordinate for the very first point will be `x[0],y[0]`.

```
# Add your data points here
x = [1,2,3,4,5,6]
y = [1,0.8,0.5,0.45,0.4,0.4]

%whos
```

Variable	Type	Data/Info
x	list	n=6
y	list	n=6

Step 2. Fit a hyperbolic curve to hypothetical data

The strength-duration curve should follow a **hyperbolic** function:

$$a = r + \frac{r * c}{t}$$

where **a** is the stimulus amplitude (in amps or volts), **r** is the rheobase, **c** is the chronaxie, and **t** is time (the width of the stimulus).

We'll first define that function below:

```
# Define the hyperbolic curve function
def func(t, r, c):
    return r+((r*c)/t) #hyperbolic function
```

To demonstrate what this function looks like, we'll use `np.linspace()` to generate a list of 100 numbers from 0 to 1, with 0.01 spacing. We'll also define a sample rheobase and chronaxie. Then, from those points, we'll fit a hyperbolic curve using `scipy.optimize.curve_fit()`, which uses non-linear least squares to fit a function to data. See the [curve_fit documentation](#).

```

# Import numpy & curve fitting package
import numpy as np
from scipy.optimize import curve_fit

# Import & configure plotting packages
import matplotlib.pyplot as plt
%matplotlib inline
%config InlineBackend.figure_format = 'retina'

# Define pulse
pulse_width = np.linspace(0.01,1,100)
rheobase = .5 # in V
chronaxie = 0.1 # in ms

amplitude = rheobase+((rheobase*chronaxie)/pulse_width)

# Use our function & fit the curve!
sample_popt, sample_pcov = curve_fit(func,pulse_width,amplitude)

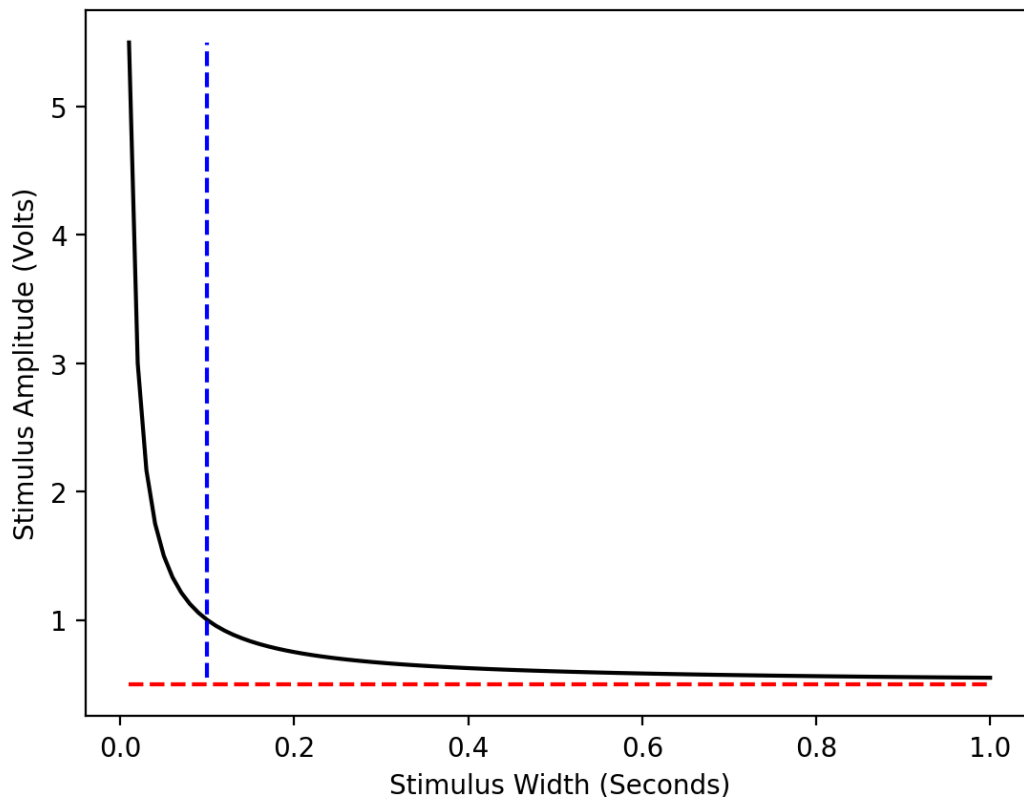
# Plot a regression line, using the slope & intercept
plt.plot(pulse_width, func(pulse_width, *sample_popt), 'black')

# Plot chronaxie & rheobase lines
plt.vlines(chronaxie,amplitude.min(),amplitude.max(),color='blue',linestyle='--')
plt.hlines(rheobase,pulse_width.min(),pulse_width.max(),color='red',linestyle='--')

# Label the axes & show plot
plt.ylabel('Stimulus Amplitude (Volts)')
plt.xlabel('Stimulus Width (Seconds)')

plt.show()

```



Step 3. Fit a hyperbolic curve to *your* data

Above, you can see a perfectly fit curve, with the **rheobase** and **chronaxie** labeled. Now, let's do the same, but with your data.

```
# Transform x & y data into arrays
x = np.array(x)
y = np.array(y)

# Fit the curve with your data!
popt, pcov = curve_fit(func,x,y)

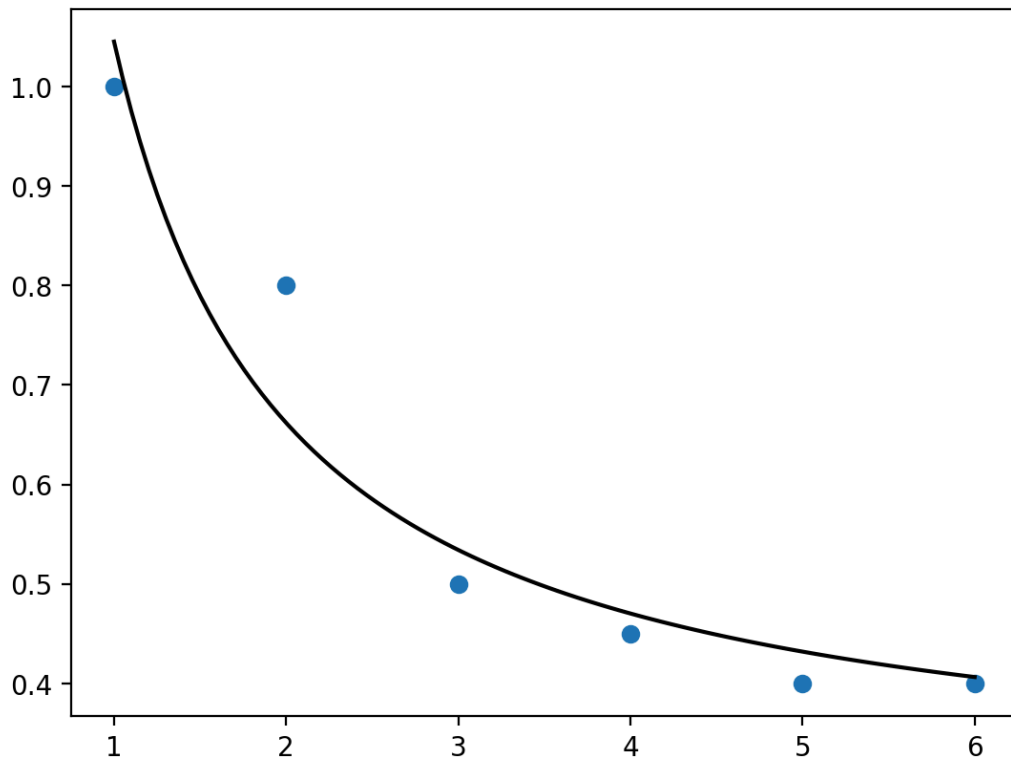
# Generate additional x-values to make a smooth curve
xnew = np.linspace(x.min(),x.max(), 100)

# Plot a regression line, using the slope & intercept
plt.plot(xnew, func(xnew, *popt), 'black')
plt.scatter(x,y)

# Optional -- can you figure out how to add rheobase & chronaxie?

# Add your labels here!

plt.show()
```



It is also important to know how well our curve fits. To do so, we'll calculate the residuals and then the sum of squares.

```

# Calculate the distance (on y axis) from data to curve
residuals = y - func(x, *popt)

# Calculate the residual sum of squares
ss_res = np.sum(residuals**2)

# Calculate the total sum of squares
ss_tot = np.sum((y-np.mean(y))**2)

# Calculate r-squared
r_squared = 1 - (ss_res / ss_tot)

print('R^2 = ', r_squared)
print('Rheobase: ', popt[0])
print('Chronaxie: ', popt[1])

```

```

R^2 = 0.9239011751642453
Rheobase: 0.27874587676845525
Chronaxie: 2.7492303745441005

```

Plot dot plot

This notebook will help you generate “Prism-style” dot plots in Python, inspect the distribution of your data, and run two-sample statistics.

Step 1. Import Data

For this notebook, we can either import our data from a CSV file, or by manually entering the values.

If you'd like to import your data from a CSV file, you will need to follow the instructions for uploading data to Colab on [the home page](#). If you are using this option, comment out the lines of code under Option 2.

Task:

1. Change `data_1` and `data_2` to be your two groups of data. *Make sure you leave these as lists, with brackets on each end, and each data point separated by a comma.*
2. Optional: Rename `Condition_1` and `Condition_2`. *Make sure you keep these in single quotes, so Python recognizes them as a string!*

```

# Option 1: Import a CSV file as a Pandas dataframe
import pandas as pd
#filename = ...
#data = pd.read_csv(filename)

# Option 2: Import your data as two lists and generate a dataframe from it
data_1 = [1,3,3,2,1,2,4,2,5]
data_2 = [3,4,5,3,2,6,7,8,3]
data = pd.DataFrame(data={'Condition_1':data_1,'Condition_2':data_2})

# Show the data
data

```

	Condition_1	Condition_2
0	1	3
1	3	4
2	3	5
3	2	3
4	1	2
5	2	6
6	4	7
7	2	8
8	5	3

Step 2. Plot Data

Below, we'll use a seaborn plotting function called [swarmplot](#) to plot each of our data points.

Notes

- This will draw a **dotted gray** line for the mean, and a **solid black line** for the median.
- Change the `plt.ylabel` line to add your own label.

```
# Import needed packages
import seaborn as sns
import matplotlib.pyplot as plt
%config InlineBackend.figure_format = 'retina'

# Set up the plot
fig, ax = plt.subplots(1, 1, figsize=(4, 4))

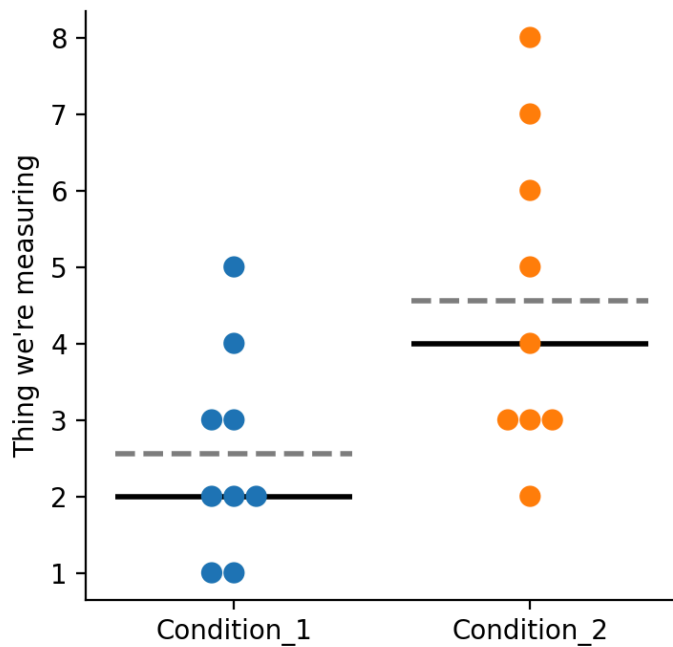
# plot the mean line
sns.boxplot(data=data, showmeans=True, meanline=True,
            meanprops={'color': 'gray', 'ls': '--', 'lw': 2},
            medianprops={'visible': True, 'color': 'black', 'ls': '-', 'lw': 2},
            whiskerprops={'visible': False},
            showfliers=False, showbox=False, showcaps=False)

# plot individual data points
sns.swarmplot(data=data, s=8)

plt.ylabel('Thing we\'re measuring')

# Make the axes look nice!
ax.spines[['right', 'top']].set_visible(False)

plt.show()
```



Step 3. Check to see how skewed the data is

Before we run any hypothesis tests, we need to know if our data is skewed or not. To test for skewness, we can use `stats.skewtest` to test. This method implements the [D'Agostino-Pearson skewness test](#), one of many different tests that can be used to check the normality of a distribution. **If the skew test gives us a p-value of less than 0.05, the population is skewed.**

Note: `skewtest` requires **at least 8 samples**. If your group has fewer than 8 data points, skip to the **Kolmogorov-Smirnov test** option below instead.

Task: Run the cell below, but then change the `sample` to `data_2` (or create a separate cell for `data_2`) to test your second group of data points.

```
from scipy import stats

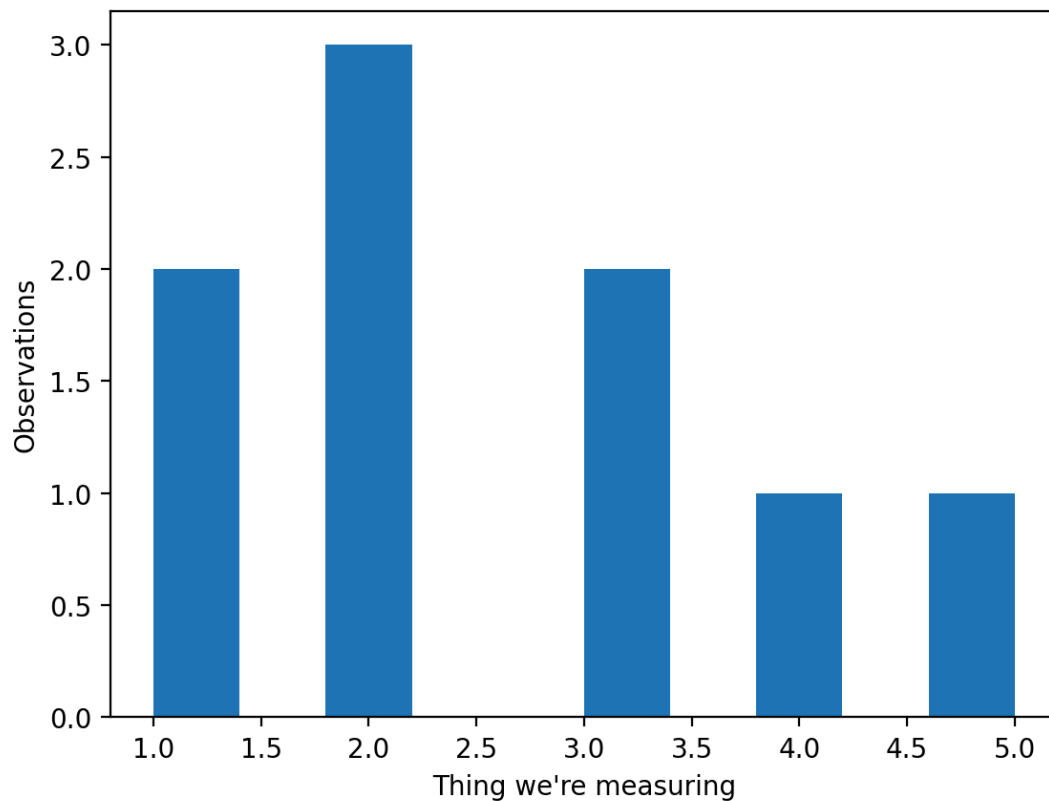
sample = data_1 # Choose which data to use

stat,pvalue = stats.skewtest(sample) # Run the skew test

# Print the p value of the skew test up to 30 decimal points
print('The skewtest p-value is ' + '%.30f' % pvalue)

plt.hist(sample) # Create a histogram
plt.ylabel('Observations')
plt.xlabel('Thing we're measuring')
plt.show()
```

The skewtest p-value is 0.346991576561619385898893597187



Option B: Kolmogorov-Smirnov test (for small samples)

The [Kolmogorov-Smirnov \(KS\) test](#) compares your sample against a theoretical normal distribution and works for any sample size. **If the KS test gives a p-value of less than 0.05, the data significantly deviates from normal.**

Task: Change `sample` to `data_1` or `data_2` and run the cell.

```
import numpy as np

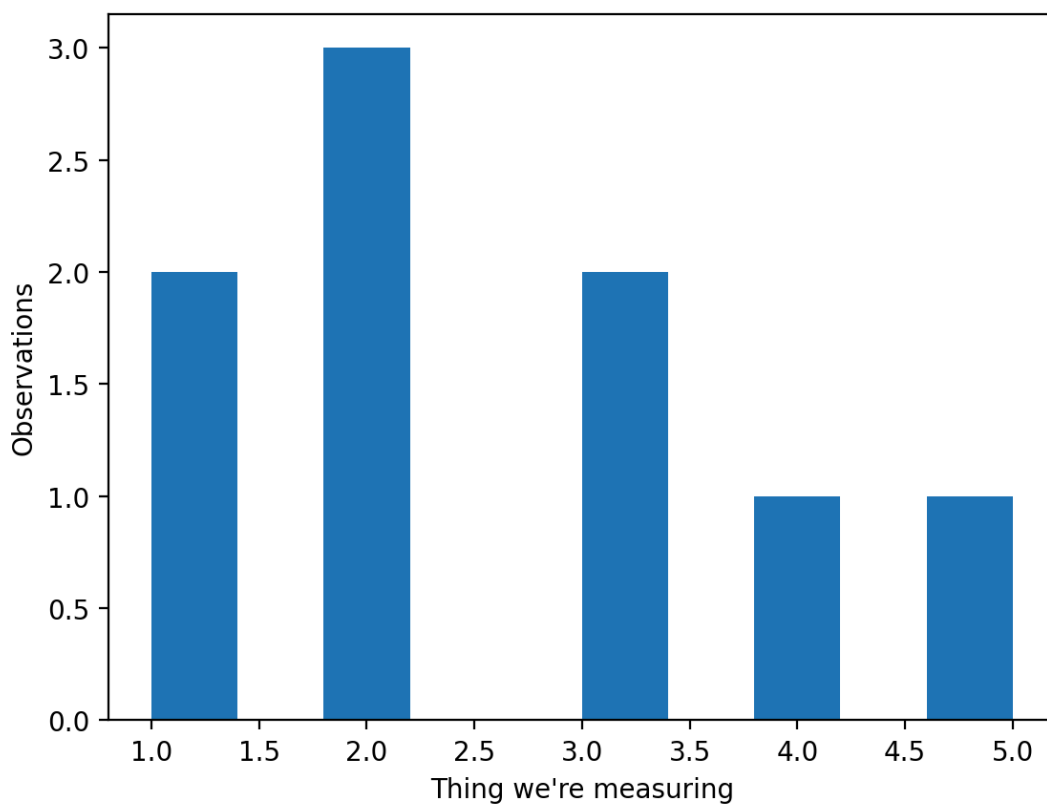
sample = data_1 # Choose which data to use

# Kolmogorov-Smirnov test for normality
# Tests whether the sample could have come from a normal distribution.
# Works for any sample size (including n < 8).
# If p < 0.05, the data significantly deviates from normal (i.e., it is skewed).
stat, pvalue = stats.kstest(sample, 'norm',
                             args=(np.mean(sample), np.std(sample)))

print('KS statistic: ' + str(round(stat, 4)))
print('KS p-value:    ' + '%.30f' % pvalue)

plt.hist(sample)
plt.ylabel('Observations')
plt.xlabel('Thing we\'re measuring')
plt.show()
```

KS statistic: 0.2263
KS p-value: 0.666861007096217206502331009688



Step 4. Run two-sample statistics

Inferential statistics generalize from observed data to the world at large

Most often, the goal of our hypothesis testing is to test whether or not two distributions are different, or if a distribution has a different mean than the underlying population distribution.

The SciPy stats package has [many hypothesis testing tools](#). For many simple cases in biology or neuroscience research, we'd like to test whether two or more distributions are different from each other.

If we know our distributions are normal (they're generated from a normal distribution!) we can use **parametric statistics** to test our hypothesis. To test for differences between normal populations, we can use the independent t-test in our stats package: `stats.ttest_ind()`.

If we had paired samples, we would use a dependent t-test [as seen here](#).

If one of our populations is skewed, however, we **cannot use a t-test**. A t-test assumes that the populations are normally distributed. For skewed populations, we can use either the [Mann-Whitney U](#) (for independent samples, `stats.mannwhitneyu()`) or the [Wilcoxon Signed Rank Test](#) (for dependent/paired samples, `stats.wilcoxon()`).

Below, there is sample code to run three different statistical tests. **You should use *only* the one that is most appropriate for your data by uncommenting that line.**

```
print(stats.ttest_ind(data_1,data_2)) # to run an independent t-test
# print(stats.ttest_rel(data_1,data_2)) # to run an dependent t-test
# print(stats.mannwhitneyu(data_1,_2)) # to run a mannwhitneyu
# print(stats.wilcoxon(data_1,data_2)) # to run a wilcoxon signed rank test
```

```
TtestResult(statistic=np.float64(-2.4382276613229465), pvalue=np.float64(0.026796307,
```

That's it for this notebook! You can adapt this code for lots of different projects (including your final project!).

Analyze EEG data

This notebook will help you prepare the figures for your EEG lab report.

To run this notebook:

1. Export two separate **LabChart Text File (.txt)** of 30 seconds of data for *both* eyes open and closed.
2. Upload the file to Colab.
3. Change the filename below to match the name of your file.

```

import numpy as np

# Change the filenames to EXACTLY match your files
closed_filename = 'EEG_Trial1_EyesClosed.txt'
open_filename = 'EEG_Trial1_EyesOpen.txt'

sampling_freq = 400

# Define column names
columns = ['time', 'recording']

# Use numpy genfromtxt to import both files
closed_data = np.genfromtxt(closed_filename, dtype=float, usecols=(0,1), skip_header=6)
open_data = np.genfromtxt(open_filename, dtype=float, usecols=(0,1), skip_header=6,

# Define a function to remove nan values
def nan_helper(y):
    return np.isnan(y), lambda z: z.nonzero()[0]

# Save recording & timestamps as variables
closed_timestamps = closed_data['time']
closed_recording = closed_data['recording']
nans, x = nan_helper(closed_recording)
closed_recording[nans] = np.interp(x(nans), x(~nans), closed_recording[~nans])

open_timestamps = open_data['time']
open_recording = open_data['recording']
nans, x = nan_helper(open_recording)
open_recording[nans] = np.interp(x(nans), x(~nans), open_recording[~nans])

closed_recording

```

```

array([-125.95 , -124.95 , -139.2875 , ...,  69.21875,  63.8625 ,
        59.76875], shape=(12140,))

```

Plot the data

Now, we can plot the data!

Notes:

- You may want to change the x and y labels below.
- You can also uncomment the line that says `plt.xlim` to change how much of the data is plotted.
- You can use this same cell to plot your eyes open data by changing `closed_timestamps` to `open_timestamps` and `closed_recording` to `open_recording`.

```

# Import plotting packages
import matplotlib.pyplot as plt
%matplotlib inline
%config InlineBackend.figure_format = 'retina'

# Set up figure
fig, ax = plt.subplots(figsize=(10,4))

# Change the variables below if you'd like to plot eyes open instead
plt.plot(closed_timestamps, closed_recording)

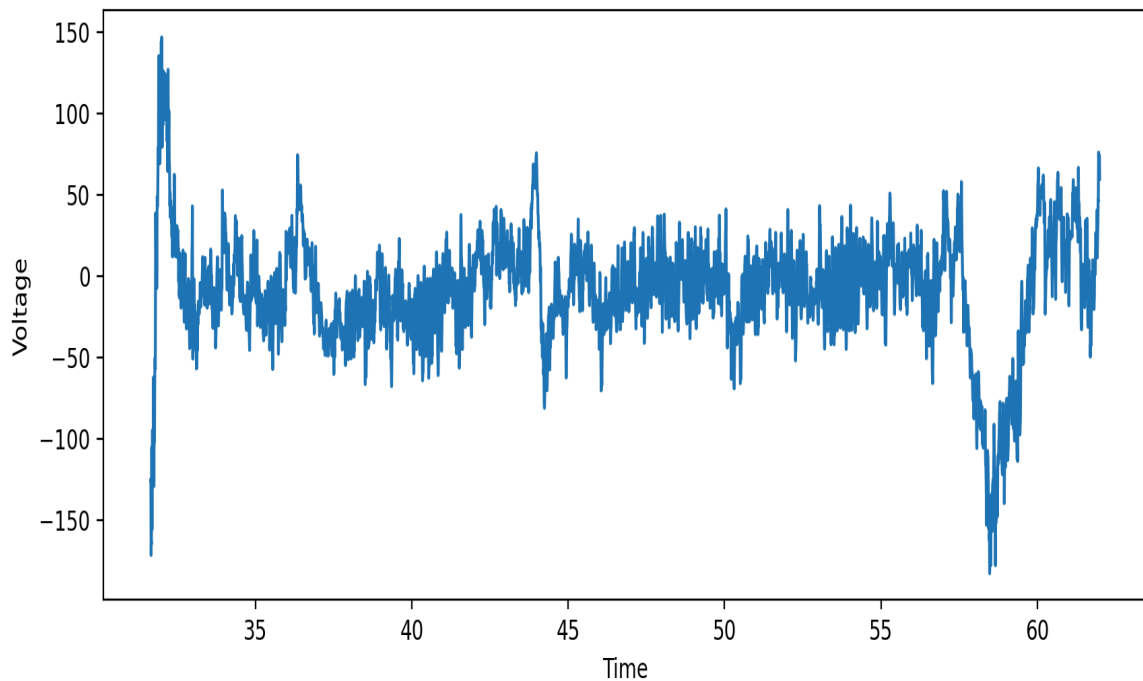
# You may need to change the x label!
plt.xlabel('Time')

# You may need to change the y label!
plt.ylabel('Voltage')

# You can uncomment the line below to restrict the x axis plotting -- for example, t
# plt.xlim([45,50])

plt.show()

```



Calculate the power spectrum of our EEG data

Below, we'll implement a **fast Fourier transform** (fft) to see the frequencies in our data.

```

# Calculate the fourier transform of complex signal
ft = np.fft.fft(closed_recording)/len(closed_timestamps) # Compute the fft, normaliz

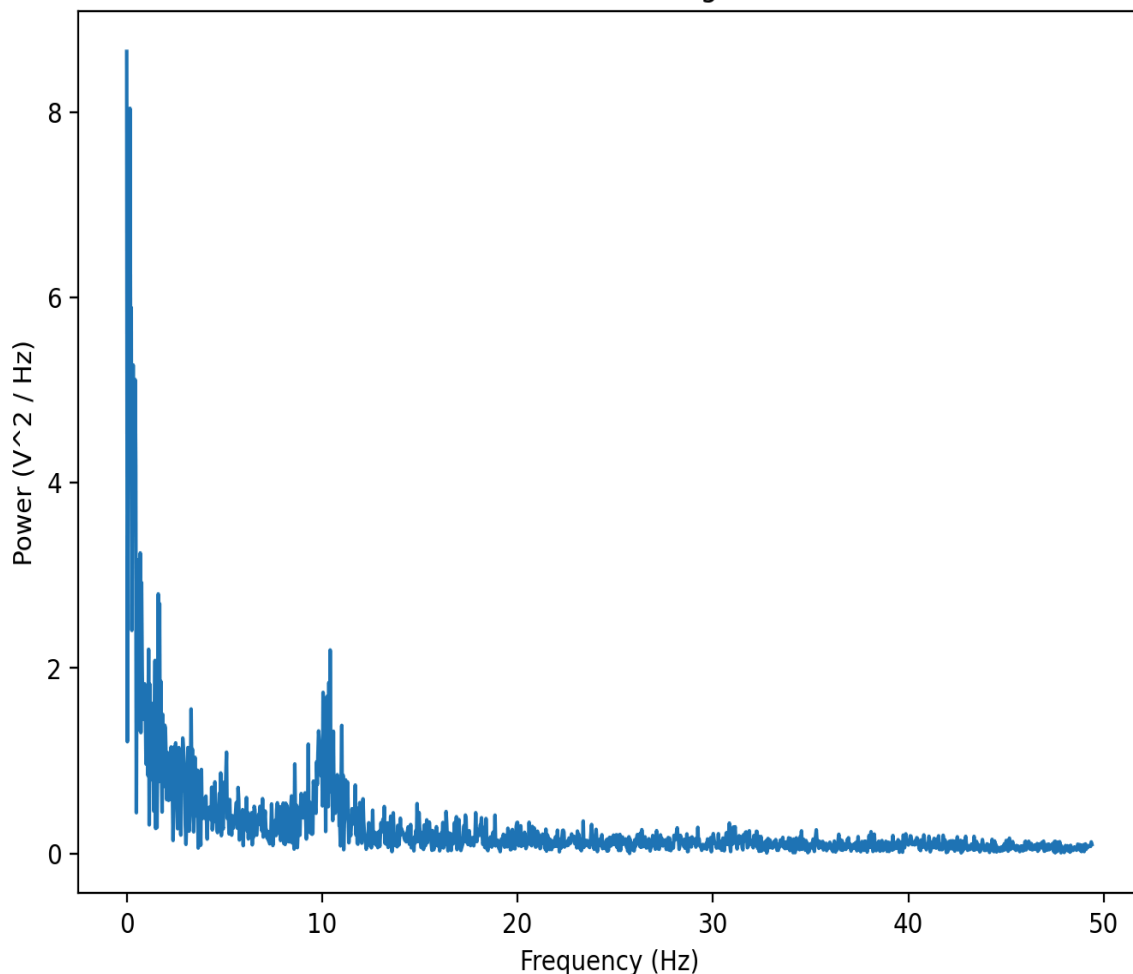
# Find frequency values for the x axis
nyq = sampling_freq/2 # Determine the nyquist frequency

# Create freq bins for plotting by creating a vector from 0 to nyquist, with as many
fx_bins = np.linspace(0,nyq,int(np.floor(len(closed_recording)/2))+1)

# plotting up to 200 Hz
plt.figure(figsize=(8, 6))
plt.plot(fx_bins[0:1500],abs(ft[0:1500]))
plt.ylabel('Power (V^2 / Hz)')
plt.xlabel('Frequency (Hz)')
plt.title('FFT of our EEG signal')
plt.show()

```

FFT of our EEG signal



Hmm, this is interesting but a bit noisy. Maybe we need a method that is *better* than the simple fast fourier transform for this type of data. Thankfully, there's a way to smooth out our fft without losing too much information.

The most widely-used method to do that is [Welch's Method](#), which consists in averaging consecutive Fourier transform of small windows of the signal, with or without overlapping. Basically, we calculate the fft of a signal across a few sliding windows, and then calculate the

mean PSD from all the sliding windows. This is how LabChart generates the **power spectral density** plot as well!

The `freqs` vector contains the x-axis (frequency bins) and the `psd` vector contains the y-axis (power spectral density). The units of the power spectral density here are V^2 per Hz, reflecting that these values are for a particular frequency range.

Notes:

- Check that the values here are *almost identical* to your PSD in LabChart. If not, you may need to change the y-axis labels.
- If you'd like to plot both the eyes opened and closed data, you can uncomment the three lines indicated below.
- You can change the x limit by uncommenting & editing the `plt.xlim` line.

```
from scipy import signal

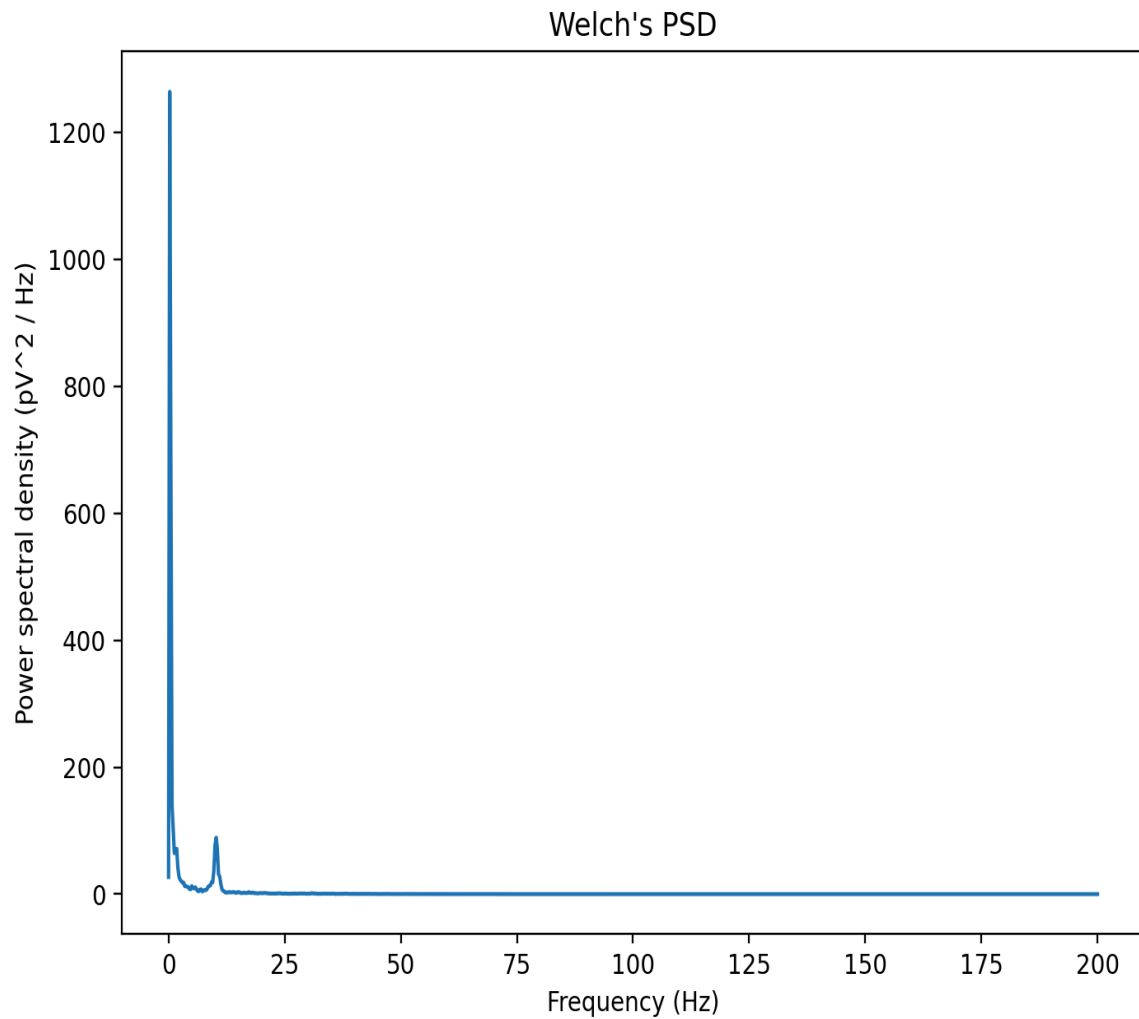
# Define sliding window length (4 seconds, which will give us 2 full cycles at 0.5 Hz)
win = 4 * sampling_freq
freqs, psd = signal.welch(closed_recording, sampling_freq, nperseg=win)

# Plot the power spectrum
plt.figure(figsize=(8, 6))
plt.plot(freqs, psd)

# Uncomment the lines below to plot two lines
#open_freqs, open_psd = signal.welch(open_recording, sampling_freq, nperseg=win)
#plt.plot(open_freqs, open_psd)
#plt.legend(['Closed', 'Open'])

plt.xlabel('Frequency (Hz)')
plt.ylabel('Power spectral density (pV2 / Hz)') # Check that these units make sense
#plt.xlim([0,75]) # Change x limit

plt.title("Welch's PSD")
plt.show()
```



Generate & plot spectrogram

As a final step, we can plot a **spectrogram** of our signal. This is a common way to visualize EEG signals. The spectrogram results from doing the FFT on the snippet of the signal that falls into a "window," plotting the frequency content in the window, then moving the window in time and plotting the frequency content again (and again) until the window has moved across the entire signal.

```

vmax = 20 # change the max value on your spectrogram -- you may need to adjust this

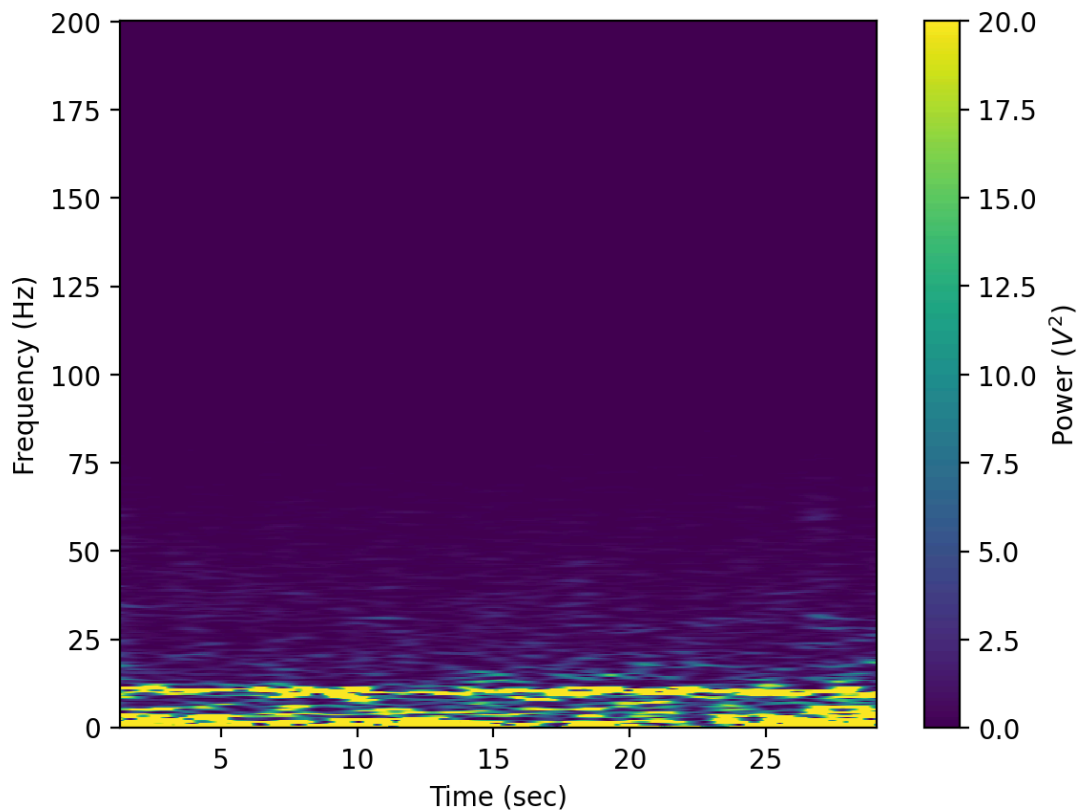
wind = np.hanning(1024) # Create a "hanning" window with a given binning window size

# Create the spectrogram and plot
fig = plt.figure()

# Change closed to open below to generate for open
f, tt, Sxx = signal.spectrogram(closed_recording,sampling_freq,wind,len(wind),len(wi

# You can change cmap & vmax if you want
plt.pcolormesh(tt,f,Sxx,cmap='viridis',vmax=vmax)
plt.ylabel('Frequency (Hz)')
plt.xlabel('Time (sec)')
# plt.ylim([0,75]) # set the ylimit
cbar = plt.colorbar()
cbar.ax.set_ylabel('Power (V2)')
plt.show()

```



Final Projects

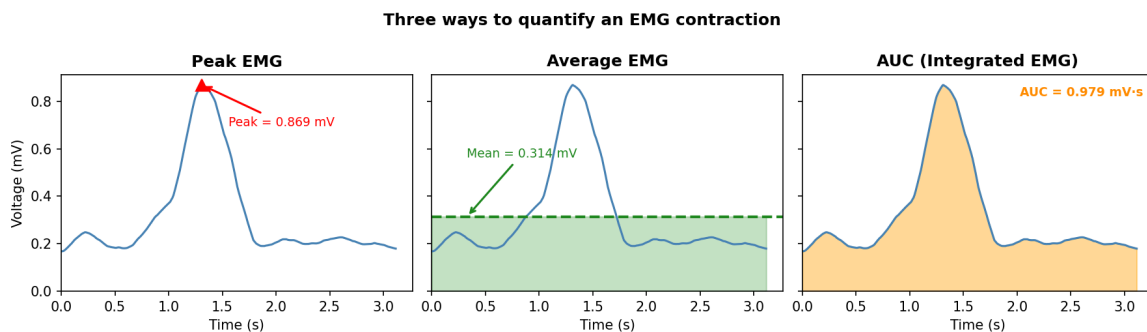
The notebooks in this section are used for final project data analysis.

Analyze EMG data

An **electromyogram (EMG)** measures the electrical activity produced by skeletal muscles. In this notebook, we will analyze the **root mean square (RMS)** of a biceps EMG signal recorded during a bicep curl. RMS processing smooths the raw EMG into a continuous envelope that reflects the overall level of muscle activation over time.

Analysis Choices

When analyzing EMG data, we can decide to measure the peak EMG, the average EMG, or the area under the EMG, often called the “integrated” EMG signal. See Figure 54 in [this guide](#) for an explanation of different quantification methods.



If you'd prefer to run this analysis in LabChart, you can do so following [these instructions](#). The code below will allow you to analyze it in Python!

Python Analysis

Before you begin:

1. Export a **LabChart Text File (.txt)** of your EMG recording.
2. Upload the file to Colab (or place it in the same folder as this notebook).
3. Change the `filename` variable in the cell below to exactly match your file name.


```

import numpy as np
import matplotlib.pyplot as plt
from scipy import signal

# Change the filename to EXACTLY match your file
filename = 'baseline_1.txt'

# Define column names
columns = ['time', 'recording']

# Load the data (skip the 6-line LabChart header)
data = np.genfromtxt(filename, dtype=float, usecols=(0, 1), skip_header=6,
                    delimiter='\t', names=columns, encoding='unicode_escape')

recording = data['recording'] # RMS voltage in mV

# LabChart text files can contain a second header block mid-recording, which
# injects NaN values and one very large spurious number. Mark anything outside
# a plausible voltage range as NaN, then interpolate across all NaN values.
recording[recording > 10.0] = np.nan
nans = np.isnan(recording)
x = np.arange(len(recording))
recording[nans] = np.interp(x[nans], x[~nans], recording[~nans])

# Build a continuous timestamp array from the sampling interval
interval = 0.005 # seconds (from the LabChart header: Interval = 0.005 s)
timestamps = np.arange(len(recording)) * interval

print(f'Loaded {len(timestamps)} samples | duration: {timestamps[-1]:.2f} s | interval: {interval:.2f} s')

```

Loaded 15466 samples | duration: 77.33 s | interval: 0.005 s

Step 1: Plot the entire dataset

Let's start by plotting the full RMS EMG trace. Each upward deflection corresponds to a moment of increased biceps muscle activity — in this case, one bicep curl. The baseline (resting) voltage is the low, flat region between curls.

```

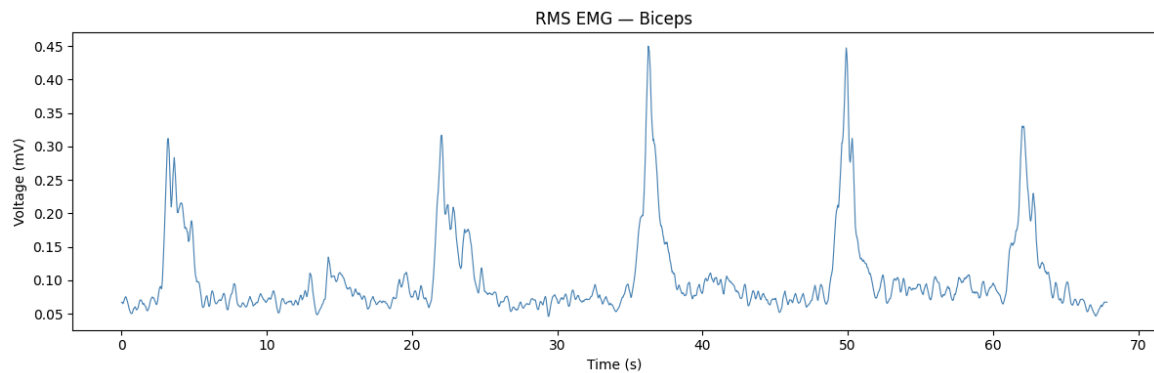
fig, ax = plt.subplots(figsize=(12, 4))

ax.plot(timestamps, recording, linewidth=0.8, color='steelblue')

ax.set_xlabel('Time (s)')
ax.set_ylabel('Voltage (mV)')
ax.set_title('RMS EMG – Biceps')

plt.tight_layout()
plt.show()

```



Step 2: Detect bicep curl activations

Next, we'll automatically detect each bicep curl by finding **peaks** in the RMS signal using `scipy.signal.find_peaks`, then clip out the full activation burst — from the local minimum **before** each peak to the local minimum **after** it.

Three key parameters control detection:

- `height_fraction` — peaks must exceed this fraction of the recording's maximum voltage (e.g., `0.6` means 60% of max). This adapts automatically to different signal amplitudes.
- `prominence_threshold` — peaks must stand out from the surrounding signal by at least this much (mV).
- `min_peak_distance` — if two peaks are closer than this many samples, only the taller one is kept.

Adjust if needed: If too many or too few curls are detected, try increasing or decreasing `height_fraction` in the cell below.

```

# --- Tunable parameters ---
height_fraction      = 0.6      # peaks must exceed this fraction of the recording's m
prominence_threshold = 0.03     # minimum peak prominence in mV
search_window        = 400      # samples to search before/after each peak (~2 s at 20
min_peak_distance    = 400      # if two peaks are within this many samples, keep only

# Adaptive threshold – scales automatically with signal amplitude
height_threshold = height_fraction * recording.max()
print(f'Peak threshold: {height_threshold:.4f} mV ({height_fraction} × max)')

# Find peaks
peaks, _ = signal.find_peaks(recording,
                             height=height_threshold,
                             prominence=prominence_threshold)

# Remove nearby duplicate peaks – greedily keep the tallest within min_peak_distance
sorted_by_height = peaks[np.argsort(recording[peaks])[:, -1]]
kept = []
for p in sorted_by_height:
    if all(abs(p - k) >= min_peak_distance for k in kept):
        kept.append(p)
peaks = np.array(sorted(kept)) # restore time order

print(f'Detected {len(peaks)} peak(s) at times: {np.round(timestamps[peaks], 2)} s')

# For each peak, find the full activation burst: local min before → local min after
bursts = []
for peak_idx in peaks:
    pre_start = max(0, peak_idx - search_window)
    start_idx = pre_start + np.argmin(recording[pre_start:peak_idx])

    post_end = min(len(recording), peak_idx + search_window)
    end_idx = peak_idx + np.argmin(recording[peak_idx:post_end])

    bursts.append((start_idx, end_idx))

# Standardize burst length – re-extract each burst centered on its peak using
# a half-width equal to the largest half-width seen across all bursts
pre_widths = [peak - start for peak, (start, _) in zip(peaks, bursts)]
post_widths = [end - peak for peak, (_, end) in zip(peaks, bursts)]
half_width = max(max(pre_widths), max(post_widths))

standardized_bursts = []
for peak_idx in peaks:
    start_idx = max(0, peak_idx - half_width)
    end_idx = min(len(recording) - 1, peak_idx + half_width)
    standardized_bursts.append((start_idx, end_idx))

print(f'Standardized burst duration: {2 * half_width * interval:.2f} s ({2 * half_wi

```

Peak threshold: 0.5212 mV (0.6 × max)
 Detected 7 peak(s) at times: [3.98 15.36 29.14 45.7 53.3 62.2 70.46] s
 Standardized burst duration: 3.61 s (722 samples)

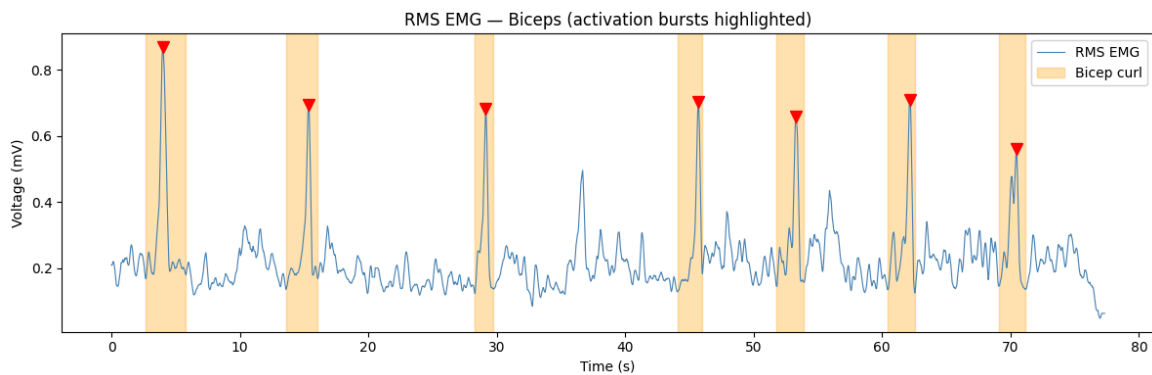
```
# --- Plot 1: Full trace with activation bursts highlighted ---
fig, ax = plt.subplots(figsize=(12, 4))

ax.plot(timestamps, recording, linewidth=0.8, color='steelblue', label='RMS EMG')

for i, (start, end) in enumerate(bursts):
    label = 'Bicep curl' if i == 0 else '_nolegend_'
    ax.axvspan(timestamps[start], timestamps[end], alpha=0.3, color='orange', label=label)
    ax.plot(timestamps[peaks[i]], recording[peaks[i]], 'rv', markersize=8) # peak m

ax.set_xlabel('Time (s)')
ax.set_ylabel('Voltage (mV)')
ax.set_title('RMS EMG — Biceps (activation bursts highlighted)')
ax.legend()

plt.tight_layout()
plt.show()
```



```
# --- Plot 2: Each activation burst displayed individually (standardized length) ---
n = len(standardized_bursts)

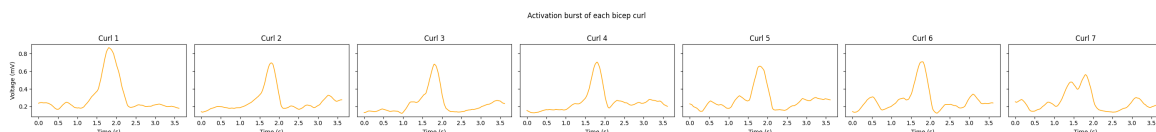
fig, axes = plt.subplots(1, n, figsize=(4 * n, 3), sharey=True)
if n == 1:
    axes = [axes]

for i, (start, end) in enumerate(standardized_bursts):
    t_seg = timestamps[start:end + 1] - timestamps[start] # zero-align time
    v_seg = recording[start:end + 1]

    axes[i].plot(t_seg, v_seg, color='orange', linewidth=1.2)
    axes[i].set_xlabel('Time (s)')
    axes[i].set_title(f'Curl {i + 1}')

axes[0].set_ylabel('Voltage (mV)')
plt.suptitle('Activation burst of each bicep curl', y=1.02)

plt.tight_layout()
plt.show()
```



Step 3: Calculate average EMG during each burst

The **average EMG** (mean voltage during a contraction) measures the typical activation intensity during a burst. Unlike the peak, it's less sensitive to single-sample noise. Unlike the AUC, it's independent of burst duration — making it useful for comparing contractions of different lengths. Below, we'll calculate the mean RMS voltage within each standardized burst.

```
# Calculate mean (average) EMG for each standardized burst
means = []
for start, end in standardized_bursts:
    v_seg = recording[start:end + 1]
    means.append(np.mean(v_seg))

# Print results
for i, mean_val in enumerate(means):
    print(f' Curl {i + 1}: Mean EMG = {mean_val:.4f} mV')
print(f'\nMean across all curls: {np.mean(means):.4f} mV')
print(f'Standard deviation:      {np.std(means):.4f} mV')
```

```
Curl 1: Mean EMG = 0.3014 mV
Curl 2: Mean EMG = 0.2580 mV
Curl 3: Mean EMG = 0.2291 mV
Curl 4: Mean EMG = 0.2547 mV
Curl 5: Mean EMG = 0.2728 mV
Curl 6: Mean EMG = 0.2724 mV
Curl 7: Mean EMG = 0.2569 mV
```

```
Mean across all curls: 0.2636 mV
Standard deviation:    0.0205 mV
```

Step 4: Calculate the area under each burst

The **area under the curve (AUC)** of an RMS EMG burst measures total muscle activation — it reflects both how strongly *and* how long the muscle was recruited during each curl. Sometimes this is called the “integrated EMG.”

Below, we'll calculate the area using the **trapezoidal rule** (`np.trapezoid`), which approximates the area by summing thin trapezoids under the signal. The result is in units of **mV·s** (millivolt-seconds).

```
# Calculate AUC for each standardized burst using the trapezoidal rule
areas = []
for start, end in standardized_bursts:
    t_seg = timestamps[start:end + 1] - timestamps[start]
    v_seg = recording[start:end + 1]
    areas.append(np.trapezoid(v_seg, t_seg))

# Print results
for i, area in enumerate(areas):
    print(f' Curl {i + 1}: AUC = {area:.4f} mV·s')
print(f'\nMean AUC: {np.mean(areas):.4f} mV·s')
print(f'Standard Deviation of AUC: {np.std(areas):.4f} mV·s')
```

```
Curl 1: AUC = 1.0886 mV·s
Curl 2: AUC = 0.9315 mV·s
Curl 3: AUC = 0.8273 mV·s
Curl 4: AUC = 0.9197 mV·s
Curl 5: AUC = 0.9849 mV·s
Curl 6: AUC = 0.9837 mV·s
Curl 7: AUC = 0.9276 mV·s
```

```
Mean AUC: 0.9519 mV·s
Standard Deviation of AUC: 0.0740 mV·s
```

Compute average PSD

This notebook builds on the EEG analysis in lab to compute an average PSD for different conditions.

Step 1

Export all of the **LabChart Text Files (.txt)** for your conditions separately. Then, run the cell below which will create as many folders for you as you have conditions.

```
# Rename the conditions here!
conditions = ['Control', 'Treatment']

for condition in conditions:
    !mkdir -p {condition}

print("Directories created for conditions: " + ", ".join(conditions))
```

```
Directories created for conditions: Control, Treatment
```

Step 2

Upload your exported .txt files into their respective folders.

```
import numpy as np
import matplotlib.pyplot as plt
from scipy import signal
import glob

filenames = {}

for condition in conditions:
    print(f"Files for {condition}:")
    filenames[condition] = glob.glob(f'{condition}/*.txt')
    print(filenames[condition])
```

```
Files for Control:
['Control/eyes open .txt', 'Control/EEG_Trial1_EyesOpen.txt', 'Control/EEG Eyes Open
Files for Treatment:
['Treatment/eyes closed.txt', 'Treatment/EEG_Trial1_EyesClosed.txt', 'Treatment/EEG I
```

Step 3

Run the cell below to define several helper functions. You should also change the sampling frequency if you changed it for some reason.

```
# Change this if it was different for your experiment!
sampling_freq = 400
nperseg = 800 # Window length in samples (2 seconds at 400 Hz)

def generate_avg_psd(filenamees):
    if len(filenamees) == 0:
        return np.array([]), np.array([]), np.array([])

    psd_list = []
    freqs = None

    for file in filenamees:
        print(file)
        columns = ['time', 'recording']
        data = np.genfromtxt(file, dtype=float, usecols=(0,1), skip_header=6, delimiter=

        recording = data['recording']

        # Interpolate NaN values
        nans = np.isnan(recording)
        x = np.arange(len(recording))
        recording[nans] = np.interp(x[nans], x[~nans], recording[~nans])

        freqs, psd_single = signal.welch(recording, sampling_freq, nperseg=nperseg)
        psd_list.append(psd_single)

    # Interpolate all PSDs to same length
    max_len = max(len(p) for p in psd_list)
    psd_matrix = np.zeros((max_len, len(filenamees)))

    for i, psd in enumerate(psd_list):
        if len(psd) < max_len:
            psd = np.interp(np.linspace(0, 1, max_len), np.linspace(0, 1, len(psd)), psd)
            psd_matrix[:, i] = psd

    average_psd = np.mean(psd_matrix, axis=1)
    std_psd = np.std(psd_matrix, axis=1) / np.sqrt(len(filenamees))

    return average_psd, std_psd, freqs

def plot_psd(average_psd, std_psd, freqs, label=None):
    plt.semilogy(freqs, average_psd, label=label)
    plt.fill_between(freqs, average_psd - std_psd, average_psd + std_psd, alpha=0.3)

    plt.xlabel('Frequency (Hz)')
    plt.ylabel('Power spectral density ( $\mu V^2$  / Hz)')

    max_freq = 75
    plt.xlim([0, max_freq])

    visible_mask = freqs <= max_freq
    ymin = min((average_psd - std_psd)[visible_mask])
    ymax = max((average_psd + std_psd)[visible_mask])
    plt.ylim(ymin, ymax)
```

Step 4

Run the cell below to use the functions to calculate your average PSD and then plot for all of your conditions.

```
average_psd = {}
std_psd = {}

plt.figure(figsize=(8, 6))

for condition in conditions:
    average_psd[condition], std_psd[condition], freqs = generate_avg_psd(filenamees[c
    plot_psd(average_psd[condition],std_psd[condition],freqs, label=condition)

plt.legend()
plt.show()
```

Control/eyes open .txt
Control/EEG_Trial1_EyesOpen.txt
Control/EEG Eyes Open.txt
Treatment/eyes closed.txt
Treatment/EEG_Trial1_EyesClosed.txt
Treatment/EEG Eyes Closed.txt

